

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



TESIS DE GRADO

**“DETECTOR DE CÁNCER DE PIEL A TRAVÉS DE UNA
FOTOGRAFÍA UTILIZANDO VISIÓN POR
COMPUTADORA”**

PARA OPTAR AL TÍTULO DE LICENCIATURA EN INFORMÁTICA MENCIÓN
INGENIERÍA DE SISTEMAS INFORMÁTICOS

POR: MIGUEL FERNANDO APAZA COLQUE

TUTOR: M.Sc. REYNALDO ZEBALLOS DAZA

LA PAZ – BOLIVIA
2023

HOJA DE CALIFICACIONES

UNIVERSIDAD MAYOR DE SAN ANDRÉS

FACULTAD DE CIENCIAS PURAS Y NATURALES

CARRERA DE INFORMÁTICA

Tesis de grado

“DETECTOR DE CÁNCER DE PIEL A TRAVÉS DE UNA FOTOGRAFÍA UTILIZANDO VISIÓN POR COMPUTADORA”

Presentado por: Apaza Colque Miguel Fernando

Para optar el grado Académico de Licenciado en Informática

Mención Ingeniería de Sistemas Informáticos

Nota Numeral:

Nota Literal:

Ha sido:

Director de la carrera de Informática: Ph.D. José María Tapia Baltazar

Tutor: Lic. ZEBALLOS DAZA REYNALDO JAVIER

Tribunal: Lic. VARGAS BLACUTT ROBERTO

Tribunal: Ph.D. REYES PACHECO JAVIER HUGO GUSTAVO

Tribunal: Ph.D. TELLEZ RAMIREZ MARISOL



**UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA**



LA CARRERA DE INFORMÁTICA DE LA FACULTAD DE CIENCIAS PURAS Y NATURALES PERTENECIENTE A LA UNIVERSIDAD MAYOR DE SAN ANDRÉS AUTORIZA EL USO DE LA INFORMACIÓN CONTENIDA EN ESTE DOCUMENTO SI LOS PROPÓSITOS SON Estrictamente Académicos.

LICENCIA DE USO

El usuario está autorizado a:

- a) visualizar el documento mediante el uso de un ordenador o dispositivo móvil.
- b) copiar, almacenar o imprimir si ha de ser de uso exclusivamente personal y privado.
- c) copiar textualmente parte(s) de su contenido **mencionando la fuente** y/o haciendo la referencia correspondiente respetando normas de redacción e investigación.

El usuario no puede publicar, distribuir o realizar emisión o exhibición alguna de este material, sin la autorización correspondiente.

TODOS LOS DERECHOS RESERVADOS. EL USO NO AUTORIZADO DE LOS CONTENIDOS PUBLICADOS EN ESTE SITIO DERIVARÁ EN EL INICIO DE ACCIONES LEGALES CONTEMPLADOS EN LA LEY DE DERECHOS DE AUTOR

Dedicatoria

Deseo dedicar este trabajo primeramente a Dios, por otorgarme los conocimientos y la inteligencia necesaria para comprender y resolver obstáculos en la vida, también por las oportunidades académicas que se me brindó para realizar este trabajo de investigación. Seguidamente dedico este trabajo a todos aquellos que han sido una parte integral de mi camino personal y académico, en este punto hago referencia a mi familia (por su amor incondicional y por apoyarme con todos los recursos necesarios para salir adelante) y a mis tutores académicos por su dedicación (M Sc. Rosa Flores Morales) y su pasión por la enseñanza (M Sc. Reynaldo Javier Zeballos Daza). Finalmente dedico este trabajo a todos los futuros lectores que harán referencia de mi investigación para mejorar o salvar las vidas de las personas mediante las nuevas tecnologías.

“Acepta las cosas que no puedes cambiar, ten el valor de cambiar las cosas que si puedes y ten la sabiduría para conocer la diferencia”

mifacfernando4@gmail.com

Resumen

Este prototipo se desarrolló con el fin de crear una segunda opinión respecto al diagnóstico del médico dermatólogo con un posible caso de melanoma en un paciente obviando las pruebas tradicionales de detección del melanoma como los Rayos X, pruebas dermatoscópicas con lámparas de Wood y biopsias innecesarias.

Para el entrenamiento del prototipo se utilizó un conjunto enorme de diagnósticos (17000 imágenes) las cuales se almacenan y llegan a denominarse *dataset*.

Luego este *dataset* pasa por un proceso de convolución, en donde se obtienen las características más relevantes de los objetos presentes en cada uno de los diagnósticos (como forma, color, contorno y demás).

Posteriormente estas características pasaran por un proceso de clasificación mediante una red neuronal profunda (pues contiene más de 1 capa oculta), en donde tendremos definidos los conjuntos de salida como valores enteros que representarían: 1 (melanoma) y 0 (carcinoma).

Para demostrar que el método de redes neuronales es óptimo para resolver mi problema de clasificación, se utilizó la validación cruzada K-Cross, en la cual obtuvimos un 7% de desviación estándar entre cada porción de datos de prueba.

Luego se realizaron los ajustes de refinamiento al modelo con el fin de mejorar el porcentaje de asertividad en el pronóstico, se calibro y asigno el valor del learning rate a 0.0008, seguidamente se realizaron pruebas de optimización con los métodos de: DCG, MKSProp y ADAM, quedándonos con este último como el optimizador de mi prototipo, esto porque nos ofrece llegar a un nivel de asertividad alto con un número de épocas menor a los otros modelos.

Al final concluimos con un nivel de asertividad de más del 80% en el pronóstico de un posible melanoma, para verificar este resultado se utilizó el método de las curvas de ROC y precisión-sensibilidad, finalmente concluimos con las recomendaciones y conclusiones del proyecto.

Palabras clave: inteligencia artificial, detector de melanomas, redes convolucionales, redes neuronales, visión artificial.

Metodología: Buchanan para el desarrollo teórico y Kanban para el práctico.

Summary

This prototype was developed in order to create a second opinion regarding the diagnosis of the dermatologist with a possible case of melanoma in a patient bypassing traditional melanoma detection tests such as X-rays, dermoscopic tests with Wood's lamps and unnecessary biopsies.

For the training of the prototype a huge set of diagnostics (17000 images) was used, which are stored and come to be called *dataset*.

Then this *dataset* goes through a convolution process, where the most relevant characteristics of the objects present in each of the diagnoses (such as shape, color, contour and others) are obtained.

Later these characteristics will go through a classification process through a deep neural network (because it contains more than 1 hidden layer), where we will have defined the output sets as integer values that would represent: 1 (melanoma) and 0 (carcinoma).

To demonstrate that the neural network method is optimal for solving my classification problem, K-Cross cross-validation was used, in which we obtained a 7% standard deviation between each portion of test data.

Then the refinement adjustments were made to the model in order to improve the percentage of assertiveness in the forecast, the learning rate value was calibrated and assigned to 0.0008, then optimization tests were carried out with the methods of: DCG, MKSProp and ADAM, leaving us with the latter as the optimizer of my prototype, this is because it offers us to reach a high level of assertiveness with a lower number of eras than the other models.

In the end we concluded with a level of assertiveness of more than 80% in the prognosis of a possible melanoma, to verify this result the method of ROC curves and precision-sensitivity was used, finally we concluded with the recommendations and conclusions of the project.

Keywords: artificial intelligence, melanoma detector, convolutional networks, neural networks, computer vision.

Methodology: Buchanan for theoretical development and Kanban for practical development.

ÍNDICE

CAPÍTULO 1: MARCO INTRODUCTORIO	15
1.1. INTRODUCCIÓN.....	15
1.2. ANTECEDENTES	16
1.3. DESCRIPCIÓN DEL PROBLEMA	17
1.4. HIPÓTESIS.....	19
1.5. OBJETIVO.....	19
1.5.1. OBJETIVO GENERAL.....	19
1.5.2. OBJETIVOS ESPECÍFICOS.....	20
1.6. JUSTIFICACIÓN	20
1.6.1. JUSTIFICACIÓN APLICATIVO/PRACTICO	20
1.6.2. JUSTIFICACIÓN METODOLÓGICA	20
1.6.3. JUSTIFICACIÓN TEÓRICA	21
1.7. ALCANCES Y LIMITES	21
1.7.1. ALCANCES.....	21
1.7.2. LIMITES.....	22
1.8. DISEÑO METODOLÓGICO.....	22
CAPÍTULO 2: MARCO TEÓRICO	24
2.1.1 EL MACHINE LEARNING	25
2.1.2 SISTEMAS CONEXIONISTAS	27
2.1.2.1 EL PERCEPTRÓN	28
2.1.2.2 LA RED NEURONAL SIMPLE	29
2.1.2.3 LA FUNCIÓN DE ACTIVACIÓN	31
2.1.2.4 LA RED NEURONAL CONVOLUCIONAL	34
2.1.2.5 LA CONVOLUCIÓN	35
2.1.2.6 EL FILTRO	38
2.1.3 VISIÓN POR COMPUTADOR	42
2.1.3.1 COMO FUNCIONA LA VISIÓN POR COMPUTADOR	43
2.1.3.2 EJEMPLOS APLICATIVOS UTILIZANDO LA VISIÓN POR COMPUTADOR.....	44

2.1.4	VALIDACIÓN CRUZADA K-CROSS.....	45
2.1.5	MÉTODOS DE OPTIMIZACIÓN.....	46
2.1.6	LA VARIABLE DEL LEARNING RATE.....	48
2.1.7	MÉTODOS DE EVALUACIÓN PARA MI MODELO	49
2.1.7.1	EVALUACIÓN DEL MODELO POR MEDIO DE LA CURVA ROC.....	50
2.1.7.2	EVALUACIÓN DEL MODELO POR MEDIO DE LA PRECISIÓN-SENSIBILIDAD	52
2.2	CÁNCER DE PIEL	52
2.2.1	EL ORIGEN DEL MELANOMA.....	53
2.2.2	FACTORES QUE AUMENTAN EL RIESGO DE PADECER MELANOMA	55
2.2.3	CÓMO SE DIAGNOSTICA EL MELANOMA.....	55
2.2.4	TRATAMIENTO ANTE EL MELANOMA.....	56
2.3.	ESTADO DEL ARTE	57
2.3.1.	SISTEMA DE CLASIFICACIÓN DE IMÁGENES DE LUNARES UTILIZANDO VISIÓN POR COMPUTADORA.....	58
2.3.2.	DETECCIÓN DE MELANOMAS DE PIEL MALIGNOS MEDIANTE PROCESAMIENTO DIGITAL DE IMÁGENES USANDO REDES NEURONALES CONVOLUCIONALES.....	59
CAPÍTULO 3: MARCO APLICATIVO		62
3.1.	FASE 1: IDENTIFICACIÓN	62
3.1.1.	IDENTIFICACIÓN DEL PROBLEMA.....	63
3.1.2.	BÚSQUEDA DE UN EXPERTO	63
3.1.3.	IDENTIFICACIÓN DE COMPONENTES	63
3.1.4.	IDENTIFICACIÓN DE RECURSOS.....	64
3.1.5.	IDENTIFICACIÓN DE LAS METAS.....	65
3.2.	FASE 2: CONCEPTUALIZACIÓN	65
3.3.	FASE 3: FORMALIZACIÓN	68
3.4.	FASE 4: IMPLEMENTACIÓN	71
3.4.1.	TAREA 1: ADQUISICIÓN.....	71
3.4.2.	TAREA 2: HERRAMIENTAS PARA EL DESARROLLO DEL SISTEMA	72
3.4.3.	TAREA 3: PREPROCESAMIENTO	74
3.4.3.1.	REDIMENSIONANDO EL DATASET	75

3.4.3.2.	RENOMBRAMIENTO DE LAS IMÁGENES	76
3.4.3.3.	DISTRIBUCIÓN DE LAS IMÁGENES.....	76
3.4.4.	TAREA 4: CONVOLUCIÓN	78
3.4.5.	TAREA 5: RED NEURONAL	82
3.4.6.	TAREA 6: DISTRIBUCIÓN Y ENTRENAMIENTO DEL MODELO	83
3.4.7.	TAREA 7: INTERPRETACIÓN DEL RESULTADO	87
3.4.8.	TAREA 8: DESARROLLO DEL ENTORNO GRÁFICO.....	88
3.5.	FASE 5: PRUEBAS DE RENDIMIENTO	93
3.5.1.	IMPLEMENTANDO LA VALIDACIÓN CRUZADA.....	93
3.5.2.	IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN SGD.....	95
3.5.3.	IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN RMSPROP	98
3.5.4.	IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN ADAM	99
3.5.5.	AJUSTE DEL LEARNING RATE DENTRO DEL MÉTODO ADAM	100
CAPÍTULO 4: EVALUACIÓN DE RESULTADOS		105
4.1	RECOLECCIÓN DE LOS DATOS	105
4.2	ANÁLISIS DE LOS DATOS.....	105
4.3	DEMOSTRACIÓN DE LA HIPÓTESIS.....	106
4.3.1	APLICACIÓN DEL MÉTODO DE CURVAS DE ROC DENTRO DEL MODELO	107
4.3.2	APLICACIÓN DEL MÉTODO DE PRECISIÓN-SENSIBILIDAD DENTRO DEL MODELO.	108
4.4	DEMOSTRACIÓN POR CASOS	110
CAPÍTULO 5: CONCLUSIONES Y RECOMENDACIONES		117
5.1	CONCLUSIONES	117
5.2	RECOMENDACIONES.....	118
FUENTES DE LA INFORMACIÓN.		119
ANEXOS.		119
REFERENCIAS BIBLIOGRÁFICAS		127

ÍNDICE DE ILUSTRACIONES

Fig. 1.1. Se muestra el tablero con sus respectivas columnas y filas.....	23
Fig. 2.1. Imágenes de entrada con etiquetas, representan un valor dado.....	26
Fig. 2.2. Imágenes sin etiquetar, solo poseen un valor numérico que las identifica.....	26
Fig. 2.3. Imagen donde se observa los componentes de un Perceptrón.....	29
Fig. 2.4. Imagen donde se describe el ejemplo de la vivienda.	30
Fig. 2.5. Clasificación Lineal de datos	30
Fig. 2.6. Región linealmente no separable.....	31
Fig. 2.7. Una región dentro de otra región.	31
Fig. 2.8. Graficas donde demuestran los rangos de valores numéricos por función.	33
Fig. 2.9. En el lado izquierdo se aplica un kernel de 3x3 a una porción de la imagen original, en la derecha se calcula mediante multiplicaciones y sumas el valor de un nuevo pixel.	35
Fig. 2.10. en la izquierda se muestra la interacción del filtro y la porción de la imagen, en la derecha se muestra el resultado y el proceso de convolución con un filtro de 3x3.	36
Fig. 2.11. Se observa el desenfoque de la imagen original aplicando un filtro de 3x3 con valores de la media.....	36
Fig. 2.12. Mediante la aplicación del filtro explicado, se obtiene los bordes verticales de los objetos enfocados en cualquier imagen.	37
Fig. 2.13. proceso de una capa convolucional respecto a un mapa de características.....	38
Fig. 2.14. En la imagen 1 se observa el filtro de 3x3 (9 pixeles) de color verde, en la imagen 2 se observa que la información del filtro verde se redujo y se aplica otra convolución de 3x3 de color azul, en la imagen 3 se observa la compresión de la información del filtro azul.	39
Fig. 2.15. La imagen 1 es la original, la imagen 2 es el mapa de características de la imagen original, en la imagen 3 se observa el tamaño real de la imagen 2 en comparación a la imagen original y la imagen 4 es otro mapa de características de la imagen 3.	39
Fig. 2.16. Se muestra la estructura de una RNC, donde a la izquierda se observa el embudo convolucional y a la derecha una RNP.	40
Imagen 2.17: A la izquierda se observa la división de los datos en partes iguales, a la derecha vemos la obtención de la media y la desviación estándar respecto de cada partición.....	46
Imagen 2.18: se observa las características de VP, VN, FP y FN separados por medio de un umbral de 0.5.	50

Imagen 2.19: se observa la gráfica de la curva ROC, donde se aprecia diferentes modelos aplicados para resolver un problema (arco azul, naranja, verde), la curva ideal de asertividad 100% (curva morada) y la curva sin entrenamiento (línea roja).	51
Fig. 2.20. Fotografía de un carcinoma.	53
Fig. 2.22. Figura donde se muestra la diferencia entre un melanoma y carcinoma por medio del diagnóstico A, B, C, D, E.	56
Figura 3.1. Se observa una imagen sometida a solo “dos” kernels, el resultado es la obtención del contorno del objeto.	69
Imagen 3.2: se muestra un método donde se redimensiona una imagen a 224x224 pixeles en formato .jpg.	75
Imagen 3.3: se muestra el método donde se renombra cada una de las imágenes con números enteros positivos.....	76
Imagen 3.4: en la imagen se muestra la estructura del proceso de convolución, donde los cuadrados amarillos son los <i>kernels</i>	79
Imagen 3.5: a la izquierda vemos la estructura de la convolución y la red neuronal en forma de código, a la derecha se muestra la estructura del modelo secuencialmente con sus parámetros en cada capa.	83
Imagen 3.6: imagen donde se muestra la creación de las listas de almacenamiento y la distribución de las imágenes.....	84
Imagen 3.7: imagen donde podemos observar la distribución de las imágenes de la carpeta de prueba con sus respectivas etiquetas.....	84
Imagen 3.8: imagen donde podemos observar el método partición.....	85
Imagen 3.9: imagen donde vemos el proceso de aleatorización y partición de las listas.....	85
Imagen 3.10: imagen donde muestra la estructura de los métodos de: “validación” y “evaluación”	86
Imagen 3.11: imagen donde muestra los ajustes de los parámetros mencionados hasta alcanzar un 90% de asertividad, además de la creación de dos listas en donde se almacenarán el nivel de asertividad y el número de épocas de cada entrenamiento.	86
Imagen 3.12: imagen donde vemos el proceso para guardar el modelo entrenado y sus pesos.	87
Imagen 3.13: en estas tres imágenes vemos que el modelo alcanza un nivel de asertividad del 88% en tan solo 44 épocas.....	87
Imagen 3.14: se observa la predicción que tiene el prototipo con una imagen seleccionada de la carpeta test.	88

Imagen 3.15: se observa la ventana de inicio del prototipo.	90
Imagen 3.16: se observa en la ventana la imagen secundaria en BGR, los datos del paciente y los botones para limpiar y capturar la imagen, además los botones deshabilitados de capturar, entrenar y reconocer	91
Imagen 3.17: se observa en la ventana la imagen secundaria seleccionada, el pronóstico en texto claro y los porcentajes de exactitud.	92
Imagen 3.18: se observa la división del <i>dataset</i> de entrenamiento en 5 partes iguales.	94
Imagen 3.19: se observa el vector de porcentajes de asertividad resultante del entrenamiento de cada porción.....	94
Imagen 3.20: se observa el vector de porcentajes de asertividad resultante del entrenamiento de cada porción.....	94
Imagen 3.21: en este código se entrena y prueba el modelo con los parámetros de entrada del método de evaluación.	96
Imagen 3.22: en este fragmento de código se valida si el entrenamiento alcanzo un cierto nivel de asertividad.....	96
Imagen 3.23: en este fragmento de código se hace referencia a la creación y la validación del modelo SGD, además se muestra los resultados de cada época con su respectiva evaluación.	97
Imagen 3.24: en este código se crea y valida el modelo RMSProp, en abajo se observa el resultado después de entrenar 48 épocas y alcanzar el 80% de asertividad.	98
Imagen 3.25: en esta imagen se muestra la creación y validación del modelo ADAM, después de entrenar y evaluar 15 épocas hasta llegar al 80% de asertividad.....	99
Imagen 3.26: en este fragmento de código se muestran vectores que almacenan los valores de prueba para el learning rate	101
Imagen 3.27: en esta imagen se muestra las épocas y los valores del <i>learning rate</i> que alcanzaron a más del 70% de asertividad.	101
Imagen 3.28: podemos observar que no existe un posible candidato con los valores del vector2	102
Imagen 3.29: podemos observar a 5 posibles candidatos para el valor del <i>learning rate</i>	102
Imagen 4.1: en la parte izquierda observamos la gráfica de la curva ROC con valores aceptables, a la derecha el valor del área por debajo de la curva ROC de los modelos que no están entrenados y los que si están entrenados (Red Neuronal).....	107
Imagen 4.2: en la parte izquierda observamos la gráfica de la sensibilidad-precisión, a la derecha el código para graficar el resultado y al final se muestra el área debajo de la curva (auc).	109

Anexo 1: este cuestionario lo completo el doctor Cristian Quiroga, las preguntas sirvieron de retroalimentación para mejorar el prototipo y su visualización en los resultados.	120
Anexo 2: fotografía del consultorio del doctor Cristian Quiroga probando el prototipo detector de melanomas.....	121
Anexo 3: grafica donde se muestra el comportamiento de los 2 métodos en cada época de entrenamiento.	122
Anexo 4: grafica donde se muestra el comportamiento de los 2 métodos en cada época de entrenamiento.	123
Anexo 5: a la izquierda se muestra la estructura que utilice y a la derecha el nivel de comportamiento de cada época respecto al asertividad.	124
Anexo 6: a la izquierda se muestra el nivel de asertividad -épocas con 10000 imágenes y a la derecha con 12500 imágenes.	124
Anexo 7: se muestra los resultados de los entrenamientos, podemos ver que después de llegar al 79% de asertividad esta baja.....	125
Anexo 8: se muestra claramente un descenso en el valor del nivel de asertividad después de llegar a 77%.....	126

ÍNDICE DE TABLAS

Tabla 3.1: Tabla donde se describe los pasos formalizados para la implementación del prototipo.....	71
Tabla 4.1: En la parte izquierda observamos en texto claro la predicción del ejemplo de la derecha (caso melanoma).....	113
Tabla 4.2: en la parte izquierda observamos los resultados del diagnóstico del prototipo y a la derecha el diagnostico (carcinoma).....	115
Tabla 4.3: observamos que los valores de color azul son los casos que se detectaron correctamente y los de color rojo los que no.	116

CAPÍTULO 1: MARCO INTRODUCTORIO

1.1.INTRODUCCIÓN.

Cuando se habla sobre el cáncer se hace referencia a más de 200 clasificaciones distintas, particularmente aparece cuando una célula de nuestro cuerpo muta y esta se va dividiendo sin control. Este tipo de células no termina su ciclo de vida de manera normal, sino que al reproducirse sin control toman el lugar de las células normales.

Generalmente el cáncer se puede diagnosticar en tres fases, las cuales son:

- **Localización temprana**, es decir que puede ser localizado a temprana edad de maduración en el organismo del paciente dentro de un área contenida.
- **Localización avanzada**, es decir que puede ser localizado cuando el cáncer invade los tejidos celulares alrededor del área afectada.
- **Localización extendida**, es cuando el cáncer es detectado infectando el torrente sanguíneo, extendiéndose por todo el organismo y afectando algunos órganos distantes al área de afección original del paciente.

Una de las clasificaciones del cáncer es el melanoma, que es un tipo de cáncer que empieza en los melanocitos (célula que produce el pigmento melanina), este puede surgir como un lunar en la piel, también en otros tejidos pigmentados como los ojos o el intestino (Instituto nacional del cancer, 2020).

Gracias a los avances científicos, se pudieron elaborar métodos de detección temprana de muchos tipos de cáncer incluido el melanoma, como las que utilizan la resonancia magnética, los rayos X, exámenes de sangre u orina (Baptist Health South Florida, 2019), sin embargo, estos métodos de detección son moderadamente costosos y requieren de mucho tiempo en tener una respuesta concreta.

En este presente trabajo de estudio se concentrará en la “localización temprana”, observando las características de la zona de afección del paciente. Para ello solo se

necesitará de herramientas computacionales y la ayuda de la IA (inteligencia artificial) en una de sus ramas: la visión por computadora.

La VC (visión por computadora) es el campo que permite a las computadoras replicar el sistema visual humano, que recopila información de imágenes o videos y los procesa para así poder obtener y definir atributos. Todo este proceso implica la adquisición, selección, análisis, identificación y la extracción de información de imágenes.

En resumen, los proyectos de visión por computadora traducen contenido visual digital en descripciones explícitas para así poder recopilar datos multidimensionales. Estos datos se convierten en un lenguaje legible por computadora para poder ayudar en el proceso de toma de decisiones.

El presente documento, es una nueva propuesta de diagnóstico secundario o de apoyo al profesional de salud al momento de la detección de cáncer de piel o melanoma cutáneo, a través de la inspección visual de la zona afectada del paciente (mancha(s) de piel), utilizando algoritmos de DL (*deep learning*), VC (visión por computador) y RN (redes neuronales).

1.2.ANTECEDENTES.

Para este punto tome la inspiración de varias contribuciones al campo de la medicina utilizando la IA como herramienta, entre estas contribuciones se tiene las siguientes:

- ✓ En septiembre del año 2017 la estudiante Paula Fernández Blázquez con ayuda de sus tutores Amaya Jimenes Moreno y Javier López Labraca de la universidad Carlos III de Madrid, desarrolló un trabajo de fin de grado denominado “Detección del cáncer de piel mediante técnicas de visión artificial”, en la cual trata de detectar un posible caso de cáncer de piel a través de una fotografía tomada de la lesión del paciente, la estudiante utiliza el método de soporte de máquinas vectoriales (SVM) para encontrar un patrón en las irregularidades de los bordes de la lesión del paciente, llegando a la conclusión de que si los bordes llegasen a tener una forma

geométrica similar al de un círculo o un ovalo la lesión solo sea un carcinoma, pero si los bordes se asemejasen a una figura amorfa es probable que la lesión sea un melanoma (uc3m, 2017).

- ✓ En octubre del año 2021 se publica un artículo de investigación a cargo del docente investigador Jairo Eduardo Marques Dias de la universidad de Cundinamarca en Bogota Colombia, denominado “Visión artificial profunda aplicada a la identificación temprana de cáncer no melanoma y queratosis actínica”, en el cual expone el resultado sobre el desarrollo de un sistema de procesamiento digital de imágenes basado en visión artificial profunda, con el fin de diagnosticar un posible caso de cáncer no melanoma (carcinoma), en esta investigación se utilizó la metodología del ABCDE para detectar patrones e irregularidades en una imagen tal como lo haría un médico dermatólogo, para ello se utilizaron métodos de segmentación y binarización para separar el objeto de interés y el fondo, también se utilizó el método de Canny para obtener los bordes del objeto con el fin de encontrar un posible borde irregular en la zona afectada, con todo este proceso computacional pesado se llega a una de las conclusiones: para que el sistema pueda reconocer un posible caso de carcinoma con un nivel alto de actividad es necesario contar con una GPU (unidad de procesamiento de gráficos) muy potente o en el peor de los casos utilizar la nube (Google Colab), esto con el fin de reducir el tiempo de entrenamiento en Matlab (scielo, 2021).

1.3.DESCRIPCIÓN DEL PROBLEMA.

El melanoma es el cáncer de piel más agresivo según la organización mundial de la salud, además se estima que cada año se producen 132000 casos de melanoma maligno en el mundo, de las cuales 66000 personas mueren al año por causa de esta enfermedad (Baptist Health South Florida, 2019).

La enfermedad del melanoma se origina por la transformación maligna de los melanocitos. Estas células nos protegen del sol, pero se vuelven malignas gracias a factores ambientales y genéticos, sin embargo, el factor más contribuyente a que estas células muten de manera maligna es la sobre exposición a los rayos solares (cinfasalud, 2015).

Al exponernos al sol, la luz ultravioleta puede generar daño en nuestro ADN, destruyendo nuestras células y en ocasiones causando quemaduras solares, este daño al ADN con el tiempo se acumula generando mutaciones que pueden llevar a las células a convertirse en malignas.

Además de este daño el sistema inmunológico deja de funcionar en el área, cuando la piel trata de repararse produciendo factores de crecimiento algunas de las células mutadas comienzan a crecer sin control y el sistema inmune alterado es incapaz de detenerlas generando un descontrol llamado melanoma (newsnetwork, 2014).

La mayoría de los melanomas al ser un cáncer de las células que le dan color a la piel, generan alteraciones muy llamativas en la pigmentación, con lunares negros, marrón, rojo, azul oscuro y gris, además de contar con bordes irregulares (mayoclinic, 2020).

Aunque la mayoría se origina en la piel, también puede afectar a las mucosas y ojos, la mayoría de las veces el melanoma cutáneo no presenta síntomas al inicio aparte de una leve comezón.

Resulta vital reconocer los melanomas y actuar rápidamente, cuando son diagnosticados a tiempo la mayoría de las lesiones se pueden curar mediante cirugía.

Sin embargo, si se detecta tardíamente el melanoma se convierte en un caso Metastásico, es decir, que se ha extendido a otras partes del cuerpo mediante el torrente sanguíneo y este tiene un pronóstico muy malo ya que puede invadir cualquier parte del cuerpo como el hígado, pulmón, sistema nervioso entre otros y ya no se puede operar (scielo, 2012).

Para poder detectar los posibles melanomas, un médico dermatólogo hace uso de múltiples herramientas tradicionales de laboratorio como (Iberomed, 2019):

- Punzones de Biopsia.
- Curetas Desechables.
- Dermatoscopio.
- Lámparas de Wood.
- Botellas de crioterapia portátiles.

Formulación Del Problema.

¿Cómo identificar si el paciente pudiese tener un posible caso de melanoma, sin el uso de herramientas tradicionales de laboratorio?

1.4.HIPÓTESIS.

A través de las redes neuronales y de la visión por computador, el prototipo detector de cáncer de piel puede identificar un posible caso de melanoma en un paciente con una confiabilidad mayor al 80%.

Variable = nivel de precisión o confiabilidad (mejorar el nivel es una prioridad).

1.5.OBJETIVO.

Debemos tomar en cuenta que la inteligencia artificial tiende muchas ramas, una de estas es la visión por computador, esta tiene un objetivo primordial y es el de enseñar a las maquinas a recopilar información de pixeles en una imagen, tomando en cuenta este párrafo tenemos el objetivo general.

1.5.1. OBJETIVO GENERAL.

“Desarrollar un prototipo de sistema en base a la visión computarizada, utilizando el reconocimiento de imágenes y un clasificador en base a las redes neuronales convolucionales, con el fin de detectar si una persona pudiese tener un carcinoma (melanoma benigno, lunar o mancha de piel) o un melanoma (cáncer de piel), mediante

una fotografía tomada en la lesión de la piel del paciente, con un nivel de confianza superior al 80%. ”

1.5.2. OBJETIVOS ESPECÍFICOS.

- Obtener una fotografía del área afectada del paciente mediante una cámara web, con dimensiones y características adecuadas para el procesamiento posterior.
- Obtener un centro de datos con imágenes de melanomas y carcinomas que posean dimensiones y características adecuadas para su posterior aplicación al entrenamiento del sistema.
- Encontrar un modelo de *machine learning* adecuado que permita identificar la lesión del paciente con el menor error posible.
- Desarrollar un entorno grafico sencillo e intuitivo vinculado al prototipo que permita que el medico dermatólogo identifique la lesión del paciente.
- Obtener la respuesta de la lesión del paciente lo más rápido posible.

1.6.JUSTIFICACIÓN.

1.6.1. JUSTIFICACIÓN APLICATIVO/PRACTICO.

El prototipo detector reducirá el tiempo y mejorará el diagnóstico de un posible caso de melanoma, brindando una atención oportuna a cada paciente, además de atraer más pacientes y generar prestigio por la confianza que se brinda, lo cual generará más y mejores ingresos.

1.6.2. JUSTIFICACIÓN METODOLÓGICA.

La computadora es una de las herramientas más colaboradas en el área de la investigación en el campo de la inteligencia artificial: por tanto, se dispone de una que servirá de apoyo y sostenimiento a nuestra labor, además se hará uso de una cámara web para obtener imágenes de la zona(s) afectada del paciente, software de programación como Anaconda

Navigator y material de escritorio (fotografías de melanomas y carcinomas impresas, lámparas led y demás).

1.6.3. JUSTIFICACIÓN TEÓRICA.

En la actualidad el melanoma es un padecimiento con dimensiones desconocidas en nuestro país, debido a que no mucho se comenta en los medios de comunicación, además que los primeros indicios de la enfermedad son fácilmente confundidos con lunares o simples manchas en la piel debido a la falta de vitaminas o malestares renales, sumado a esto que los servicios de salud pública no cuentan con la tecnología adecuada para identificar la enfermedad en las primeras etapas, y los servicios médicos privados que cuentan con ellos no están al alcance económico de la mayoría de la población.

El desconocimiento de las personas acerca de los primeros síntomas del melanoma juega un papel muy determinante, ya que, si es tratada de manera adecuada y a tiempo, podría significar salvar la vida de una persona.

1.7.ALCANCES Y LIMITES.

1.7.1. ALCANCES.

- El prototipo detector de melanomas proporcionara un diagnóstico a pacientes que pudiesen padecer de cáncer de piel.
- El prototipo será útil para el especialista médico, pues genera un diagnóstico que servirá de segunda opinión respecto a la lesión del paciente.
- Cada vez que se realice una captura fotográfica con el prototipo, este generará una carpeta con los datos del paciente en donde se encontrará la fotografía(s) con su respectiva lesión, generando así una colección de datos que irá en aumento y que a futuro servirá para un nuevo entrenamiento del prototipo.

1.7.2. LIMITES.

- El proceso de entrenamiento del sistema tendrá como fuente principal de información imágenes de tamaño 224 pixeles de ancho por 244 pixeles de alto con formato JPG.
- El prototipo no puede reconocer el nivel de gravedad de la lesión, solo reconoce si pudiese ser melanoma o carcinoma.
- El prototipo solo se limita a la determinación del diagnóstico de la enfermedad.
- El sistema funcionara correctamente siempre y cuando la zona afectada no esté cubierta de vellos o arrugas en exceso.
- El sistema no sustituirá la experiencia del experto médico, pero si ayudara con un diagnóstico que sirva de segunda opinión respecto a la lesión del paciente.

1.8. DISEÑO METODOLÓGICO.

Para el desarrollo del prototipo detector de melanomas utilizare dos metodologías, la primera es la metodología **Buchanan**, esta metodología me permitirá referenciar una serie de fases para producir mi prototipo a nivel documental o teórico ya que está orientada a la creación de sistemas expertos y la segunda metodología es la **Kanban**, esta metodología me permitirá organizar los pasos para la implementación del sistema.

Existen seis fases dentro de la metodología **Buchanan**:

- **Identificación:** en esta fase se reconocen aspectos importantes del problema, participantes con sus respectivos roles, recursos y las metas a alcanzar.
- **Conceptualización:** se identifica el flujo de la información acerca de los conocimientos del experto (medico dermatólogo).
- **Formalización:** en esta fase se identifican los conceptos más importantes provenientes de la fase de conceptualización.
- **Implementación:** en esta fase se formalizan los conceptos otorgados por el experto (medico dermatólogo) y se escoge el lenguaje junto con las herramientas

de programación, además es en esta fase es donde se implementará la segunda metodología **Kanban** para el desarrollo del prototipo.

- **Pruebas de rendimiento:** en esta fase se observa el comportamiento de la salida del modelo de clasificación (redes neuronales) evaluando el resultado de la función de pérdida y el nivel de asertividad.

Una vez terminado de comprender las diferentes fases de la metodología Buchanan, paso a explicar la metodología que se utilizara en la fase de implementación, esta metodología se denomina Kanban, que significa tablero visual, este concepto se aplica a un tablero de tarjetas (ver figura 1.1) que permite mejorar el flujo de trabajo y alcanzar un ritmo libre de estrés. El tablero se compone de columnas y filas donde se tiene:

- las **columnas** del tablero corresponden a los estados de flujo de tareas determinadas.
 - La primera está destinada a tareas **pendientes (Que hacer)**.
 - La segunda columna está destinada a tareas en **proceso (Haciendo)**.
 - La tercera está destinada a tareas **terminadas (Hecho)**.
- Las **filas** representan las tareas que pasaran por las tres columnas ya mencionadas.



Fig. 1.1. Se muestra el tablero con sus respectivas columnas y filas.
Fuente: tecnosoluciones.com, 2011

CAPÍTULO 2: MARCO TEÓRICO

2.1. LA INTELIGENCIA ARTIFICIAL.

Cuando se habla de la IA (inteligencia artificial) se hace referencia a sistemas o máquinas que tratan de imitar a la inteligencia humana, esto con el fin de poder efectuar diferentes tareas que puedan mejorar iterativamente a partir de toda la información que recopilan.

Según McCarthy (2020) la IA se puede clasificar en:

- **IA débil:** Este tipo de IA incorpora un sistema diseñado para llevar a cabo **un** trabajo en particular. Los sistemas de IA débiles incluyen a los videojuegos y dispositivos electrónicos como Alexa y Siri.
- **IA fuerte:** Este tipo de IA lleva a cabo las tareas consideradas como humanas. Se trata de sistemas más complejos y sofisticados; están programados para manejar situaciones en las que se les puede requerir que resuelvan **varios** problemas sin que una persona intervenga. Este tipo de sistemas se pueden encontrar en aplicaciones como vehículos autónomos, en algunos quirófanos de hospitales o en los robots de Boston Dynamics. Esta clasificación entra en la categoría del ML (*machine learning*) y el DL (*deep learning*).

Algunos ejemplos del uso de la IA en la vida cotidiana son los siguientes:

- **Los motores de recomendación:** son sistemas que proporcionan recomendaciones automatizadas de acuerdo al historial de búsqueda de un usuario.
- **Los chatbots:** son asistentes virtuales automatizados que se comunican con los usuarios a través de mensajes de texto.
- **Los asistentes inteligentes:** es un software que puede realizar tareas u ofrecer servicios a un usuario, estos servicios están basados en datos de entrada del usuario.

La IA no pretende reemplazar a los humanos. Su objetivo es mejorar significativamente las capacidades y contribuciones humanas. Eso la convierte en un activo empresarial muy valioso (oracle, 2019).

Dentro del campo de la IA podemos encontrar diferentes subcategorías que responden a diferentes comportamientos o agentes inteligentes, por ejemplo:

- **El campo de la robótica:** hace referencia a la capacidad de moverse y adaptarse al entorno.
- **El campo del N.L.P (*natural language processing*):** se refiere a la capacidad de entender el lenguaje del ser humano.
- **El campo de Voz:** se hace referencia a la capacidad de hablar, se estudia tanto la conversión de voz a texto y viceversa.
- **El campo de la visión artificial (VA):** hace referencia a la capacidad que tiene una máquina de poder ver su entorno y analizarlo, de manera que pueda obtener un valor que permita clasificar un objeto.

Todas estas capacidades conforman campos de estudios propios dentro del mundo de la IA, pero sin embargo si hay una capacidad que define a un agente inteligente es la capacidad de aprender es decir el ML (*machine learning*).

2.1.1 EL MACHINE LEARNING.

El ML o el aprendizaje automático estudia como dotar a las máquinas de la capacidad del aprendizaje a partir de un conjunto de experiencias, estas pueden ser de tres maneras diferentes:

- **Aprendizaje Supervisado:** En este aprendizaje los algoritmos trabajan con datos etiquetados para el entrenamiento (véase Fig. 2.1), intentado encontrar una función que, dadas las variables de entrada, les asigne la etiqueta de salida adecuada. El algoritmo se entrena con un “histórico” de datos y así aprende a asignar la etiqueta de salida adecuada a un nuevo valor, en otras palabras, predice

el valor de salida (GONZÁLES, L., 2018), suele llamarse así debido a que el desarrollador actúa como una guía para enseñar al algoritmo a llegar a las salidas deseadas.



Fig. 2.1. Imágenes de entrada con etiquetas, representan un valor dado.

- **Aprendizaje No supervisado:** el aprendizaje no supervisado tiene lugar cuando no se dispone de datos etiquetados para el entrenamiento (véase Fig. 2.2). Sólo conocemos los datos de entrada, este modelo agrupa los datos de entrada y trata de clasificarlos en clases por medio de sus patrones que los componen.



Fig. 2.2. Imágenes sin etiquetar, solo poseen un valor numérico que las identifica.

- **Aprendizaje reforzado:** este tipo aprendizaje se basa en mejorar la respuesta del modelo usando un proceso de retroalimentación. El algoritmo aprende observando el mundo que le rodea, su información de entrada es la misma retroalimentación, que la obtiene del mundo exterior como respuesta a sus acciones. Por lo tanto, el sistema aprende a base de prueba y error.

Entonces decimos que el ML es una disciplina dentro del campo de la IA, pero no es una disciplina cualquiera, es un componente que se relaciona y conecta con el resto de las subcategorías mencionadas anteriormente (robótica, N.L.P, Voz, VA), esto porque las subcategorías pueden ser imitadas ya sea porque alguien las haya programado o en el mejor de los casos porque el propio sistema haya aprendido a realizarlas. Podemos decir que es muy distinto programar una máquina para que esta pueda moverse en un entorno y otra es programarla para que aprenda a moverse en su entorno. Entonces podemos llegar a la conclusión de que ML es un subconjunto o rama de la IA.

Dentro del ML nos encontramos con un nuevo mundo donde existen diferentes técnicas de aprendizaje que sirven para cubrir diferentes tipos de aplicaciones, por ejemplo, tenemos la técnica de los árboles de decisión, modelos de regresión, modelos de clasificación, técnicas de *clusterización* y muchísimas otras más, sin embargo, si una de estas técnicas ha dado fama al campo del ML durante la última década son los sistemas conexionistas.

2.1.2 SISTEMAS CONEXIONISTAS.

El sistema conexionista es un conjunto de enfoques que abarca los ámbitos de la IA, psicología cognitiva, ciencia cognitiva, neurociencia y filosofía de la mente, que presenta a los fenómenos de la mente y del comportamiento como procesos que emergen de redes formadas por unidades sencillas interconectadas.

Hay muchas formas de conexionismo, pero la forma más común es el modelo de las redes neuronales, este modelo se basa en la interconexión de neuronas artificiales (perceptrón) que funcionan como pequeños mecanismos que son capaces de otorgar una salida simple mediante un proceso matemático, sin embargo, la unión en paralelo de estas neuronas artificiales genera una capa y esta capa se conecta con otras capas generando un resultado que es el de clasificar la información de entrada.

2.1.2.1 EL PERCEPTRÓN.

Un Perceptrón es una unidad básica de procesamiento que se encuentra dentro de una RN (red neuronal), que es similar a una neurona biológica, estas neuronas tienen conexiones de entrada a través de las cuales reciben un estímulo externo, el cual se denomina valor de entrada, con estos valores la neurona realiza un cálculo interno, la cual generará un valor de salida llamado resultado.

Sin embargo, detrás del nombre de Perceptrón, solo está presente una función matemática (función de activación), la cual utilizará todos los valores de entrada para realizar una suma ponderada de las entradas, además a cada valor se le asigna un peso (w), es decir que cada conexión relacionada a la neurona tendrá un valor que servirá para definir la intensidad que afectará a la neurona en su totalidad.

Estas son las descripciones de los componentes de un Perceptrón (véase Fig. 2.3):

- **Entradas:** corresponden a un vector de características (X_i)
- **Pesos:** son variables constantes que cambian su valor según las evaluaciones que requiera el modelo y salida (W_i).
- **Sumatoria y umbral:** es una operación matemática donde se multiplican los pesos por las entradas correspondientes ($\sum(X_i * W_i)$), cada resultado se suma obteniendo un único resultado.
- **Función de activación:** agarra el resultado lineal de la sumatoria y la pasa por una función específica (f), con el fin de obtener un resultado no-lineal que podamos interpretar en un rango definido.
- **Salida:** interpretamos la salida de la función de activación según nuestros requerimientos, por ejemplo, si la función de activación fuese Sigmoid tendríamos 0 o 1 y etiquetaríamos estas posibles salidas como si 1 fuera positivo (melanoma) y 0 negativo (carcinoma)

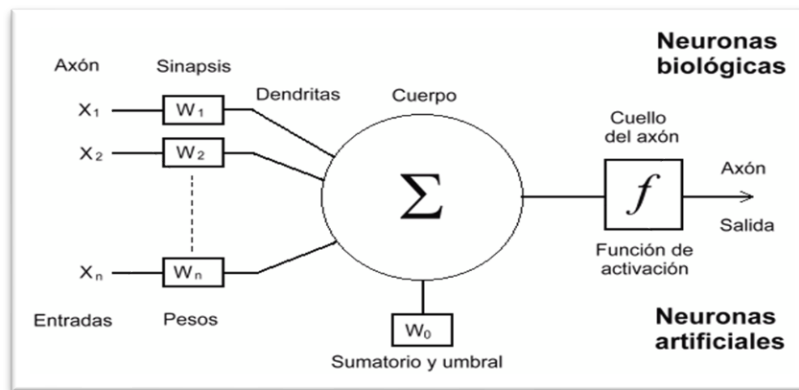


Fig. 2.3. Imagen donde se observa los componentes de un Perceptrón.
Fuente: um.es, 2019

El conjunto de varias neuronas trabajando paralelamente forman una capa y el conjunto de tres capas (capa de entrada, oculta, salida) forman una RN simple.

2.1.2.2 LA RED NEURONAL SIMPLE.

El objetivo de cualquier RN (red neuronal) es el de aprender a clasificar elementos u objetos modificándose automáticamente a sí mismo (atriainnovation, 2019), una RN simple está compuesta de 3 capas:

- **La capa de entrada:** esta capa tiene como principal función recibir parámetros que describan un objeto de estudio (X_i) y multiplicarlas por los pesos correspondientes (W_i).
- **La capa oculta:** esta capa recibe los resultados de la anterior capa y realiza el mismo trabajo de multiplicación de los pesos por las entradas, luego se suma una variable denominada “bias” al resultado de la multiplicación, esto para obtener una salida lineal.
- **La capa de salida:** esta capa realiza la interpretación del resultado proveniente de la salida de la capa oculta, en la mayoría de los casos la salida de esta capa suele ser en decodificación *OneHot* (resultado binario 0 y 1) y suele contener una sola neurona.

Planteemos un ejemplo de la vida real, supongamos que queremos hallar el valor económico de una vivienda utilizando una RN (red neuronal) simple (ver fig. 2.4), como entradas tenemos a los parámetros más esenciales de esa vivienda como ser: la superficie en metros, la edad en años, el número de dormitorios que posee y la distancia hacia la ciudad en kilómetros, toda esa información se procesa en la capa oculta y en la última capa se obtiene un valor de salida que clasificara el valor económico de la vivienda.

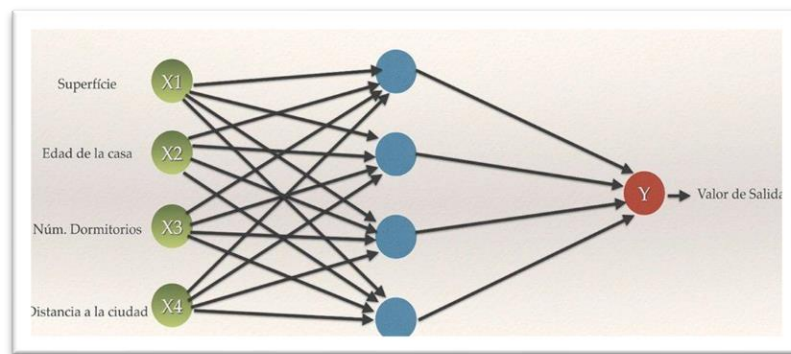


Fig. 2.4. Imagen donde se describe el ejemplo de la vivienda.
Fuente: (iartificial, 2020)

Una RN (red neuronal) simple otorga soluciones a problemas de clasificación lineal (ver fig. 2.5), sin embargo, si los datos a clasificar no son linealmente separables se necesita de dos capas ocultas y funciones de activación para formar una separación mediante un plano (ver fig. 2.6), pero ¿qué pasa cuando una región de los datos con patrones que los identifica está dentro de la otra región con patrones diferentes? (ver fig. 2.7), entonces lo aconsejable sería aplicar tres capas ocultas con sus respectivas funciones de activación formando una separación mediante un hiperplano.

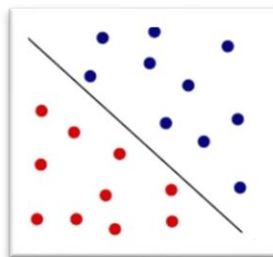


Fig. 2.5. Clasificación Lineal de datos.
Fuente: (Xpικούς, 2020)

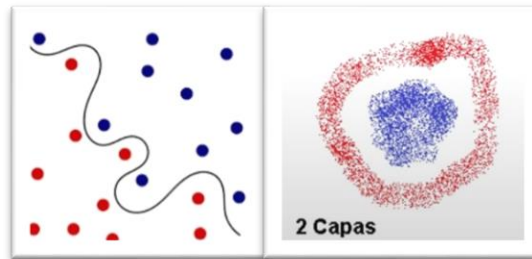


Fig. 2.6. Región linealmente no separable.
Fuente: (Xpicuos, 2020)

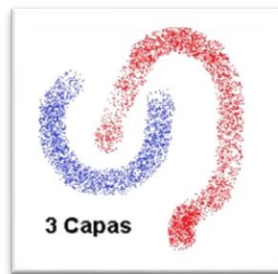


Fig. 2.7. Una región dentro de otra región.
Fuente: (Xpicuos, 2020)

Cuando aplicamos más de una capa oculta a nuestra RN (red neuronal) simple esta se vuelve una RN profunda ya que al aumentar más capas ocultas también aumenta su complejidad de clasificación. Sin embargo, no debemos olvidar que las RN tanto simples como profundas dependen de las salidas de cada capa oculta que posean, estas salidas dependen de la función de activación para poder llegar a un resultado abstracto.

2.1.2.3 LA FUNCIÓN DE ACTIVACIÓN.

Es un proceso matemático que dependerá de lo que nosotros requiramos y como nosotros hayamos definido las etiquetas de salida, esta salida estará representada por un rango numérico de valores (ver fig. 2.8), este rango dependerá de la función que apliquemos, existen muchos tipos de funciones de activación, los más populares son los siguientes (diegocalvo, 2018):

- **Sigmoid:** la función Sigmoid o Sigmoido transforma los valores introducidos a una escala (0,1), donde los valores altos tienen de manera asintótica a 1 y los valores muy bajos tienden a acercarse a 0. Algunas características de esta función son:
 - Satura y mata el gradiente.
 - Lenta convergencia.
 - Esta acotada entre 0 y 1.
 - Buen rendimiento en la última capa.

$$f_{(z)} = \frac{1}{1 + e^{-z}}$$

*donde z: es el valor de la salida de la neurona $((X * W) + b)$.*

- **Tanh:** también conocida como función Tangente Hiperbólica, transforma los valores introducidos a una escala (-1,1), donde los valores altos tienen de manera asintótica a 1 y los valores muy bajos tienden a clasificarse como -1. Algunas características de esta función son las siguientes:
 - Muy similar a la Sigmoido.
 - Lenta convergencia.
 - Esta acotada entre -1 y 1.
 - Se usa para decidir entre una opción y su inversa o contraria.
 - Buen desempeño en redes recurrentes.

$$f_{(z)} = \frac{2}{1 + e^{-2z}} - 1$$

*donde z: es el valor de la salida de la neurona $((X * W) + b)$.*

- **ReLU:** también conocido como unidad de rectificación lineal, la función ReLU transforma los valores introducidos anulando los valores negativos convirtiéndolos en 0 y dejando los positivos tal y como entran. Algunas características de esta función son:
 - Solo se activa si son positivos.

- No esta acotada.
- Es posible que algunas neuronas dejen de funcionar.
- Se comportan bien con imágenes.
- Buen desempeño en redes convolucionales.

$$f(z) = \max(0, z) = \begin{cases} \text{sera } 0 & \text{si } z < 0 \\ \text{sera } 1 & \text{si } z \geq 0 \end{cases}$$

donde z : es el valor de la salida de la neurona $((X * W) + b)$.

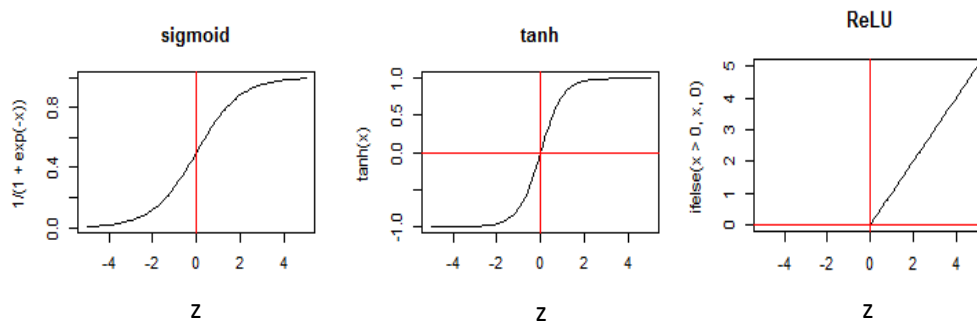


Fig. 2.8. Graficas donde demuestran los rangos de valores numéricos por función.
Fuente: (mql5, 2017)

Entonces tenemos claro que dentro de una RN (red neuronal) existen elementos que realizan diferentes procesos matemáticos los cuales desembocan en la capa de salida, esta respuesta se interpreta según el diseñador de la RN y luego se toma una decisión, este diseño va muy bien con datos numéricos que describen el problema como los datos del ejemplo de la vivienda (superficie, edad, distancia), pero ¿Una RN funcionaria para clasificar una imagen?, la respuesta es que efectivamente una RN puede clasificar una imagen convirtiéndola en valores numéricos para que pueda ser admitida por la capa de entrada, este tipo de RN se denomina RNC (red neuronal convolucional).

2.1.2.4 LA RED NEURONAL CONVOLUCIONAL.

Es una RN (red neuronal) especializada para trabajar con imágenes, la importancia de estas redes viene de su capacidad de descifrar los patrones más complejos almacenados en enormes *datasets* de imágenes (una colección de imágenes) dotando de ojos a maquinas que pueden observar rostros de personas, radiografías de pacientes y los peatones que se cruzan al frente de un vehículo autónomo, en resumen: se ha conseguido que las maquinas puedan percibir el mundo que las rodea.

Para poder comprender a una RNC (red neuronal convolucional) es necesario comprender los principios básicos que hay detrás del proceso de la visión biológica, por ejemplo, si vemos una fotografía podremos reconocer fácilmente los objetos que hay en ella, esto es porque seguramente hemos escaneado la imagen y hemos detectado la presencia de elementos que reconocemos porque en nuestra vivencia lo hemos aprendido. Si en la fotografía se muestra un rostro fácilmente podremos decir que pertenece a un hombre o a una mujer, esto es porque hemos identificado las partes que conforman a este: los ojos, la boca, la nariz, el cabello, etc. Tomemos un ejemplo, si vemos una fotografía en donde estamos con nuestros amigos o familiares podemos observar muchos objetos alrededor de nosotros, ahora bien si nos concentramos en los rostros observamos diferentes características y rasgos que nos identifican del resto de las demás personas, uno de esos rasgos son los ojos, pero ¿cómo sabemos que una estructura es un ojo humano?, esto es porque hemos detectado una serie de elementos que conforman normalmente a un ojo: una pupila, pestañas, iris, cristalino, etc. Y todo esto lo hemos reconocido porque hemos detectado patrones como: formas geométricas, contrastes y texturas.

Al final si reproducimos los pasos que realiza tu córtex visual (zona cerebral responsable del procesamiento visual humano) nos encontraremos con un procesamiento en cascada donde primero se identifican aquellos elementos básicos y generales (líneas horizontales, verticales y demás), que en posteriores capas estos se combinan para generar patrones cada vez más complejos (formas de diferentes texturas, colores y demás) (Crespo, 2020)

Una RNC tiene un diseño que extrae la estructura espacial (esbozo) de cualquier imagen, para lograr todo aquello se debe aplicar una capa matricial donde se realiza una operación matemática conocida como convolución.

2.1.2.5 LA CONVOLUCIÓN.

Una convolución aplicada sobre una imagen no es más que una operación que juega con los valores de los pixeles de dicha imagen para producir otra imagen. Cada pixel nuevo que vayamos a generar se calcula colocando una matriz de números que llamaremos filtros o kernel sobre la imagen original en donde “multiplicaremos y sumaremos” los valores de cada pixel vecino para obtener así el nuevo valor (ver fig. 2.9).



Fig. 2.9. En el lado izquierdo se aplica un kernel de 3x3 a una porción de la imagen original, en la derecha se calcula mediante multiplicaciones y sumas el valor de un nuevo pixel.

Fuente: (Carlos Santana, 2020)

Si desplazamos el filtro realizando estas operaciones matemáticas por toda nuestra imagen lo que obtendremos como resultado será una nueva imagen. Esta nueva imagen dependerá de cómo configuremos los parámetros de nuestro filtro, en la cual podremos obtener un resultado u otro. Por ejemplo, si tenemos un filtro de 3x3 además de un valor central de uno y el resto de los vecinos son cero nos dará una copia exacta a la imagen original (ver fig. 2.10).

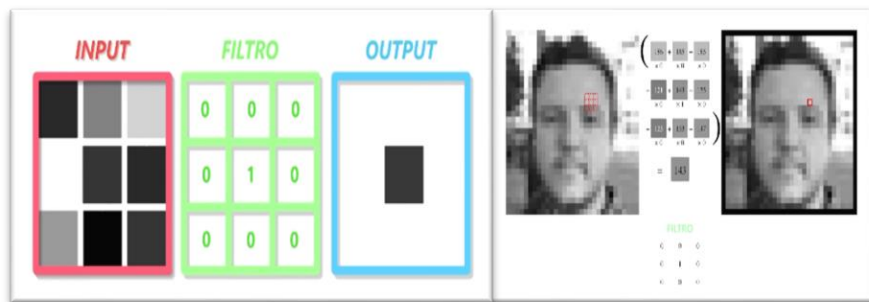


Fig. 2.10. en la izquierda se muestra la interacción del filtro y la porción de la imagen, en la derecha se muestra el resultado y el proceso de convolución con un filtro de 3x3.

Fuente: (Carlos Santana, 2020)

Ahora si multiplicamos con un filtro de 3x3 en donde cada valor será la media de los colores de los pixeles ($1/9$, pues en una matriz de 3x3 hay 9 elementos) nos dará como resultado un desenfoque en la imagen original, este es un método que se utiliza para implementar los efectos que vemos hoy en día en muchos de los softwares de edición de imágenes y video (ver fig. 2.11).



Fig. 2.11. Se observa el desenfoque de la imagen original aplicando un filtro de 3x3 con valores de la media.

Fuente: (Carlos Santana, 2020)

Ahora si multiplicamos la imagen original con un filtro de 3x3 en donde cada valor será el siguiente: la columna izquierda del filtro tendrá valores de unos negativos, la columna central tendrá valores de ceros y la columna derecha tendrá valores de unos, obtendremos un filtro que se activará cuando encuentre diferencias de contraste y por lo tanto que sirva para detectar bordes verticales (ver fig. 2.12), para detectar los bordes horizontales solo trasponemos el filtro.

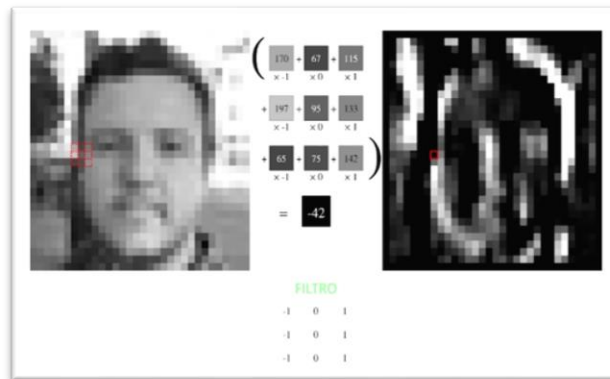


Fig. 2.12. Mediante la aplicación del filtro explicado, se obtiene los bordes verticales de los objetos enfocados en cualquier imagen.
Fuente: (Carlos Santana, 2020)

En resumen, la operación de convolución sobre una imagen puede detectar cosas diferentes según cuales sean los valores del filtro que definamos. Cuando apliquemos la convolución dentro de una RN (red neuronal) los valores que tendrá el filtro no lo definiremos nosotros, sino que estos valores la ira aprendiendo poco a poco la RN para poder hacer mejor su tarea, aprender estos filtros para detectar patrones es el principal trabajo de la RNC (red neuronal convolucional).

A cada una de las nuevas imágenes generadas por filtros se los conoce como mapa de características, ya que actúa como un mapa donde se nos indica en que parte de la imagen se ha detectado la característica buscada por dicho filtro, cada pixel blanco será una activación que nos indique que el elemento buscado esta allí, como ejemplo tenemos la figura 2.11, en donde se busca los bordes verticales de una imagen y en el mapa de características se denota los bordes verticales como pixeles blancos.

Por tanto, resumiendo: entra una imagen, se aplica una serie de convoluciones y genera un conjunto de mapa de características. Sin embargo eso no es todo, el potencial de este tipo de redes se encuentra en que esta operación se va a realizar secuencialmente, donde la salida de una de las capas se convertirá en la entrada de la siguiente, vamos a ponernos un ejemplo: si tenemos una imagen a color con sus tres canales RGB(rojo, verde, azul) esto se podría interpretar como si en realidad tuviésemos tres mapas de características diferentes donde se han detectado elementos en rojo, en verde y en azul, ahora tenemos

tres mapas de características donde si realizamos la operación de convolución 16 veces (tomemos como ejemplo 16 filtros) podremos detectar 16 cosas diferentes que nos darán como resultado 16 mapas de características (ver fig. 2.13). Estos 16 mapas de características pasaran a ser la entrada de la siguiente capa convolucional.

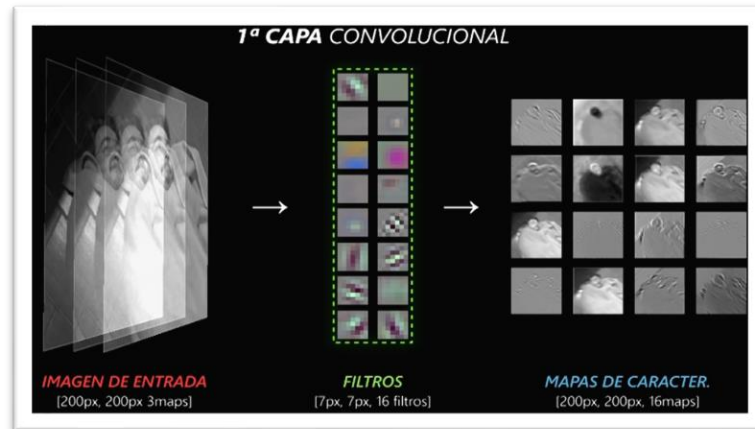


Fig. 2.13. proceso de una capa convolucional respecto a un mapa de características.
Fuente: (HealtBigData, 2020)

Entonces decimos que la imagen resultante o mapa de características dependerá del filtro que apliquemos.

2.1.2.6 EL FILTRO.

El filtro también denominado Kernel, no es más que una matriz de valores numéricos, esta posee una dimensión de $N \times N$, lo que quiere decir que el filtro es cuadrado, los filtros más comunes son de 3x3, 5x5 y 7x7 pixeles, pero ¿Sera posible que un filtro tan pequeño pueda detectar patrones tan complejos como un ojo o una rueda?

La respuesta la encontramos en la sucesión de capas o la profundidad de la arquitectura de nuestra RNC (red neuronal convolucional), lo que sucede es que la operación de convolución cada vez se va haciendo más potente, un ejemplo seria: si tomamos una región de 9 pixeles de la imagen original, nuestro filtro lo ha convertido en un único pixel de información y por lo tanto si volvemos a aplicar una convolución sobre estos mapas de características en realidad estaremos accediendo a más información espacial de la imagen

original (ver fig. 2.14), esto es algo que además llega a reducir cada cierto tiempo la resolución de nuestros mapas de características (ver fig. 2.15). En resumen, las convoluciones con la ayuda de los filtros pueden detectar cambios de contraste, textura y superficies planas de los objetos en una imagen.

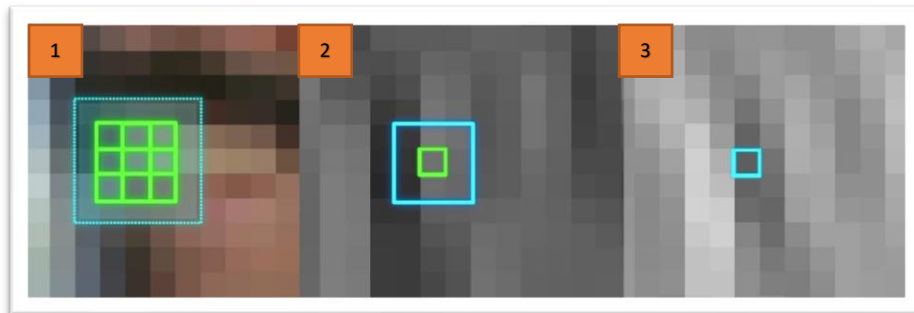


Fig. 2.14. En la imagen 1 se observa el filtro de 3x3 (9 píxeles) de color verde, en la imagen 2 se observa que la información del filtro verde se redujo y se aplica otra convolución de 3x3 de color azul, en la imagen 3 se observa la compresión de la información del filtro azul.

Fuente: (HealtBigData, 2020)

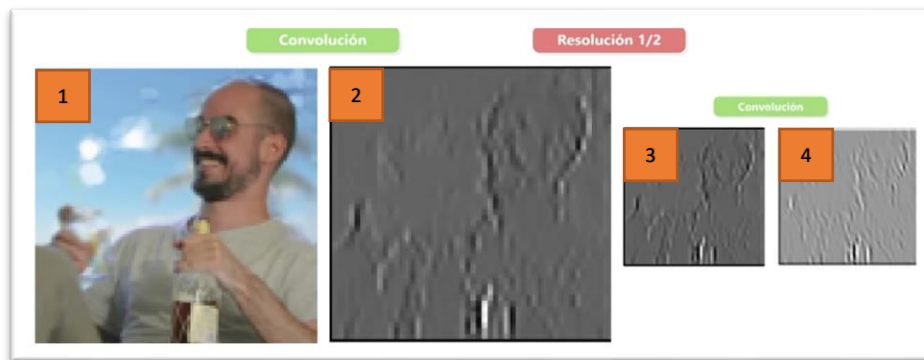


Fig. 2.15. La imagen 1 es la original, la imagen 2 es el mapa de características de la imagen original, en la imagen 3 se observa el tamaño real de la imagen 2 en comparación a la imagen original y la imagen 4 es otro mapa de características de la imagen 3.

Fuente: (HealtBigData, 2020)

Al final la operación de convolución por si sola tampoco puede detectar cosas muy complejas más que bordes y patrones muy simples, sin embargo, si volvemos a hacer estas detecciones sobre las anteriores se nos permitiría componer cada vez patrones más complejos. Es por eso que una arquitectura convolucional normalmente se representa como un embudo donde la imagen inicial se va comprimiendo espacialmente, es decir su resolución va disminuyendo al mismo tiempo que su grosor va aumentando, en otras

palabras el número de mapas de características que vamos detectando va en aumento, todo este proceso llega a su fin cuando ya abramos detectado todos los patrones necesarios y contemos con muchos mapas de características que ahora si podremos volverla como una entrada (*input*) independiente dentro de una RNP (red neuronal profunda), luego esta acabara por tomar la decisión clasificando la imagen (ver fig. 2.16).

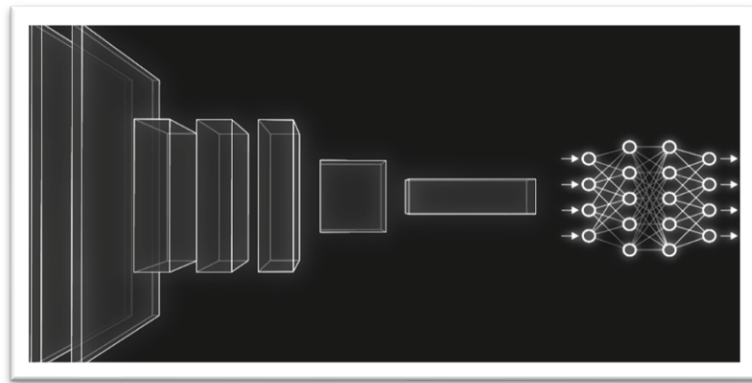


Fig. 2.16. Se muestra la estructura de una RNC, donde a la izquierda se observa el embudo convolucional y a la derecha una RNP.
Fuente: (HealtBigData, 2020)

En resumen, tenemos que una convolución es el resultado de aplicar un filtro mediante operaciones matemáticas en una imagen con el fin de obtener patrones y características de los objetos de dicha imagen, esto se aplica de manera secuencial hasta obtener los suficientes mapas de características que servirán de entrada a una RN (red neuronal).

Como curiosidad, para conocer el tamaño de los mapas de características debemos conocer el tamaño de la imagen original, el tamaño del filtro, el padding (pixeles de relleno alrededor de la imagen original) y el Stride (salto por cada operación entre Kernel y matriz original) y luego aplicarlo en la siguiente fórmula para hallar el alto y ancho (debemos recordar que como es una imagen cuadrada ambos valores son los mismos).

$$tamaño = \frac{h - k + 2p}{s} + 1$$

Donde:

- **tamaño:** puede comprenderse como el **alto o ancho** del mapa de características.
- **h:** el tamaño de las columnas o filas de la imagen original.
- **k:** el tamaño de las columnas o filas del Kernel.
- **p:** el padding de relleno (usualmente es 0).
- **s:** los saltos entre el producto del Kernel y la imagen original (usualmente es 1).

Para conocer el número de parámetros calculables (parámetros actualizables) después de cada convolución es necesario aplicar la siguiente formula (Codificando_Bits, 2019):

Si es la primera convolución aplicar:

$$((PK * NK) * 3) + bNK$$

Donde:

- PK=el número de pixeles de un solo *kernel*.
- NK=el número de *kernels* a aplicar en la convolución.
- bNK=la cantidad de *bias* por *kernel*, suele tener un valor similar a NK.
- Además, que el numero 3 representa una constante de los 3 canales de colores en una imagen (RGB).

Para calcular el número de parámetros de una convolución distinta a la primera:

$$((PK * NK) * KA) + bNK$$

Donde:

- KA=el número de *kernels* de la convolución anterior.

Sabemos que la función principal de una RN (red neuronal) es la de clasificar la información introducida, además son capaces de aprender de forma jerarquizada, es decir que la información se aprende por niveles, donde en las primeras capas se aprenden

conceptos muy concretos como por ejemplo ¿Qué es un tornillo?, ¿Qué es un espejo?, ¿Qué es una rueda?, ¿Qué es un volante? Y en las capas posteriores se usa la información aprendida previamente para aprender conceptos más abstractos, por ejemplo: un automóvil, un camión o un bus.

Esto hace que a medida que aplicamos más capas al modelo de RN (red neuronal) la información que se aprenderá será cada vez más abstracta e interesante, convirtiéndose en algoritmos cada vez más complejos. Este incremento en el número de capas y en la complejidad es lo que hace que estos algoritmos sean conocidos como algoritmos de DL (*Deep Learning*) o aprendizaje profundo.

2.1.3 VISIÓN POR COMPUTADOR.

De nuestros cinco sentidos, la visión es sin duda el más importante, pues dependemos de la visión para analizar nuestro entorno y tomar decisiones de acuerdo a los elementos que hay en él. Lograr que un ordenador aprenda a ver y a clasificar objetos tal como lo hacen las personas, no resulta ser una tarea sencilla, por lo que es necesario implementar un sistema de reconocimiento y clasificación de imágenes denominado VC (visión por computador).

Se podría decir que la visión humana involucra tareas de procesamiento de la información, ya que para poder saber qué objetos nos rodean en nuestro entorno, nuestros cerebros deben ser capaces de representar esta información (color, forma y tamaño) en toda su abundancia, con el fin de utilizar esta información como base para las decisiones sobre nuestros pensamientos y acciones. Algunos autores tienen argumentos que señalan lo siguiente sobre la VC:

La Universidad Internacional de Valencia define que la visión por computador llega a asemejarse a la de un niño pequeño, pues el resultado de clasificar imágenes dependerá de la supervisión que se le dé en la etapa del entrenamiento, existen casos en los que al momento de clasificar objetos mediante imágenes resulta ser menos dificultoso para un

ordenador el poder reconocerlas; sin embargo, si se trata de reconocer objetos en estado que no se espera, el reconocimiento resulta ser más complejo (TICONA, 2020).

Por otro lado, Crespo y Algorry (CRESPO, ALGORRY, 2019), señalan que un sistema puede reconocer imágenes a partir de los ejemplos con los que se haya llegado a entrenar la red neuronal, siendo que la misma a través de los procesos de aprendizaje automático, llega a modificar los pasos de las interconexiones para emitir un estado en el que el error de la predicción sea lo más mínimo posible.

Otro aspecto que cabe señalar según información proporcionada por Bense (BENSE, T., 2007), es que, al tratarse con imágenes lo que se debe hacer es seleccionar las áreas de interés bajo formas de rectángulos o polígonos para que de esa manera los datos numéricos queden archivados en la computadora como regiones de interés, los cuales serán tomados como datos de entrenamiento, a la vez cabe mencionar que para realizar dicha selección el analista debe tener un conocimiento previo del área de estudio.

Se define que un sistema de reconocimiento y clasificación de imágenes, es como cualquier sistema de inteligencia artificial, capaz procesar información visual (VIU, 2019).

2.1.3.1 COMO FUNCIONA LA VISIÓN POR COMPUTADOR.

En cuanto a la funcionalidad de la VC, existen varios autores que la definen de la siguiente manera:

La VC consiste en la extracción automatizada de información de las imágenes. En cuanto a su funcionamiento, se afirma que para que un sistema pueda reconocer imágenes, es necesario la creación de una RN (red neuronal), la cual será capaz de procesar los píxeles individuales de una imagen, convirtiéndolos en información valiosa a la hora de tomar una decisión (VIU, 2019).

Sin embargo, Crespo y Algorry(2020), afirman que un sistema puede llegar a reconocer imágenes a partir de los diferentes ejemplos introducidos en la etapa del entrenamiento de

la red neuronal, modificando con cada convolución los pasos de las interconexiones para emitir un estado en el que el error de la predicción sea lo más mínimo posible.

Según lo aprendido desde los capítulos 2.1 hasta 2.1.2.6, podemos decir que la VC (visión por computador) trata de comprender los objetos de su entorno mediante una imagen, esta imagen entrará al embudo convolucional la cual nos devolverá un conjunto de mapas de características que servirán de entrada para la RN (red neuronal) formando así una RNC (red neuronal convolucional) y obteniendo un clasificador de objetos.

2.1.3.2 EJEMPLOS APLICATIVOS UTILIZANDO LA VISIÓN POR COMPUTADOR.

La VC (visión por computador) es un área con más de 40 años de investigación, por lo que ya contamos con varias aplicaciones. Estas aplicaciones incluyen los siguientes aspectos (Bootcamp AI, 2019):

- **Reconocimiento óptico de caracteres:** consiste en la identificación automáticamente de caracteres que pertenecen a un determinado alfabeto, para luego almacenarlos en forma de datos.
- **Inspección robotizada:** se trata de un análisis visual de piezas o productos con el fin de garantizar la calidad de los componentes de fabricación, para ello se utiliza una visión estéreo con iluminación especializada.
- **Construcción de modelos 3D:** se refiere a la construcción automatizada de modelos 3D a partir de fotografías.
- **Imágenes médicas:** es utilizada para analizar fotografías radiográficas con el fin de detectar lesiones en el paciente.
- **Seguridad automotriz:** ayuda a detectar obstáculos mediante un sistema de conducción asistida utilizando diferentes cámaras.
- **Vigilancia:** se trata de monitorear intrusos, analizar del tráfico vial y monitoreo de piscinas para posibles víctimas de ahogamiento.

- **Reconocimiento de huellas dactilares y biometría:** se utiliza para la identificación automática de accesos y también utilizada para aplicaciones forenses.
- **Detección de rostros:** utilizado para mejorar el foco de las cámaras y para hacer una búsqueda más relevante de personas en imágenes mediante rasgos faciales.

Una vez que ya definimos los conceptos del prototipo ahora toca preguntarnos, ¿Utilizar el método de redes neuronales como clasificador de parámetros, podría llevarme a alcanzar mi objetivo de superar el 80% de asertividad? Para poder responder a la pregunta existe un método llamado Validación Cruzada, este método permite analizar los resultados de asertividad del modelo y verificar si es el más adecuado para clasificar un *dataset* en específico (en mi caso imágenes de melanomas y carcinomas).

2.1.4 VALIDACIÓN CRUZADA K-CROSS.

Este método consiste en lo siguiente:

- Agarramos la porción de entrenamiento del modelo y dividimos en partes simétricas, según el autor Ignacio Forrester (dataEvo, 2018) la mejor partición es de cinco (ver imagen 2.17).
- Utiliza cuatro porciones de las cinco para entrenar el modelo y al final utiliza la quinta porción sobrante para probar el modelo.
- Se debe realizar el proceso anterior una y otra vez hasta que hayamos probado con cada una de las particiones por separado, dando a lugar 5 scores (nivel de asertividad) distintos.
- Cuando tengamos estos niveles de asertividad, los promediaremos y obtendremos su desviación estándar.
- Luego verificaremos lo siguiente:
 - Si la desviación estándar es lo suficientemente grande quiere decir que el modelo no es el adecuado para dicha tarea.

- Sin embargo, si es lo suficientemente pequeña, decimos que el modelo pudiese ser el más adecuado para resolver el problema.

TRAIN	1	Test - 5: 0.50
TRAIN	2	Test - 4: 0.52
TRAIN	3	Test - 3: 0.48
TEST	4	Test - 2: 0.49
TRAIN	5	Test - 1: 0.53
		XVAL: 0.504 +/- 0.021

Imagen 2.17: A la izquierda se observa la división de los datos en partes iguales, a la derecha vemos la obtención de la media y la desviación estándar respecto de cada partición.

Fuente: dataEvo, 2018

Es recomendable aplicar el método de validación K-Cross con una porción del *dataset* que no sea menos de 100 y más de 3000 ejemplos (dataEvo, 2018).

Cuando un ejemplo entra en la red neuronal por primera vez, se ajustan los pesos y los *bias* de la red neuronal de tal manera que la próxima vez que otro ejemplo entre en la red neuronal esta pueda mejorar el nivel de predicción, este proceso se hace demasiadas veces y se denomina etapa de entrenamiento mediante “*Backpropagation*”, mientras más ejemplos ingresen a la red neuronal más definidos estarán los ajustes respecto a los pesos y *bias*, por lo tanto tendremos un mejor nivel de predicción y podremos decir que nuestra red neuronal está aprendiendo. Sin embargo, para que este proceso sea efectivo es necesario conocer las fórmulas y metodologías de optimización que permiten aprender y ajustar los pesos y *bias* de la red neuronal.

2.1.5 MÉTODOS DE OPTIMIZACIÓN.

Existen actualmente muchos métodos de optimización para las redes neuronales, sin embargo, las más utilizadas son tres:

- **Stochastic Gradient Descent (SGD):** Es el método básico de optimización de las redes neuronales y otros algoritmos de *machine learning*. Las fórmulas para actualizar los pesos y las *bias* están definidas como:

$$\bullet \quad W = W - \alpha \frac{\partial J}{\partial W}$$

$$\bullet \quad b = b - \alpha \frac{\partial J}{\partial b}$$

Donde tenemos que:

- W = el valor de los pesos de la capa de salida (es una matriz).
- b = el valor del *bias* (es un vector).
- α = el valor del *Learning rate* (por defecto en 0.002).
- $\frac{\partial J}{\partial W}$ = la derivada de la función de pérdida respecto a los pesos.
- $\frac{\partial J}{\partial b}$ = la derivada de la función de pérdida respecto a los *bias*.

- **Root Mean Square Propagation (RMSProp):** es similar el método de SGD sin embargo converge mucho más rápido, esto sucede porque no cuenta los mismos valores de los ejes cuando estamos optimizando.

$$\bullet \quad W = W - \alpha \frac{\frac{\partial W}{\partial W}}{\sqrt{S_{\partial W} + e^{-9}}}$$

$$\bullet \quad b = b - \alpha \frac{\frac{\partial b}{\partial b}}{\sqrt{S_{\partial b} + e^{-9}}}$$

Donde tenemos que:

- $\sqrt{S_{\partial W}}$ = es la raíz cuadrada respecto a la razón de cambio de W (razón de cambio pequeño).
- $\sqrt{S_{\partial b}}$ = es la raíz cuadrada respecto a la razón de cambio de b (razón de cambio grande).
- e^{-9} = es un valor estacionario 10^{-9}

- **Estimación adaptativa de momentos (ADAM):** literalmente es el método RMSProp con momentum, sin embargo es el método más usado de los anteriores dos mencionados.

$$\bullet \quad W = W - \alpha \frac{v \partial W}{\sqrt{s_{\partial W} + \epsilon^{-9}}}$$

$$\bullet \quad b = b - \alpha \frac{v \partial b}{\sqrt{s_{\partial b} + \epsilon^{-9}}}$$

- Donde tenemos que:
 - $v \partial W$ = es el valor de momentum respecto a los pesos 0.9 por defecto.
 - $v \partial b$ = es el valor de momentum respecto al *bias* 0.999 por defecto.

2.1.6 LA VARIABLE DEL LEARNING RATE.

Esta es una variable muy importante dentro de los métodos de optimización, tiene como objetivo actualizar los pesos y *bias* en cada iteración por medio del proceso de *backpropagation*, esta tiene un rango de valores flotantes desde el 0 hasta el 1, en las fórmulas matemáticas se lo denota como α , hay que tener un enorme cuidado al ajustarlo con valores grandes (0.1, 0.2, 0.3, etc.) pues es posible que nunca se pudiese alcanzar el valor mínimo de la función de pérdida, también hay que cuidar de aplicar valores demasiado pequeños (0.00001, 0.0002, etc.) pues en aquel caso tardaría demasiado en llegar al valor estimado consumiendo muchas épocas de entrenamiento y recursos computacionales. La forma más simple de calcular y ajustar esta variable es a través de la prueba y error.

Luego de ajustar los parámetros del Learning Rate y escoger el método de optimización que se ajuste a mi prototipo, obtendremos un nivel de asertividad mediante el entrenamiento, este valor hay que sustentarlo y probarlo mediante métodos de evaluación que se ajusten a mi proyecto.

2.1.7 MÉTODOS DE EVALUACIÓN PARA MI MODELO.

Para interpretar correctamente la predicción realizada por mi modelo de clasificación binaria (carcinomas o melanomas) se utilizarán las curvas ROC y las curvas de precisión-sensibilidad (*Precision-Recall*).

La salida del modelo de predicción suele ser un valor flotante entre 0 y 1 (probabilidad), este resultado se evalúa según un umbral para escoger a que tipo de clase pertenecería el valor de entrada, por ejemplo podríamos pronosticar el tipo de clase perteneciente a un valor de entrada si utilizamos un umbral de 0.5 y decidiendo que toda aquella predicción con una probabilidad inferior al umbral (<0.5) se considerará que pertenece a la clase 0 (carcinoma) mientras que toda aquella predicción con un valor igual o superior al umbral (0.5) se considerará que pertenece a la clase 1 (melanoma). Este umbral puede ser ajustado para modificar el comportamiento de nuestro modelo para un problema en específico. La modificación del umbral serviría si se quisiera inclinar el modelo hacia una de las dos clases, si hacemos esto estaríamos orientando el modelo hacia una clase en particular y serviría para tratar de minimizar los fallos de predicción de una determinada clase.

Tanto las curvas ROC y la de precisión-sensibilidad están basadas en la matriz de confusión, para explicar mejor sus estructuras es necesario conocer los siguientes parámetros (ver imagen 3.18):

- **Verdadero positivo (VP):** se hace referencia a cuando nuestro prototipo diagnostique un caso de melanoma (1) y que efectivamente el paciente cuente con un melanoma (1).
- **Verdadero negativo (VN):** se hace referencia a cuando nuestro prototipo diagnostique un caso de carcinoma (0) y que efectivamente el paciente cuente con un carcinoma (0).
- **Falso positivo (FP):** hace referencia a cuando nuestro prototipo diagnostique un caso de melanoma (1) y que el paciente cuente con un carcinoma (0).

- **Falso negativo (FN):** este punto es el más importante y el que más se tiene que reducir, se hace referencia a cuando nuestro prototipo diagnostique un caso de carcinoma (0) y que el paciente cuente con un melanoma (1).

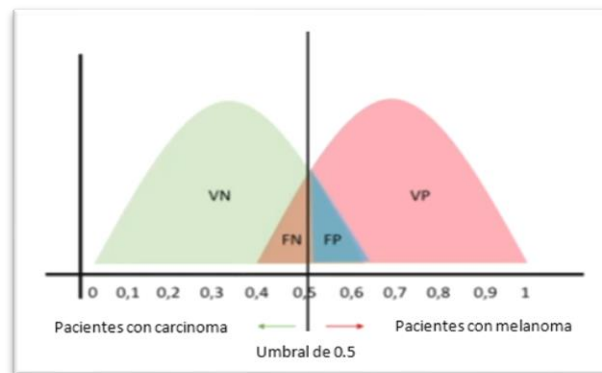


Imagen 2.18: se observa las características de VP, VN, FP y FN separados por medio de un umbral de 0.5.

2.1.7.1 EVALUACIÓN DEL MODELO POR MEDIO DE LA CURVA ROC.

Es una gráfica que enfrenta los valores de falsos positivos (situados en el eje x) con los verdaderos positivos (situados en el eje y). Estos valores los va obteniendo en función de una serie de umbrales definidos entre 0 y 1, en otras palabras, comparara una posible falsa alarma contra la tasa de éxito.

La tasa de falsos positivos se calcula como el número de positivos verdaderos (VP) divididos entre el número de positivos verdaderos (VP) sumado los casos de falsos negativos (FN), el resultado describirá que tan bueno es nuestro modelo prediciendo las clases positivas cuando la salida real es positiva. También se conoce esta tasa como **sensibilidad**.

$$sensibilidad = \frac{VP}{VP + FN}$$

La **especificidad** es la inversa de la tasa de falsos positivos. Se obtiene dividiendo el número total de casos de verdaderos negativos entre la suma de los verdaderos negativos y los falsos positivos.

$$especificidad = \frac{VN}{VN + FP}$$

También debo aclarar que la curva de ROC es bastante útil para identificar al mejor modelo según su rendimiento como clasificador, además el área que está por debajo de la curva se la denomina AUC, esta se la utiliza como índice para la calidad del modelo, cuanto más esté hacia la izquierda la curva ROC, más área existirá contenida bajo ella y, por ende, mejor será el clasificador.

Dentro de la gráfica (ver imagen 2.19) tendremos que tomar en cuenta los siguientes resultados:

- valores pequeños en el eje X indican pocos casos de falsos positivos y muchos de verdaderos negativos.
- valores grandes en el eje Y indican casos elevados de verdaderos positivos y pocos de falsos negativos.

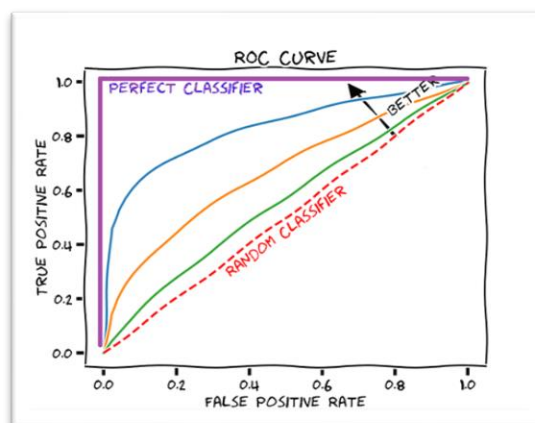


Imagen 2.19: se observa la gráfica de la curva ROC, donde se aprecia diferentes modelos aplicados para resolver un problema (arco azul, naranja, verde), la curva ideal de asertividad 100% (curva morada) y la curva sin entrenamiento (línea roja).

Fuente: themachinelearners.com, 2018.

2.1.7.2 EVALUACIÓN DEL MODELO POR MEDIO DE LA PRECISIÓN-SENSIBILIDAD.

La **precisión** se calcula como el número de verdaderos positivos (VP) entre la suma de verdaderos positivos (VP) y de falsos positivos (FP), esta describe que tan bueno es el modelo a la hora de predecir las salidas de la clase positiva. Mientras tanto que la **sensibilidad** se calcula como el número de verdaderos positivos (VP) divididos entre la suma de verdaderos positivos (VP) y de falsos negativos (FN), es la capacidad de poder detectar correctamente la enfermedad entre los pacientes.

En resumen, la curva de precisión-sensibilidad enfrenta la **precisión** (eje y) con la **sensibilidad** (eje x) para diferentes umbrales. La curva de precisión-sensibilidad es especialmente útil en aquellos casos en los que tengamos clases desbalanceadas, en donde suele existir muchos ejemplos dentro del *dataset* que tengan diagnósticos negativos (clase 0 carcinoma) y muy pocos positivos (clase 1 melanoma) o viceversa.

2.2 CÁNCER DE PIEL.

Es un padecimiento de la piel donde existe el crecimiento anormal y descontrolado de las células de renovación en la piel.

El cáncer de piel tiene muchas manifestaciones, partiendo de ese punto, esta afección clínica tiene dos divisiones:

- **El carcinoma:** es una protuberancia que crece, aumenta de tamaño (véase imagen. 2.20) y si no se detecta a tiempo puede producir graves mutilaciones o cicatrices al momento de extirparse. También se debe recalcar que no causa el deceso del paciente.
- **El melanoma:** aparece como un lunar o una peca que crece, se oscurece, cambia de forma y sangra (véase imagen. 2.21), también si no se detecta y se extirpa a tiempo puede producir la mortalidad del paciente hasta en un 50% en los primeros 5 años de aparición (IVO fundacion de oecologia valeciana, 2015).



Fig. 2.20. Fotografía de un carcinoma.

Fuente: mdc_manuals.com, 2021



Fig. 2.21. Fotografía de un melanoma.

Fuente: salud180.com, 2018

2.2.1 EL ORIGEN DEL MELANOMA.

Todo comienza con los melanocitos, que son responsables de producir melanina, la cual en condiciones normales ayuda a la protección de la piel de los daños causados por los rayos UV (ultra violeta).

El melanoma es un tumor maligno de los melanocitos que se desarrolla como resultado a factores intrínsecos como la genética y factores extrínsecos como la sobre exposición a los rayos solares.

La radiación UV (ultra violeta), genera dímeros de Timina (una de las 4 bases del ADN) formando mutaciones genéticas, al ocurrir la división celular, estas mutaciones genéticas son transmitidas a nuevas generaciones de células, originando un crecimiento descontrolado y la formación del tumor celular.

En contraste a los melanocitos de una persona sana, los cuales responden a un limitado número de factores de crecimiento, las células del melanoma crecen de manera autónoma y secretan abundantes citoquinas (proteínas que controlan la actividad celular) y factores de crecimiento, esta producción aberrante les permite a las células cancerosas proliferar de forma independiente.

La producción de citoquinas por el melanoma induce a una proliferación tumoral que simula angiogénesis (formación de vasos sanguíneos) y la adhesión molecular celular

(formación y producción de tejidos) y genera una resistencia a las proteínas inhibitorias de la proliferación celular (cancer.org, 2019).

Esta resistencia genera inmunosupresión al bloquear la maduración y formación de células presentadoras de antígenos (producción de anticuerpos), que dan como resultado un déficit en la presentación de antígenos. Sin embargo, hoy en día existen diferentes tratamientos y terapias que ayudan a contrarrestar este mal.

La IL2 (interleucina 2) es una proteína que existe de forma natural en el cuerpo y juega un papel importante en la activación y creación de células T (anticuerpos que combaten el cáncer e infecciones). La terapia con IL2 artificial es utilizada como primera o segunda línea de terapia para el melanoma avanzado. Posee las mismas propiedades que la IL2 natural y ayuda a incrementar la creación de las células T y la síntesis en células B (células del sistema inmunitario), células NK (*natural killer*) (células que forman la primera línea de defensa ante patógenos externos) y linfocitos T citotóxicos (célula inmunitaria que destruye ciertas células), los cuales ejercen un papel crítico en la función del sistema inmune.

Una vez activado el linfocito T, este secreta citoquinas como factor de necrosis (muerte de un tejido) tumoral e interferón gama (anticuerpos que destruye cierto tipo de virus), ambos tienen efectos antitumorales (misistemainmune.es, 2015). Los linfocitos T citotóxicos y células NK atacan directamente a las células tumorales.

Es probable que la manipulación mediante fármacos del bloque CTLA4 (proteína que se encuentra en las células T) incremente la respuesta inmune, promoviendo así la proliferación de las células T y la liberación de citoquinas proinflamatorias y aumentando la respuesta de antígenos específicos tumorales (medicina y salud pública, 2021).

2.2.2 FACTORES QUE AUMENTAN EL RIESGO DE PADECER MELANOMA.

Uno de los factores para la aparición del melanoma en las personas es la sobre exposición a los rayos UV (ultra violeta) en partes expuestas al sol por periodos extendidos sin usar bloqueador solar o sombreros. Algunos casos que también pueden contribuir con la aparición de un melanoma son los factores hereditarios o genéticos y otros factores ambientales que todavía están por determinarse (cancer.org, 2019).

También aumentan los riesgos de contraer melanoma la exposición a luces artificiales que generan rayos A, B, UVB y UBA, como las cabinas bronceadoras.

2.2.3 CÓMO SE DIAGNOSTICA EL MELANOMA.

Los especialistas médicos dermatólogos tienen muchos métodos para diagnosticar el melanoma, uno de los más usados es el diagnóstico del A, B, C, D, E (véase Fig. 2.22).

- **A:** por asimetría en la forma, en otras palabras, que una mitad de la mancha o lunar sea diferente a la otra mitad.
- **B:** por borde irregular en el lunar o mancha en la piel.
- **C:** por color no uniforme, significa que la mancha o lunar posee más de un color teniendo combinaciones de marrón, negro, azul, rojo y blanco dentro del lunar o mancha en la piel.
- **D:** por diámetro de más de 6 milímetros, significa que el lunar o mancha en la piel sea más grande que otras manchas al su alrededor.
- **E:** por evolución rápida, en otras palabras, que esté creciendo de tamaño el lunar o mancha en la piel de manera veloz.

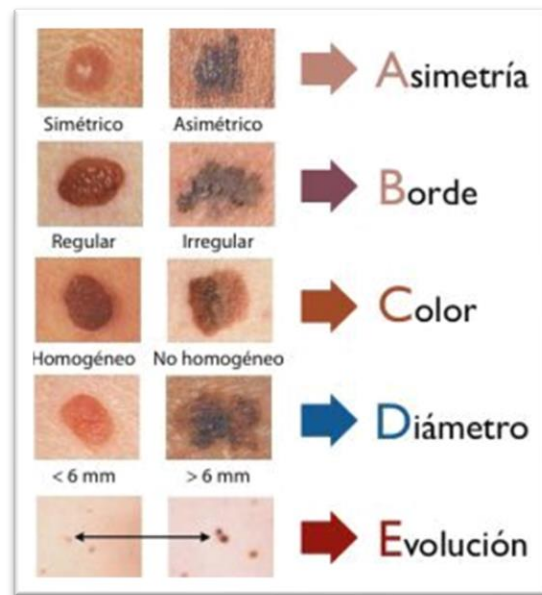


Fig. 2.22. Figura donde se muestra la diferencia entre un melanoma y carcinoma por medio del diagnóstico A, B, C, D, E.
Fuente: conicet.com, 2021

Otro método sería el de analizar las siguientes características (mskcc, 2022):

- ✓ Analizar si los lunares o manchas pudiesen sangrar a menudo.
- ✓ Si la mancha o lunar cuenta con bordes de color blanco.
- ✓ Escozor en la mancha o lunar.
- ✓ También preguntan al paciente si en el transcurso de que les apareció el lunar este cambio de color, de forma o tamaño.

2.2.4 TRATAMIENTO ANTE EL MELANOMA.

El especialista dermatólogo realiza los siguientes pasos cuando ya se confirmó que el paciente cuenta con un melanoma:

1. Primeramente, se realiza una biopsia excisional, es decir quitar todo el melanoma, pero sin ningún tipo de margen, esto se lo hace para enviárselo al patólogo.
2. Cuando recibimos la respuesta del patólogo tenemos dos opciones:

- 2.1. Melanoma in situ: es decir que todavía el melanoma se encuentra sobre la membrana basal y por lo tanto el pronóstico hacia el paciente será muy bueno. En este caso solamente se amplía la cicatriz 0.5 centímetros en todo el perímetro de la cicatriz y hacer revisiones periódicas al paciente.
- 2.2. Melanoma maligno o invasivo: es decir que en este caso el melanoma ya penetra la membrana basal, junto con esta respuesta el patólogo nos da el nivel de “Breslow”, esta es una medida donde el melanoma está incrustado en la piel y se la expresa en milímetros.
- 2.2.1. “Breslow” < 1 milímetro: se amplía la cicatriz en 1 centímetro en todo el perímetro de la cicatriz y se tiene que hacer revisiones periódicas al paciente.
- 2.2.2. “Breslow” > 1 milímetro: el pronóstico del paciente es malo, el paciente desarrolla metástasis linfática, por lo tanto, lo que se debe hacer es buscar las metástasis linfáticas o adenopatías palpables (bultos de piel) y ampliar la cicatriz en 2 centímetros en todo el perímetro de la cicatriz, una vez encontradas se realiza una biopsia y se realiza una linfadenectomía más un tratamiento con interferón esperando que sea solo una inflamación. Sin embargo, cuando no tiene adenopatías palpables se busca micro metástasis linfáticas por el método del ganglio centinela y pueden pasar dos cosas:
- 2.2.2.1. Que el ganglio centinela sea negativo, es decir que no se vea metástasis en esa zona.
- 2.2.2.2. Que el ganglio centinela sea positivo, se debe realizar una linfadenectomía más un tratamiento con interferón.

2.3. ESTADO DEL ARTE.

Existen algunas propuestas de sistemas que diagnostican lesiones en la piel humana, estas están basadas en diferentes criterios de reconocimiento como lunares y manchas, algunas de ellas intentan distinguir un único patrón como la forma geométrica de la lesión y otras buscan varios patrones como el color, el tamaño y el contorno de la lesión.

2.3.1. SISTEMA DE CLASIFICACIÓN DE IMÁGENES DE LUNARES UTILIZANDO VISIÓN POR COMPUTADORA.

Francisco Andrés Reales Castro “Las herramientas se moldean con cada década que transcurre llegando a utilizar herramientas imaginarias (digitales) que a veces son más útiles que las herramientas físicas”, Trabajo de grado de la universidad Javeriana en Bogotá Colombia (javeriana, 2017).

Objetivo: la creación de un sistema capaz de poder clasificar entre un **lunar** cancerígeno y uno benigno mediante visión por computadora.

Muestra: para el proyecto se utilizó 66 fotografías de lunares malignos y 104 de lunares benignos, haciendo un total de 170 muestras para el proceso de entrenamiento, todas de tamaño de 20x20 píxeles.

Antecedentes: el ingeniero afirma que los médicos dermatólogos tienen un 75-84% de índice de asertividad al momento de diagnosticar un lunar maligno, es por ello que se decidió implementar un sistema capaz de reconocer lunares malignos en la piel de los pacientes mediante una fotografía tomada al área afectada, este sistema brindará un apoyo al diagnóstico del médico, según el ingeniero el sistema nunca deberá usarse como diagnóstico directo.

Funcionamiento: el sistema funcionara cuando se tome una fotografía al área afectada del paciente, esta área deberá estar iluminada y en el mejor de los casos sin obstrucción (sin pelos corporales, sin tatuajes y sombras), una vez tomada la imagen esta pasara por un embudo convolucional llegando a obtener datos primordiales para la clasificación, el ingeniero utilizo los métodos de clasificación de k-vecinos cercanos, SVM (soporte de máquinas vectoriales) y redes neuronales, dando un índice de error relativamente pequeño el método de K-vecinos cercanos. El resultado de la clasificación se expresará en binario (0 benigno y 1 maligno).

Factores: entre los factores asociados de la creación del sistema Clasificador de Lunares, el ingeniero toma en cuenta la reducción de la capa de ozono por medio del calentamiento global, esta reducción aumenta significativamente los rayos ultravioleta en la atmosfera terrestre, lo cual genera un aumento de posibles casos de lunares malignos, otro factor que toma en cuenta el ingeniero es el artículo publicado en la revista *Caris Life Sciences* (carislifesciences, 2018), el cual explica que en el país de los E.E.U.U se triplicaron los casos de lunares malignos desde el año 2006 hasta el año 2017, haciendo que sea la 5° enfermedad más letal respecto al cáncer.

Resultados: el ingeniero logro un resultado de efectividad de su sistema en un 94,9%, esto utilizando el método de clasificación de K-vecinos cercanos.

Conclusiones: el clasificador con mejor desempeño es el K-vecinos cercanos, con $k=1$, ya que solo se hace la comparación con el vecino más cercano, conforme aumenta el valor de k para el clasificador de K-vecinos cercanos disminuye la efectividad (mientras más ejemplos exista mayor será el error), también se obtuvo la conclusión de que el sistema funciona como herramienta de control del dermatólogo para el paciente, pero nunca como diagnóstico directo.

2.3.2. DETECCIÓN DE MELANOMAS DE PIEL MALIGNOS MEDIANTE PROCESAMIENTO DIGITAL DE IMÁGENES USANDO REDES NEURONALES CONVOLUCIONALES.

Sebastián Riaño Borda, “Tener parametrizados todos los valores acerca de una enfermedad permite la creación de soluciones controladas”, Proyecto de grado de maestría en la universidad Santo Tomas, Bogotá Colombia (usta, 2022).

Objetivo: la creación de un sistema capaz de detectar y diagnosticar un posible caso de melanoma tomando en cuenta una fotografía del área de la lesión del paciente.

Muestra: para el proceso de entrenamiento se utilizó **600** imágenes de tamaño de 100x100 píxeles, las cuales están divididas entre 300 imágenes de melanomas y 300 imágenes de carcinomas, además de utilizar 190 imágenes para la etapa de prueba.

Factores: el ingeniero Sebastián Riaño Borda creó este proyecto por la necesidad de mejorar las condiciones y calidad de vida a la humanidad llegando a una era donde la tecnología es capaz de interactuar con otras áreas de estudio, permitiendo que algunas tareas sean desarrolladas por dispositivos inanimados, gastando menos tiempo y manteniendo una mayor precisión.

Funcionamiento: el proyecto propone diseñar una RN (red neuronal) para el reconocimiento de melanomas, mediante el uso de la técnica de RNC (redes neuronales convolucionales), el ingeniero explica que esta técnica es mayormente utilizada en VC (visión artificial), aplicada en el reconocimiento de patrones sobre lunares en la piel y determinar la existencia de un melanoma maligno o no, todo esto a partir de un *dataset* (colección de datos) limitado. También utiliza las capas de convolución y *pooling* apiladas entre sí para formar la red propuesta, con un clasificador de 1 o 2 salidas, el ingeniero utiliza un *dataset* gratuito proporcionado por *International Skin Imaging Collaboration* (ISIC).

Resultados: según el ingeniero se llegó a un resultado con una precisión del **87.82%** de asertividad.

Conclusiones: teniendo en cuenta los experimentos y los resultados obtenidos en el proyecto de maestría, el ingeniero afirma que la mejor combinación de funciones de activación para extraer los parámetros de las imágenes y poder obtener la mejor predicción, es usar **Relu** en las capas convolucionales, con el fin de obtener una mejor y más rápida extracción de información que le permita tener un aprendizaje más rápido y **softmax** en la salida para así obtener la probabilidad que tiene cada caso de pertenecer a una clase o la otra.

Conclusiones: teniendo en cuenta los experimentos y los resultados obtenidos en el proyecto de maestría, el ingeniero afirma que la mejor combinación de funciones de activación para extraer los parámetros de las imágenes y poder obtener la mejor predicción, es usar **Relu** en las capas convolucionales, esto con el fin de obtener una mejor y más rápida extracción de información que permita tener un aprendizaje más rápido y utilizar la función de activación de **softmax** en la capa de salida, con el fin de obtener una clasificación binaria.

CAPÍTULO 3: MARCO APLICATIVO.

En este capítulo se explicará el desarrollo paso a paso del prototipo detector de melanomas, para dicho proceso se utilizará la metodología Buchanan, la cual está compuesto de cinco fases, la mayoría de estas fases sirve para la recopilación y abstracción de la información provenientes de los expertos.

También se utilizará la metodología Kanban evocada en la fase cuatro de la metodología Buchanan (implementación) donde se abstraerá y aplicará lo más importante de los conocimientos adquiridos para luego programar el prototipo, para ello se tiene que empezar identificando los problemas y los actores que intervienen en el proceso de recopilación de la información.

3.1. FASE 1: IDENTIFICACIÓN.

En esta fase se formula el problema a tratar y luego se investiga profundamente las posibles soluciones para resolver el problema. Para esto deberé empaparme de conocimientos sobre medicina (dermatología) e Inteligencia artificial (redes neuronales convolucionales), estos conocimientos los iré adquiriendo mientras más publicaciones en revistas, libros, tesis y proyectos lea (historia del arte y antecedentes). Sin embargo, hay que priorizar los conocimientos que tiene un experto del tema (medico dermatólogo u oncólogo), pues estos conocimientos se ganaron a través del tiempo y experiencia al tratar casos de pacientes.

También se identifica a los participantes y sus diferentes roles (componentes del sistema), además de los recursos (necesarios o disponibles) para la construcción del prototipo y por ultimo las metas propuestas a alcanzar, para este último punto mencionado es necesario empezar por identificar el problema que queremos resolver.

3.1.1. IDENTIFICACIÓN DEL PROBLEMA.

Para esta sección se utilizará el problema ya formulado en el punto 1.3 (formulación del problema) el cual describe:

¿Cómo identificar si el paciente pudiese tener un posible caso de melanoma, sin el uso de herramientas tradicionales de laboratorio?

Para poder resolver este problema es necesario contar con conocimientos de medicina, más explícitamente de conocimientos dermatológicos, es por ello que es fundamental contar con un médico experto en el área.

3.1.2. BÚSQUEDA DE UN EXPERTO.

En esta etapa se hace referencia a la búsqueda de un experto médico (dermatólogo) que esté dispuesto en colaborar en la construcción del prototipo, aportando conocimientos y experiencias en el momento de diagnosticar un posible caso de melanoma en un paciente.

Para ello se cuenta de la experiencia del médico Cristian Quiroga, quien posee la especialidad de médico cirujano dermatólogo en la clínica QuiroDerma, el cual accedió a colaborar aportando pautas y experiencias de su trabajo, así también de tareas idóneas para ser realizadas por el prototipo. Sin embargo, existen más componentes expertos (personas que influyeron en la construcción del prototipo) no solo con conocimientos en medicina sino también de programación.

3.1.3. IDENTIFICACIÓN DE COMPONENTES.

En esta sección se describen a los participantes que aportaron con conocimientos, experiencias y metodologías para la elaboración del prototipo:

- **experto en el área:** se hace referencia al personal médico de un centro de salud puntualmente señalando a los doctores dermatólogos, estos aportaron experiencias referentes a casos en el momento del diagnóstico de un posible melanoma, este

punto se mencionó en la parte de “Búsqueda de un experto”. También en esta sección entran los expertos en programación y desarrollo de sistemas, en mi caso se cuenta con el apoyo del grupo de estudio “NeuroTech” de la universidad Católica Boliviana y el docente M Sc. Reynaldo Zeballos Daza.

- **ingeniero del conocimiento:** es la persona que se encarga de realizar el prototipo con la información obtenida de los diferentes expertos, esta información se obtuvo mediante cuestionarios y entrevistas personales.
- **usuarios:** son personas que probaron el funcionamiento del prototipo, sus aportes son las recomendaciones que servirán de retroalimentación para pulir el prototipo. En esta parte incluiré solo a los médicos dermatólogos.

Una vez que hayamos identificado a los componentes y obtenido la información necesaria proveniente de los expertos, será hora de identificar los recursos para el desarrollo del prototipo (tanto software como hardware).

3.1.4. IDENTIFICACIÓN DE RECURSOS.

En esta sección se describen a los recursos necesarios y los recursos disponibles para la elaboración del prototipo:

- **recursos necesarios:** son recursos digitales o recursos tangibles indispensables para la construcción del prototipo, estos recursos son:
 - **cámara web:** para obtener fotografías digitales de las lesiones del paciente.
 - **computadora:** para poder procesar la información obtenida de la cámara web y programar el prototipo.
 - **datasets:** para poder entrenar el prototipo. Mientras más grande sea el conjunto de datos (diagnósticos de melanomas positivos y negativos) más confiable será el nivel de predicción.
- **recursos disponibles:** son elementos que no necesariamente tienen que estar presentes en la construcción del prototipo, sin embargo, el uso de uno de estos

recursos puede aportar positivamente en la salida (disminuir el error de predicción o aumentar el nivel de asertividad):

- **lenguajes de programación:** para programar el prototipo utilizare el lenguaje de programación de Python3, sin embargo, es posible realizarlo con otros tipos de lenguajes como Java o C++, en mi caso utilizo Python3 pues contiene muchas librerías que aportan a la construcción del mi prototipo.
- **iluminación:** para poder obtener una fotografía donde se muestre con claridad la zona afectada del paciente, es necesario contar con buena iluminación en el ambiente, esta se consigue mediante luz blanca (diodos led) o tubos fluorescentes, si el consultorio del médico experto cuenta con iluminación adecuada (ventanas bien iluminadas), no será necesario otras fuentes de luz.

3.1.5. IDENTIFICACIÓN DE LAS METAS.

En esta sección se idealizan las metas que deseamos generar al desarrollar el prototipo, estas están definidas en los puntos “1.5.2 objetivos específicos” y “1.5.1 objetivo general”.

3.2. FASE 2: CONCEPTUALIZACIÓN.

Esta fase es fundamental, pues aquí es donde se menciona el conocimiento a partir de distintas fuentes, estas están mencionadas en la sección “3.1 Fase: identificación”, por mi parte en esta sección mencionare solo el conocimiento adquirido mediante cuestionarios y entrevistas a expertos (médicos y programadores) ya que el conocimiento adquirido mediante lecturas esta descrito en la sección de Historia del arte y Antecedentes.

Los siguientes puntos son conocimientos extraídos del médico experto dermatólogo, hacia la detección de un supuesto caso de melanoma en un paciente:

- existen dos tipos de carcinoma, uno se denomina carcinoma benigno, en donde al paciente solo se le realiza un tratamiento de extirpación y constantes visitas al médico para confirmar que no volvió a propagarse en otras zonas de la piel. Sin embargo, el otro carcinoma se lo conoce como maligno o melanoma, es en este caso donde el carcinoma atraviesa la membrana basal (capa que separa la dermis de la epidermis) del paciente, en este caso tanto la detección y tratamiento es crucial para el paciente.
- **¿cómo reconocer el melanoma?:** es importante verificar los siguientes elementos que posee un lunar o mancha en la piel, esto para determinar un posible melanoma:
 - **la asimetría:** un lunar con dos mitades diferentes podría ser un lunar maligno.
 - **los bordes:** Un lunar con bordes irregulares, dentados o estrellados podría ser un lunar atípico y aumentaría la sospecha de la presencia de un melanoma.
 - **el color:** los colores de un lunar o una mancha normal son homogéneos, marrón, negro o un tono oscuro, sin embargo, si este posee colores distintos como ser la conformación del marrón, rojo, negro o tonos azulados podría ser un candidato a un posible melanoma.
 - **el diámetro:** cualquier lunar o mancha más de 5 milímetros tiene que ser revisado por un dermatólogo.
 - **la evolución:** se refiere que exista cambios en el lunar como el cambio repentino del color, la aparición de un borde blanco, el sangrado de un lunar o el escozor de un lunar.
- **¿qué características posee un lunar estable o normal?:** un lunar normal siempre permanecerá estable de cambios morfológicos, tendrá un crecimiento lento y este será de no más de 5 milímetros.
- **¿qué herramienta se utiliza en la actualidad para detectar un posible caso de melanoma?:** en la actualidad se cuentan con muchas herramientas para detectar un melanoma, el más utilizado se denomina “dermatoscopio”, es una lente similar

a un microscopio que sirve para analizar los rasgos de un lunar o una mancha en la piel, de esta manera se puede definir si un lunar o mancha con rasgos sospechoso pudiese ser un posible melanoma o no, sin embargo el costo de adquisición de una herramienta como esta es elevada y también hay que tener mucha cautela al manipular esta herramienta pues cualquier tipo de caída o golpe pudiese quebrar el cristal de aumento lo que generaría en un cambio de cristal y esto demoraría mucho tiempo y dinero, pues las piezas de recambio se encuentran en el extranjero.

- **¿cómo prevenir el melanoma?:** las formas más frecuentes de prevenir el melanoma pudiesen ser:
 - la protección solar correcta (mediante protector solar o usar sombreros y camisas de mangas largas), además de evitar las fuentes artificiales de rayos ultravioletas.
 - en segundo lugar, la autoexploración del lunar o mancha, fijarse que este no crezca más de 5 milímetros de diámetro y que tenga una forma casi circular.
 - por último, la visita al dermatólogo por lo menos 1 vez al año, el que nos dirá con más certeza si es un lunar o un posible candidato a melanoma.

Los siguientes puntos son conocimientos extraídos de los expertos programadores, con respecto al uso de IA (inteligencia artificial) en el campo de la medicina:

- hoy en día existen múltiples aplicaciones de la IA al campo de la medicina, una de ellas precisamente es la asistencia a la telecirugía.
- a veces al usar una cantidad enorme de épocas para entrenar el modelo no siempre es efectivo, a veces puede empeorar el resultado, devorar la memoria y el procesador de la computadora.
- es posible enseñar a una maquina a ver y extraer patrones de una imagen para crear un diagnóstico, pero nunca superará a la experiencia de los médicos más veteranos, por lo tanto, este proyecto solo se realizará con fines demostrativos.

- al contar con un numero pequeño de imágenes para el entrenamiento (menor a 1000) el error de la predicción será bajo y casi siempre tendremos un nivel de asertividad alto (mayor al 85%), sin embargo, si se cuenta con un numero grande de imágenes para el entrenamiento (mayor a 3000) el error crecerá y bajará el nivel de asertividad, por lo tanto, habría que ajustar ciertos parámetros en la red neuronal convolucional para que se llegue a un buen nivel de aceptación.
- no es recomendable usar Matlab para la convolución de imágenes, esto porque el programa Matlab es muy pesado de ejecutar.
- todas las imágenes deberán tener el mismo tamaño de resolución, esto con tal de que no varíe los parámetros de ajuste dentro del modelo convolucional.
- cuando se obtiene un conjunto de datos (*dataset* de imágenes) hay que dividirlo en un porcentaje para la etapa de entrenamiento y otra para la de prueba, lo más habitual es una proporción de un 70% para el entrenamiento y un 30% para las pruebas.

3.3. FASE 3: FORMALIZACIÓN.

En esta etapa el desarrollador juega un papel muy importante pues deberá formalizar los conocimientos obtenidos previamente de diferentes expertos. Esta tarea definirá una mejor organización del conocimiento al momento de la implementación del prototipo.

Como ejemplo tenemos algunas descripciones formalizadas del conocimiento adquirido en los anteriores puntos:

- el medico dermatólogo buscara indicios de un posible melanoma en la piel del paciente, esto lo realizara gracias a una herramienta llamada dermatoscopio.
- el medico dermatólogo al buscar indicios de un posible melanoma, estará buscando características mencionadas del diagnóstico del A, B, C, D, E.
- el medico dermatólogo una vez que obtiene su diagnóstico con respecto a la lesión del paciente, este necesitara una segunda opinión respecto al caso, es aquí donde

entra el prototipo de detección de melanomas, generando un segundo diagnostico para apoyar o refutar el diagnóstico del médico dermatólogo.

- entonces para generar un diagnóstico confiable, el prototipo detector de melanomas tendrá que identificar cada característica peculiar que conforma una lesión de un melanoma o carcinoma (casi imitando a un médico dermatólogo), esto se logra gracias al entrenamiento del prototipo, este entrenamiento se basa en identificar y agrupar cientos de características de un caso a la vez (melanoma o carcinoma), este proceso se mejora mientras más ejemplos de casos tengamos y la organización de la estructura que utilicemos para el modelo.

Una vez identificados los pasos que realiza un médico dermatólogo a la hora de diagnosticar un posible caso de melanoma, es hora de formalizar el prototipo para su posterior implementación.

Para poder obtener un diagnóstico del prototipo, primeramente se obtiene una imagen de la zona afectada del paciente, esta imagen estará sometida al proceso de convolución por varios *kernels* (ver figura 3.1) y se obtendrá una matriz con características resaltantes de la imagen (son las características peculiares de cada caso), luego estas pasaran a formar un “array” de datos las cuales se convertirán en la entrada para la red neuronal, finalmente se obtendrá un resultado(este estará en binario), si la salida tuviese el valor cercano a 1 significa que existen más características que lo asemejan a un posible melanoma, pero si la salida fuese más cercano a 0 seria todo lo contrario.

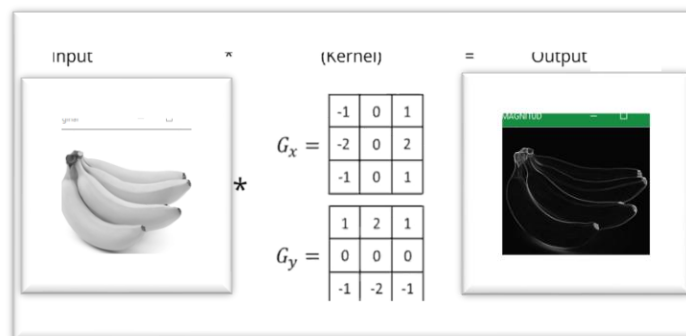


Figura 3.1. Se observa una imagen sometida a solo “dos” kernels, el resultado es la obtención del contorno del objeto.

Luego de obtener la información en la fase de conceptualización se realiza una tabla (tabla 3.1) donde se describe los pasos de manera resumida que servirán como guía en la fase de implementación.

ETAPAS DEL PROTOTIPO DE RECONOCIMIENTO DEL MELANOMA	IMPLEMENTACIÓN DE LAS ETAPAS EN EL DESARROLLO DEL SISTEMA.
ADQUISICIÓN	Para la etapa del entrenamiento se utilizó una gran cantidad de imágenes que representan el <i>dataset</i> de mi prototipo, estas imágenes fueron obtenidas de dos fuentes confiables, la primera fuente es el Ingeniero Electrónico Francisco Reales de la universidad Javeriana en Colombia, el cual me facilitó una colección de más de 3000 diagnósticos en imágenes de melanomas y carcinomas, este repositorio se encuentra en: https://mega.nz/file/9dRQ2A4b#uZBdeI24AfuRaaRxRDwc2S0XZd4KPGbLRXUroGkQWjI El segundo <i>dataset</i> que encontré fue de la universidad de Harvard llamada <i>DataVerse</i>, esta colección de imágenes contiene alrededor de más de 14000 diagnósticos en imágenes referentes a melanomas y carcinomas, esta se encuentra en el siguiente enlace: https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/DBW86T
HERRAMIENTAS PARA EL DESARROLLO DEL SISTEMA	En esta etapa se definen las herramientas digitales y físicas para implementar el prototipo: • Python (TensorFlow, OpenCv, Imutils). • Anaconda Navigator (Spyder, Jupyter Notebook). • TKinter para el entorno gráfico. • Datasets de diagnósticos en formato de imágenes. • Computadora con cámara web.
PREPROCESAMIENTO	Los diagnósticos estarán en formato de imagen, estos están separados en carpetas de acuerdo a su clasificación (melanoma y carcinoma), todas tienen resoluciones distintas (tamaños no uniformes) y portan etiquetas (nombres de pacientes y códigos hospitalarios), por lo tanto, hay que priorizar en realizar una limpieza de datos antes de introducirlas en el prototipo. Luego se tiene que separar estas imágenes de acuerdo a un porcentaje del 70% para el entrenamiento y 30 % para la prueba del prototipo, tratando de llegar a la siguiente estructura: ○ Test (30% de las imágenes) ▪ Benigno (15% de las imágenes). ▪ Maligno (15% de las imágenes). ○ Train (70% de las imágenes) ▪ Benigno (35% de las imágenes). ▪ Maligno (35% de las imágenes).
CONVOLUCIÓN	En esta tarea, se estructura el proceso de convolución para las imágenes de entrada, este proceso se lo realiza principalmente para minimizar el tamaño de la matriz original y obtener las características más relevantes de la imagen de entrada, las capas donde se añadirán ente proceso serán apiladas de forma secuencial.

RED NEURONAL	En esta sección crearemos la estructura de la red neuronal, esta contara con una capa de entrada, capas ocultas y una capa de salida. La salida de la tarea de convolución será la entrada vinculada a la primera capa de nuestra red neuronal, luego estos pasaran a ser clasificados por las neuronas y funciones de activación dentro de las capas ocultas y al final obtendremos una salida con representación binaria.
DISTRIBUCIÓN Y ENTRENAMIENTO DEL MODELO	En esta tarea se desarrollará la distribución y etiquetación de los datos para los procesos de entrenamiento y prueba. Una vez las imágenes se hayan distribuido y etiquetado, estas se almacenarán en una lista la cual estará ordenada de manera aleatoria, el fin de esto es para obtener un mejor ingreso al modelo. Finalmente es en esta tarea donde el modelo se entrenará con los datos de prueba y entrenamiento.
INTERPRETACIÓN DEL RESULTADO	En esta tarea se realizará la interpretación del resultado del modelo al insertar una imagen nunca antes vista por el modelo, el resultado será un numero Binario (0 o 1).

Tabla 3.1: Tabla donde se describe los pasos formalizados para la implementación del prototipo.

3.4. FASE 4: IMPLEMENTACIÓN.

Una vez concluida la fase de formalización, podemos empezar a implementar nuestro prototipo de reconocimiento de melanomas. Para ello utilizaremos la tabla 3.1 como un modelo guía en esta fase, recordemos que el método Kanban utiliza como entradas una serie de “tareas” definidas que en mi caso corresponderá a la columna de “etapas del prototipo de reconocimiento del melanoma” correspondientes a la tabla anterior.

3.4.1. TAREA 1: ADQUISICIÓN.

Cuando se trabaja con *machine learning* y más explícitamente con redes neuronales, es necesario de contar con un gran número de ejemplos que sirvan de datos de entrada, mientras más ejemplos se tenga mucho mejor será la predicción al final del prototipo, sin embargo antes de que estos ejemplos se conviertan en datos de entrada del prototipo se tiene que realizar la etapa de limpieza de datos, pues es necesario que todos estos ejemplos tengan una uniformidad (tamaño o resolución) al momento de introducirse como entradas del prototipo.

Al descargar estos ejemplos en las referencias ya mencionadas en la tabla 3.1 Adquisición, podemos observar que de la primera referencia se obtuvieron más de 3000 imágenes, todas con resolución de 224x224 pixeles, estas están divididas en carpetas de entrenamiento y

prueba y cada carpeta contiene dos subcarpetas donde encontramos imágenes de melanomas y carcinomas.

Al hacer clic en la segunda referencia podemos observar que nos redirige a la página oficial del *dataverse* de la universidad de Harvard, en donde descargamos los archivos [HAM10000_images_part_1.zip](#) y [HAM10000_images_part_2.zip](#) que contienen alrededor de más de 14000 imágenes de melanomas y carcinomas, estas imágenes poseen diferentes tamaños y también están etiquetadas con identificadores de pacientes y hospitales.

Con toda esta información podemos concluir que la fase de adquisición esta completa, pues ya contamos con diagnósticos de melanomas y carcinomas provenientes de fuentes confiables, solo faltaría realizar la limpieza de los datos para que tengan una cierta uniformidad, pero para realizar esa tarea es necesario conocer las herramientas y librerías de programación que se utilizaran para estructurar el prototipo.

3.4.2. TAREA 2: HERRAMIENTAS PARA EL DESARROLLO DEL SISTEMA.

Para desarrollar en prototipo detector de melanomas utilice Python3 con la versión v.3.7.13, el motivo de la versión es que algunas librerías son incompatibles con versiones superiores o inferiores, estas están descritas de la siguiente manera:

- **TensorFlow v.1.15:** la biblioteca *TensorFlow* sirve para construir, entrenar redes neuronales y descifrar patrones.
- **Scikit-learn:** la biblioteca *scikit-learn* contiene muchas herramientas eficientes para aprendizaje automático y modelado estadístico, incluyendo la clasificación, regresión, agrupación, y reducción de dimensionalidad.
- **Opencv:** más explícitamente *opencv-contrib-python* v.4.2.0.32, la biblioteca *Opencv* sirve para la visión artificial y es libre, también lee imágenes en el canal BGR, las cuales se pueden manipular en forma de matriz.

- **Scikit-image:** esta biblioteca es una colección de algoritmos para el procesamiento de imágenes en Python.
- **Keras:** la biblioteca *Keras* está orientado para *Deep Learning*, sirve para crear redes neuronales, proporciona bloques modulares para luego desarrollar proyectos complejos.
- **Sequential:** la biblioteca *Sequential* permite que las capas que creemos se agreguen en secuencia, es un modelo básico.
- **Dense:** la librería *Dense* sirve para recibir una entrada unidimensional de un array proveniente de la imagen, sirve muy bien para imágenes pequeñas (por eso uso 224x224 pixeles) y es la que añade capas en la red neuronal.
- **Convolution2D:** esta librería sirve para capturar imágenes, realizar operaciones con diferentes *Kernels* y crear datos más digeribles para las capas posteriores.
- **MaxPooling2D:** esta librería aplica un filtro que reduce la dimensión de la imagen de entrada, este proceso lo realiza escogiendo el valor más grande dentro de una ventana dimensionada (3x3, 5x5 o 7x7 pixeles).
- **Flatten:** esta librería convierte los elementos de la matriz (donde existen los caracteres más significativos de la imagen) en un array plano de datos, esto para que se asimile en la capa de entrada de la red neuronal.
- **Numpy:** esta librería sirve para crear y hacer operaciones entre vectores, listas y matrices.
- **Glob:** es una librería que devuelve una lista con todas las rutas de los elementos almacenados en el lugar seleccionado dentro del sistema operativo.
- **Os:** es una librería que realiza operaciones dentro del sistema operativo como crear carpetas, listar contenidos de la carpeta y muchas más.
- **Shuffle:** esta librería se encarga de realizar un proceso aleatorio de datos (mezcla datos internos mediante un pivote simulando aleatoriedad), es un barajeo de forma aleatoria de los datos, similar a las cartas de un casino.
- **Imutils:** es una librería que permite redimensionar (agrandar o disminuir) una imagen, tratando de conservar su estándar de calidad.

- **TKinter:** es una librería que permite crear un entorno grafico amigable utilizando Python.

Todas estas librerías y bibliotecas deberán aplicarse bajo una IDE (entorno de desarrollo integrado) el cual deberá trabajar con Python3 y además ser intuitivo para el usuario, para ello encuentre la suite (un programa compuesto de otros sub programas independientes) de “Anaconda Navigator” en su versión 2.1.3, el cual posee varios IDEs que permiten trabajar con Python3 de manera intuitiva y es una muy buena opción cuando se empieza a programar algoritmos de inteligencia artificial. De todas las IDEs que ofrece Anaconda Navigator escogí “Spyder” en su versión 5.0.0 para crear la estructura del prototipo y su entorno gráfico.

También utilice la IDE de “Jupyter notebook” en su versión 6.5.2 para realizar las pruebas de entrenamiento y analizar los resultados estadísticamente, todas estas IDEs son bastante livianas en comparación de otros programas como MatLab, sin embargo Anaconda Navigator posee una memoria cache es bastante limitada al momento de aplicar demasiadas variables al mismo tiempo.

Una vez terminado de definir los entornos de programación y escoger las bibliotecas y librerías para estructurar el prototipo, es hora de crear nuestro primer algoritmo que permita limpiar los datos obtenidos (diagnóstico en imágenes) de la “tarea 1: adquisición”, para que posteriormente puedan ingresar a la red neuronal convolucional y así obtener un pronóstico aceptable.

3.4.3. TAREA 3: PREPROCESAMIENTO.

Una vez ya definido el IDE para empezar a programar el prototipo y haber conseguido el *dataset* necesario (conjunto de imágenes de diagnósticos de melanomas y carcinomas) para alimentar la entrada de la red neuronal convolucional, es hora de empezar con la limpieza de los datos adquiridos, primeramente vemos que tenemos un conjunto de más de 17000 diagnósticos en imágenes repartidos en grupos de melanomas y carcinomas, si analizamos

estos grupos podemos ver que muchas de estas imágenes poseen diferentes tamaños y que están etiquetadas con nombres de pacientes y códigos hospitalarios.

Recordemos que al construir una red neuronal convolucional uno de los requisitos importantes es que todas las imágenes de entrada deberán poseer el mismo tamaño, también como vimos en el punto (3.4.2. Tarea 2) tendremos en cuenta la librería Dense que trabaja muy bien con imágenes de resolución pequeña, por lo tanto, mi primera tarea es el de redimensionar todas las imágenes del *dataset* a un tamaño de 224x224 píxeles y renombrarlas con números enteros positivos, la razón de esta conclusión es que la gran mayoría de estas 17000 imágenes poseen el tamaño de 224x224 píxeles y además es perfecto para utilizar la librería Dense ya que no son muy grandes.

3.4.3.1. REDIMENSIONANDO EL DATASET.

Primeramente, debemos juntar todas las imágenes de los diagnósticos de melanomas y carcinomas obtenidas (esto se menciona en el punto 3.4.1 Tarea 1) en una sola ruta, luego todas estas imágenes pasaran por un proceso de redimensionamiento gracias a la librería de “imutils”, una vez redimensionada la imagen esta se almacenará en una carpeta nueva denominada Melanoma_escalado o Carcinoma_escalado según su diagnóstico, todo esto se ve reflejado en la imagen 3.2 donde la variable ruta contiene la dirección de donde se almacenan las imágenes para redimensionar.

```
def escalar():
    global ruta
    os.chdir(ruta)
    for f in os.listdir():
        im = Image.open(f)
        texto, extencion=os.path.splitext(f)
        imResize = im.resize((224,224), Image.ANTIALIAS)
        imResize.save(destino+texto + '.jpg', 'JPEG', quality=90)
```

Imagen 3.2: se muestra un método donde se redimensiona una imagen a 224x224 píxeles en formato .jpg.

Una vez redimensionadas y almacenadas las imágenes de melanomas y carcinomas pasamos a la fase del etiquetado, es decir al renombramiento de dichas imágenes con

valores de números enteros positivos, esto solo con el fin de que posteriormente podamos dividirla en un cierto porcentaje para la fase de entrenamiento y de prueba.

3.4.3.2. RENOMBRAMIENTO DE LAS IMÁGENES.

En esta etapa vamos a renombrar todas las imágenes obtenidas de la etapa de redimensionamiento.

Primeramente, accedemos a la ruta de las imágenes resultantes de la anterior etapa, luego creamos un contador que comienza en 1 y finalmente gracias a la librería `Os` podemos renombrar cada imagen almacenada de dicha ruta con números enteros positivos, todo esto se hace con el fin de que en la siguiente fase se pueda distribuir de manera equitativa (30% prueba y 70 % entrenamiento), en la imagen 3.3 podemos observar dicho método de renombramiento llamado etiquetar.

A screenshot of a code editor with a dark background and light-colored text. The code is a Python function named 'etiquetar()' that takes no arguments. It declares a global variable 'destino'. It initializes a counter 'i' to 1. It changes the current directory to 'destino' using 'os.chdir(destino)'. It enters a loop 'for f in os.listdir():' where it splits the filename 'f' into 'texto' and 'extencion' using 'os.path.splitext(f)'. It then renames the file 'f' to 'str(i)+extencion' using 'os.rename(f, str(i)+extencion)'. Finally, it increments the counter 'i' by 1 using 'i=i+1'.

```
def etiquetar():  
    global destino  
    i=1  
    os.chdir(destino)  
    for f in os.listdir():  
        texto, extencion=os.path.splitext(f)  
        os.rename(f, str(i)+extencion)  
        i=i+1
```

Imagen 3.3: se muestra el método donde se renombra cada una de las imágenes con números enteros positivos.

3.4.3.3. DISTRIBUCIÓN DE LAS IMÁGENES.

En esta etapa vamos a distribuir las imágenes de manera equitativa a los métodos de entrenamiento y de prueba, para ello debemos conocer que una red neuronal necesita datos para ser entrenada y otros datos para ser puesta a prueba, según Exponetis (Gliese710, 2019) la mejor manera de distribuir una gran cantidad de datos para entrenar una red neuronal es dividiendo todos los datos de la siguiente manera: 70% para entrenamiento y 30% para pruebas.

Para la fase de entrenamiento es necesario de contar con una gran cantidad de datos, esto con el fin de obtener una buena predicción de salida, la red neuronal ajustara sus pesos (W) y sus *bias* (b) de acuerdo al número de ejemplos que se provea y al número de etapas que se invierta, por lo tanto, la mejor manera seria dedicar un 70% (cantidad de imágenes) o más al proceso de entrenamiento y el resto iría a parar al proceso de prueba, esto con el fin de que el sistema evalúe las imágenes nunca antes vistas con un cierto nivel de aprendizaje obtenido en la etapa de entrenamiento.

En mi caso contamos con aproximadamente 17539 imágenes de melanomas y carcinomas de manera desequilibrada entre ambos casos, es por eso que solamente tomare como mi *dataset* a 17000 imágenes.

Entonces tendríamos la siguiente distribución:

- el 70% del *dataset* seria alrededor de 11900 ejemplos, sin embargo, según Exponetis (Gliese710, 2019) mientras más ejemplos tomemos para el entrenamiento mucho mejor será, entonces redondeamos hasta llegar a 12000 imágenes para el *dataset* de entrenamiento de mi modelo.
- el 30% del *dataset* seria 5100 ejemplos, sin embargo, si tomamos en cuenta el punto anterior nos estaría quedando 5000 imágenes para el *dataset* de prueba de mi modelo.

Al final tendremos la siguiente distribución de directorios:

- Data (donde se guardará el *dataset* de 17000 imágenes).
 - Train (12000 imágenes en total).
 - Carcinoma_escalado (6000 aproximadamente).
 - Melanoma_escalado (6000 aproximadamente).
 - Test (5000 imágenes en total).
 - Carcinoma_escalado (2500 aproximadamente).
 - Melanoma_escalado (2500 aproximadamente).

Una vez creada la distribución de las imágenes (*dataset*) comenzamos con la tarea de crear un método que obtenga todas las características más importantes (contorno, forma, etc.) de cada objeto en cada una de estas imágenes para que al final estas características halladas pasen a una red neuronal y que esta pueda clasificar el objeto en un grupo determinado (melanoma o carcinoma).

3.4.4. TAREA 4: CONVOLUCIÓN.

Se define a un proceso de convolución como la multiplicación de una matriz (valores de una imagen) con otra (*kernel*) para obtener una matriz resultante en base a los valores de la primera, pero con respecto a las características especiales de la segunda. Este proceso se puede realizar “n” veces con “m” cantidad de *kernels* pero siempre al final nos quedara un conjunto de matrices con características especiales.

En mi caso aplicare la siguiente estructura secuencial en el proceso de convolución:

- una capa de convolución que estará procesada por 4 *kernels* de 3x3 pixeles, con una entrada ajustada para el tamaño de las imágenes que sería de 224x224 pixeles y finalmente como función de activación a Relu la cual nos devolverá mapas de características de tamaño 222x222 pixeles, además se generará 112 parámetros actualizables en esta capa.
- una segunda capa de convolución, esta contará con 8 *kernels* de 3x3 pixeles y que también tendrá como función de activación a Relu la cual nos devolverá mapas de características de tamaño 220x220 pixeles, además se generará 296 parámetros actualizables en esta capa.
- una capa de MaxPooling de tamaño 2x2 pixeles, esta capa tomará los valores más significantes de los mapas de características anteriores y los resumirá en la mitad del tamaño de la última capa de convolución, en mi caso sería 110x110 pixeles.
- una tercera capa de convolución, esta contará con 16 *kernels* de 3x3 pixeles y que también tendrá como función de activación a Relu la cual nos devolverá mapas de

características de tamaño 108x108 píxeles, además se generará 1168 parámetros actualizables en esta capa.

- una cuarta capa de convolución, esta contará con 32 *kernels* de 3x3 píxeles y que también tendrá como función de activación a Relu la cual nos devolverá mapas de características de tamaño 106x106 píxeles, además se generará 4640 parámetros actualizables en esta capa.
- finalmente se añadirá al modelo una capa de MaxPooling de tamaño 2x2 píxeles, esta capa también tomará los valores más significantes de los mapas de características anteriores y los resumirá en la mitad del tamaño de la última capa de convolución, en mi caso sería 53x53 píxeles.

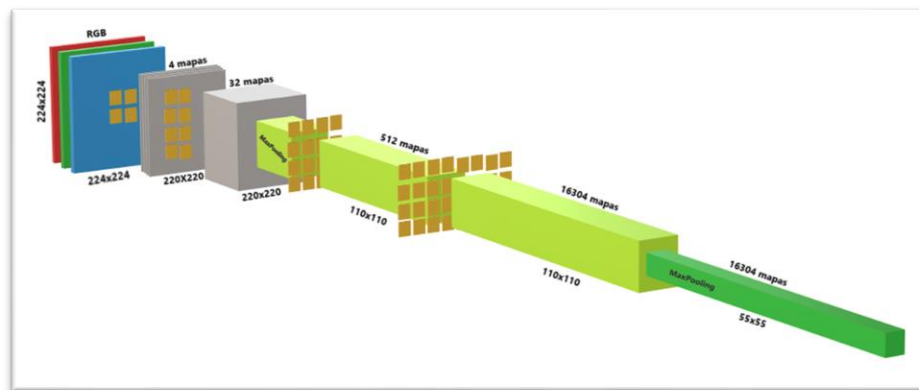


Imagen 3.4: en la imagen se muestra la estructura del proceso de convolución, donde los cuadrados amarillos son los *kernels*.

Todo el proceso estructural anteriormente explicado lo implementare en un archivo con extensión “. ipynb”, este archivo se puede abrir mediante “Jupyter Notebook” y contiene la estructura de la red neuronal convolucional profunda de mi prototipo, en este archivo entrenare adecuadamente el modelo llegando a obtener un nivel de asertividad de más del 80% junto con sus pruebas respectivas de validación, este archivo se puede descargar del siguiente enlace:

https://drive.google.com/file/d/1yj3u4Cnn7d5zJ41Lm8NiHSSabuPbXP2v/view?usp=share_link .

Comencemos con la primera capa de convolución, en donde aplicare el método “convolution2D”, este método me permite realizar una convolución con una imagen de entrada y un numero de *kernels* definido, en mi caso utilizare 4 *kernels* con una dimensión de 3x3 pixeles, estos *kernels* recorrerán la matriz de entrada (se dice matriz de entrada a la imagen convertida en valores) de 224x224 pixeles de izquierda a derecha, finalmente aplicare una función de activación “ReLU” como salida de esta capa, esto con el fin de dejar intactos los valores positivos de cada celda en el mapa de características y volver a nulos los valores negativos. En esta parte es donde el tamaño de la imagen original se reduce gracias a la multiplicación del *kernel* y se suma una columna y fila, es decir si mi imagen es de 224x224 pixeles se le restara la dimensión del *kernel* que estemos usando es decir en mi caso 3x3 pixeles, dando como resultado 4 mapas de características con dimensiones de 221x221 pixeles, luego se le sumara una fila y columna de valores nulos (0) haciendo que la imagen resultante sea de 222x222 pixeles en esta capa, esto último es para que no se pierda la información mediante el proceso de convolución entre la matriz y el *kernel*.

Para calcular el número de parámetros actualizables la primera convolución utilice la siguiente formula:

$$((PK * NK) * 3) + bNK$$

$$((9 * 4) * 3) + 4 = \mathbf{112} \text{ parametros actualizables.}$$

En la segunda capa aplicare nuevamente el método de convolucion2D, pero esta vez con 8 *kernels* de 3x3 pixeles, igualmente la dimensión del mapa de características se reducirá en 3, luego se le sumará una fila y columna dando como resultado 32 mapas de características con dimensiones de 220x220 pixeles. Finalmente obtenemos los parámetros actualizables de esta convolución.

$$((PK * NK) * KA) + bNK$$

$$((9 * 8) * 4) + 8 = \mathbf{296} \text{ parametros actualizables.}$$

La siguiente capa en apilarse contendrá la librería de “MaxPooling2D”, este proceso realiza un recorrido de toda la matriz de entrada (mapa de características de la anterior capa) de izquierda a derecha y de arriba hacia abajo, buscando los valores más altos dentro de una ventana de rango, esto se lo realiza también para encontrar rasgos importantes de un objeto, no importa si este está volteado o invertido los rasgos importantes del objeto siempre tendrán un valor alto, usualmente esta ventana suele ser de 2x2 pixeles. Luego de encontrar el valor más alto dentro de su ventana de rango, este lo guarda en una matriz nueva, siendo así la reducción de la imagen resultante en la anterior capa y obteniendo los valores más altos dentro de la ventana de 2x2 pixeles. En esta parte también se reduce el tamaño de la matriz resultante de la anterior capa, esto sucede al obtener los valores más relevantes en una ventana de 2x2 pixeles, en nuestro caso la matriz obtenida del proceso anterior nos dejó con tamaño de 220x220 pixeles, cuando apliquemos la ventana de 2x2 pixeles la matriz resultante sería una de 110x110 pixeles, en el remoto caso de que se aplicase una ventana de 3x3 pixeles la matriz se reduciría hasta en una tercera. En esta parte el número de parámetros actualizables es nulo.

En la tercera capa aplicare nuevamente el método de convolucion2D, pero esta vez con 16 *kernels* de 3x3 pixeles, dando como resultado 512 mapas de características con unas dimensiones de 108x108 pixeles. Finalmente obtenemos los parámetros actualizables de esta convolución.

$$((9 * 16) * 8) + 16 = \mathbf{1168} \text{ } \textit{parametros actualizables}.$$

En la cuarta capa aplicare convolucion2D con 32 *kernels* de 3x3 pixeles, dando como resultado 16304 mapas de características con unas dimensiones de 106x106 pixeles. Finalmente obtenemos los parámetros actualizables de esta convolución.

$$((9 * 32) * 16) + 32 = \mathbf{4640} \text{ } \textit{parametros actualizables}.$$

Luego se aplicará una capa de “MaxPooling2D” donde se reducirá a la mitad las dimensiones de los mapas de características a 53x53 pixeles.

En la capa siguiente tenemos el proceso de “Flatten”, el cual consiste en modificar la matriz resultante del anterior proceso y convertirla en un “*array*” de datos, esto se lo hace porque la siguiente capa es la capa de entrada de la red neuronal y como bien sabemos esta solo acepta “*arrays*” como entradas no matrices. En nuestro caso la longitud del array seria de:

$$(53 * 53) + 32 = \mathbf{89888} \text{ de longitud}$$

Donde:

- 53*53= es la dimensión de los mapas de características al final de nuestra convolución.
- 32=es la dimensión del último *kernel*.

3.4.5. TAREA 5: RED NEURONAL.

En esta etapa se creará la red neuronal, esta clasificará la entrada (array de la capa de convolución) en grupos que contengan similares patrones de entrada (melanomas o carcinomas).

Comenzamos creando la primera capa de la red neuronal, la cual recibirá como entrada el “*array*” de datos de la capa de convolución, esta capa de entrada tendrá un tamaño de 128 neuronas, estas neuronas se encargarán de recibir los valores provenientes del “*array*”, la salida de cada neurona estará sujeto a la función de activación “ReLU”.

Luego se añadirán dos capas ocultas con 100 neuronas cada una, estas tendrán salidas reajustadas a la función de activación “ReLU”, cuando una red neuronal posee más de una capa oculta se la denomina red neuronal profunda.

Finalmente, en la capa de salida tendremos solo 1 neurona, esta contará con la función de activación “Sigmoid”, esto se lo hace para que el resultado sea Binario (ver imagen 3.5).



The image shows a Keras model being built and its summary. On the left, the Python code defines a sequential model with the following layers: Convolution2D(4, (3,3), input_shape=(224,224,3), activation='relu'), Convolution2D(8, (3,3), activation='relu'), MaxPooling2D(pool_size=(2,2)), Convolution2D(16, (3,3), activation='relu'), Convolution2D(32, (3,3), activation='relu'), MaxPooling2D(pool_size=(2,2)), Flatten(), Dense(128, activation='relu'), Dense(100, activation='relu'), Dense(100, activation='relu'), and Dense(1, activation='sigmoid'). On the right, the model summary is displayed as a table.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 4)	112
conv2d_1 (Conv2D)	(None, 220, 220, 8)	296
max_pooling2d (MaxPooling2D)	(None, 110, 110, 8)	0
conv2d_2 (Conv2D)	(None, 108, 108, 16)	1168
conv2d_3 (Conv2D)	(None, 106, 106, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 53, 53, 32)	0
flatten (Flatten)	(None, 89888)	0
dense (Dense)	(None, 128)	11505792
dense_1 (Dense)	(None, 100)	12900
dense_2 (Dense)	(None, 100)	10100
dense_3 (Dense)	(None, 1)	101
Total params: 11,535,109		
Trainable params: 11,535,109		
Non-trainable params: 0		

Imagen 3.5: a la izquierda vemos la estructura de la convolución y la red neuronal en forma de código, a la derecha se muestra la estructura del modelo secuencialmente con sus parámetros en cada capa.

3.4.6. TAREA 6: DISTRIBUCIÓN Y ENTRENAMIENTO DEL MODELO.

En esta etapa se entrenará el modelo con las estructuras anteriores (convolución y red neuronal), para ello será necesario almacenar las imágenes de las carpetas Melanoma_escalado y Carcinoma_Escalado en diferentes listas a las cuales las etiquetaremos según su diagnóstico:

- Imágenes con diagnóstico de melanoma recibirán la etiqueta 1.
- Imágenes con diagnóstico de carcinoma recibirán la etiqueta 0.

Primeramente, se crea 4 listas vacías, donde se almacenarán los datos (imágenes y etiquetas) para su posterior procesamiento:

- X_train= []: en esta lista se almacenarán las imágenes del *dataset* de entrenamiento tanto de melanomas y carcinomas.
- Y_train= []: en esta lista se almacenarán las etiquetas correspondientes a las imágenes almacenadas en X_train, esta lista servirá para el entrenamiento.

- `X_test= []`: en esta lista se almacenarán las imágenes del *dataset* de prueba tanto de melanomas y carcinomas.
- `Y_test= []`: en esta lista se almacenarán las etiquetas correspondientes a las imágenes en `X_test` para el proceso de prueba.
- `dataTr = []`: en esta lista se almacenarán los tanto imágenes como etiquetas de forma aleatoria.

Seguidamente creamos la estructura para almacenar tanto las imágenes de la carpeta `Melanoma_escalado` y `Carcinoma_Escalado` con sus respectivas etiquetas mencionadas anteriormente (ver imagen 3.6).

```

1 X_train=[]
2 X_test=[]
3 Y_test=[]
4 Y_train=[]
5 dataTr=[]
6 for filename in glob.glob(os.path.join('D:/UMSA/TESIS/Tesis de grado/data/entrenamiento/Melanoma_escalado','*.jpg')):
7     dataTr.append([1,cv2.imread(filename)])
8 for filename in glob.glob(os.path.join('D:/UMSA/TESIS/Tesis de grado/data/entrenamiento/Carcinoma_escalado','*.jpg')):
9     dataTr.append([0,cv2.imread(filename)])

```

Imagen 3.6: imagen donde se muestra la creación de las listas de almacenamiento y la distribución de las imágenes.

Posteriormente creamos una lista denominada “`dataTe`” en donde repetiremos el mismo proceso anterior, solo que esta vez con las imágenes almacenadas en la carpeta prueba (ver imagen 3.7).

```

1 dataTe=[]
2 for filename in glob.glob(os.path.join('D:/UMSA/TESIS/Tesis de grado/data/prueba/melanoma_escalado','*.jpg')):
3     dataTe.append([1,cv2.imread(filename)])
4 for filename in glob.glob(os.path.join('D:/UMSA/TESIS/Tesis de grado/data/prueba/carcinoma_escalado','*.jpg')):
5     dataTe.append([0,cv2.imread(filename)])

```

Imagen 3.7: imagen donde podemos observar la distribución de las imágenes de la carpeta de prueba con sus respectivas etiquetas.

Luego crearemos un método llamado “`partición`”, donde recibirá como un parámetro una lista que contiene imágenes y etiquetas, luego se separarán las imágenes de las etiquetas y las almacenarán en diferentes listas, finalmente estas se convertirán en *arrays* (ver imagen 3.8).

```

1 def particion(datos):
2     imagenes=[]
3     etiquetas=[]
4     for i,j in datos:
5         imagenes.append(j)
6         etiquetas.append(i)
7     imagenes=np.array(imagenes)
8     etiquetas=np.array(etiquetas)
9     return (imagenes,etiquetas)

```

Imagen 3.8: imagen donde podemos observar el método partición.

Siguientemente crearemos la estructura donde aleatorizaremos los datos de las listas “dataTr” y “dataTe” e imprimiremos por consola la longitud de dichas listas, luego asignamos 12000 imágenes con sus etiquetas correspondientes al *dataset* de entrenamiento a una nueva lista a la cual denominaremos “porcion1”, haremos exactamente lo mismo con 5000 imágenes del *dataset* de prueba dentro de la nueva lista denominada “porcion2” y finalmente separaremos las etiquetas de las imágenes en diferentes *arrays* gracias al método “partición”, donde tenemos la siguiente estructura (ver imagen 3.9):

- x_e= el *array* donde estará almacenado imágenes del *dataset* de entrenamiento.
- y_e= el *array* donde se almacenarán las etiquetas correspondientes a “x_e”.
- x_p= el *array* donde estará almacenado imágenes del *dataset* de prueba.
- y_p= el *array* donde se almacenarán las etiquetas correspondientes a “x_p”.

```

1 shuffle(dataTr)
2 print("en total tenemos: "+str(len(dataTr))+ " imagenes dentro de la carpeta train")
3 shuffle(dataTe)
4 print("en total tenemos: "+str(len(dataTe))+ " imagenes dentro de la carpeta test")

en total tenemos: 12279 imagenes dentro de la carpeta train
en total tenemos: 5261 imagenes dentro de la carpeta test

1 porcion1=dataTr[0:12000]
2 porcion2=dataTe[0:5000]
3 print("para entrenamiento tendremos: "+str(len(porcion1))+ " imagenes de la carpeta de train")
4 print("para prueba tendremos: "+str(len(porcion2))+ " imagenes de la carpeta de test")

para entrenamiento tendremos: 12000 imagenes de la carpeta de train
para prueba tendremos: 5000 imagenes de la carpeta de test

1 x_e,y_e=particion(porcion1)
2 x_p,y_p=particion(porcion2)

```

Imagen 3.9: imagen donde vemos el proceso de aleatorización y partición de las listas.

Posteriormente pasamos a crear dos métodos importantes que sirven para entrenar al modelo, el primer método se denomina “validación” y consiste en entrenar el modelo con los parámetros pasados por el segundo método, además evaluará el modelo con los datos de prueba y devolverá el nivel de asertividad. El segundo método nombrado “evaluación” tiene como objetivo llamar al método “validación” hasta alcanzar un cierto nivel de asertividad introducido como parámetro, además de almacenar los niveles de asertividad de cada entrenamiento y la cantidad de épocas que le tomo al modelo alcanzar hasta llegar a superar el nivel de asertividad (ver imagen 3.10).

```

1 def validacion(x_e,y_e,x_p,y_p,modelo1,epocas):
2     entre=modelo1.fit(x_e,y_e,batch_size=32,epochs=epocas,validation_data=(x_p,y_p))
3     a=modelo1.evaluate(x_p,y_p)
4     return a[1]

1 def evaluacion(modelo1,porcentaje,nombre,v1,v2):
2     prediccion=0
3     epocas=1
4     while(True):
5         if prediccion < porcentaje:
6             prediccion=(validacion(x_e,y_e,x_p,y_p,modelo1,1))*100
7             epocas +=1
8             v1.append(epocas-1)
9             v2.append(prediccion)
10        else:
11            print("==> Para el metodo "+nombre+" se utilizo: "+str(epocas-1)+" epocas para llegar a mas del "+str(porcentaje)
12            break

```

Imagen 3.10: imagen donde muestra la estructura de los métodos de: “validación” y “evaluación”.

Finalmente ajustamos algunos parámetros como el tipo de optimizador, el nivel del *learning rate* y el nivel de asertividad que pudiese alcanzar nuestro modelo. Una vez ajustados estos parámetros llamamos al método “evaluación” que se encargara de entrenar el modelo hasta alcanzar el nivel de asertividad que nosotros le ordenemos (ver figura 3.11).

```

1 modelo_adam = keras.optimizers.Adam(learning_rate=0.0008)
2 modelo.compile(optimizer=modelo_adam,loss='binary_crossentropy',metrics=['accuracy'])
3 v_eA=[]
4 v_aA=[]
5 evaluacion(modelo,90,"ADAM",v_eA,v_aA)

```

Imagen 3.11: imagen donde muestra los ajustes de los parámetros mencionados hasta alcanzar un 90% de asertividad, además de la creación de dos listas en donde se almacenarán el nivel de asertividad y el número de épocas de cada entrenamiento.

Si vemos que el modelo es prometedor (si pudiese alcanzar o superar el nivel de asertividad) entonces lo guardaremos junto con sus pesos para que a posterior sea utilizado como referente a la hora de clasificar una imagen como melanoma o carcinoma (ver imagen 2.12).

```
1 modelo.save("D:/UMSA/Documentos/CNN cancer de piel/MODELLOS/17000_4_3_FINAL/modelo")
1 modelo.save_weights("D:/UMSA/Documentos/CNN cancer de piel/PESOS/17000_4_3_FINAL/modelo_pesos")
```

Imagen 3.12: imagen donde vemos el proceso para guardar el modelo entrenado y sus pesos.

3.4.7. TAREA 7: INTERPRETACIÓN DEL RESULTADO.

Una vez concluida la estructuración, pasamos a entrenar el modelo con el método de optimización “Adam” hasta alcanzar un nivel de asertividad del 90% (figura 3.13), los resultados fueron favorables, después del entrenamiento el modelo alcanzo un nivel de asertividad del 88% en tan solo 44 épocas de entrenamiento.

# de EPOCAS	Valor del Accuracy				
epoca 1 =>	77.38000154495239	epoca 16 =>	86.69999837875366	epoca 30 =>	87.95999884605408
epoca 2 =>	82.63999819755554	epoca 17 =>	84.7999989864197	epoca 31 =>	85.29999852180481
epoca 3 =>	84.7000002861023	epoca 18 =>	86.14000082015991	epoca 32 =>	87.73999810218811
epoca 4 =>	83.74000191688538	epoca 19 =>	86.54000163078308	epoca 33 =>	87.95999884605408
epoca 5 =>	85.57999730110168	epoca 20 =>	84.43999886512756	epoca 34 =>	88.13999891281128
epoca 6 =>	85.4200005531311	epoca 21 =>	84.7000002861023	epoca 35 =>	87.95999884605408
epoca 7 =>	85.01999974250793	epoca 22 =>	87.48000264167786	epoca 36 =>	87.87999749183655
epoca 8 =>	85.55999994277954	epoca 23 =>	82.16000199317932	epoca 37 =>	87.99999952316284
epoca 9 =>	86.18000149726868	epoca 24 =>	87.99999952316284	epoca 38 =>	87.99999952316284
epoca 10 =>	86.82000041007996	epoca 25 =>	84.52000021934509	epoca 39 =>	87.94000148773193
epoca 11 =>	86.33999824523926	epoca 26 =>	87.30000257492065	epoca 40 =>	88.0400002002716
epoca 12 =>	85.92000007629395	epoca 27 =>	84.96000170707703	epoca 41 =>	87.95999884605408
epoca 13 =>	85.6000006198883	epoca 28 =>	87.40000128746033	epoca 42 =>	88.0400002002716
epoca 14 =>	85.29999852180481	epoca 29 =>	85.7200026512146	epoca 43 =>	88.17999958992004
epoca 15 =>	85.5400025844574	epoca 30 =>	87.95999884605408	epoca 44 =>	88.22000026702881

Imagen 3.13: en estas tres imágenes vemos que el modelo alcanza un nivel de asertividad del 88% en tan solo 44 épocas.

Una vez entrenado el modelo pasaremos a realizar una prueba con una imagen dentro de la carpeta test, la imagen será puesta a prueba con el método de “predict”, este método pasará la imagen por las estructuras ya entrenadas (red convolucional y neuronal) con el fin de obtener un valor decimal que prediga si la imagen tuviese más rasgos de ser un melanoma (0.5 a 0.9) o carcinoma (0.0001 a 0.49), el resultado decimal se redondeara y

luego se mostrara la imagen puesta a prueba donde tendrá como titulo la predicción del modelo.

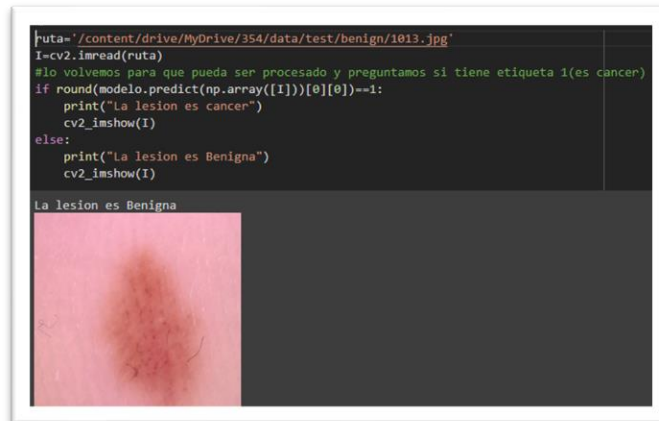


Imagen 3.14: se observa la predicción que tiene el prototipo con una imagen seleccionada de la carpeta test.

Con esta tarea finalizada podemos decir que el entrenamiento del modelo y la clasificación están concluidos, sin embargo, para que el prototipo sea más amigable al usuario (medico dermatólogo) se debe realizar un entorno gráfico intuitivo a la hora del uso del prototipo.

3.4.8. TAREA 8: DESARROLLO DEL ENTORNO GRÁFICO.

Al momento de implementar el entorno grafico se tuvo en cuenta las recomendaciones del médico dermatólogo, el entorno tiene que ser muy intuitivo a la hora de pronosticar una lesión del paciente, además de guardar en una carpeta las imágenes tomadas del paciente, se tuvo en cuenta las siguientes sugerencias:

- Para tomar la imagen del paciente:
 - Un botón que active la cámara.
 - Un delimitador que muestre donde posicionar la cámara para obtener una buena muestra.
 - Un botón para capturar la muestra fotográfica.
 - Que se pueda almacenar de manera automática la muestra en la computadora del médico con el nombre y carnet del paciente.

- Una segunda imagen generada a través de la primera, que muestre algunas posibles lesiones cercanas al objetivo.
- Un botón para limpiar todo lo anterior.
- Para pronosticar el resultado:
 - Un botón que muestre el almacenamiento de las imágenes recolectadas.
 - Mostrar la imagen que se usó de prueba.
 - Mostrar el resultado de la predicción en texto comprensible para el paciente y médico.
 - Mostrar el nivel de asertividad de la predicción en porcentaje.
 - Mostrar las posibles causas (test del A, B, C, D, E) que tomo en cuenta el prototipo para llegar a la predicción final.
 - Mostrar una imagen donde solo se vea la lesión del paciente en escala de grises, sin la piel de fondo.

Una vez obtenido los requisitos que debería llevar el prototipo a la hora de efectuar una tarea en específico, se empieza por elegir las librerías para dicho entorno, esta vez utilizare TKinter de python3, esto por su sencillas y su compatibilidad con el código anteriormente implementado.

Primeramente, cargamos las librerías necesarias para realizar la interfaz gráfica, seguidamente se crea el lienzo de dibujo donde añadiremos los botones, cajas de comentarios y demás herramientas del prototipo, a este lienzo lo llamare “root”, luego añadiremos un método llamado “cerrar” que se ejecutara cuando cerremos la pestaña de nuestra interfaz gráfica, seguido colocaremos un título a nuestra interfaz gráfica cuando esta aparezca en forma de ventana al usuario (ver imagen 3.15).

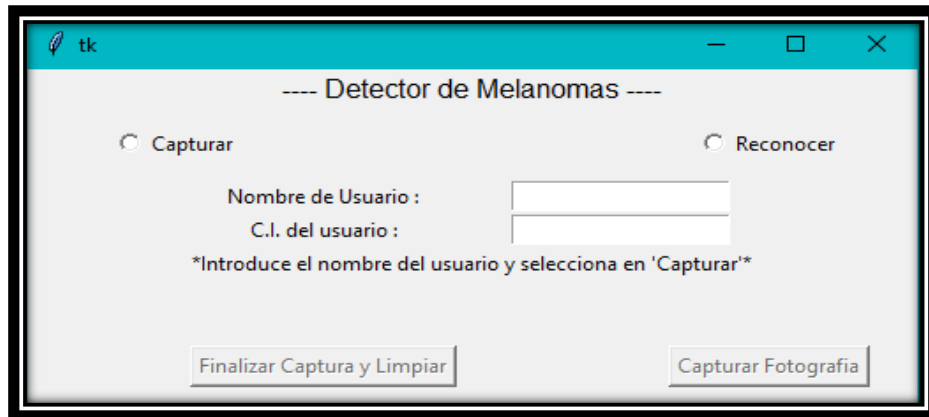


Imagen 3.15: se observa la ventana de inicio del prototipo.

En la parte de los 2 botones en formato de “Radiobutton”, tendrán las siguientes características según las sugerencias anteriormente mencionadas:

- **Capturar:** al presionar el botón capturar se activarán las siguientes opciones (ver imagen 3.16):
 - se bloquearán los botones de capturar y reconocer hasta finalizar la captura de la imagen y los datos del paciente.
 - se habilitan los campos de texto donde se capturarán los datos del paciente como el nombre y su carnet de identidad para luego convertirlos en el nombre de la carpeta correspondiente a cada usuario, dentro de estas carpetas se guardarán las imágenes de los pacientes.
 - se habilitará la cámara donde se posicionará la lesión del paciente, lo más recomendado es que el centro de la lesión se ubique en el punto rojo, los limites o bordes de la lesión estén ajustados con el borde azul y que exista un entorno de piel del paciente que este ajustado con el borde verde.
 - se activará el botón de la parte inferior del programa denominado “capturar fotografia”, donde permite almacenar las imágenes en una carpeta personal del paciente.
 - al presionar el botón “capturar fotografia” Se visualiza en el lado derecho la imagen capturada en formato BGR, pues al visualizar de esta manera la

captura de la lesión del paciente se puede tener bordes de color blanco sobre cualquier objeto relevante, en este caso otras posibles lesiones a tomar en cuenta, además remarca los bordes de la lesión principal con color azul.

- también se activará un segundo botón denominado “finalizar captura y limpiar”, este botón se encarga de resetear las variables de cuadros de texto, además desactivará la cámara y desbloqueará la opción de Reconocer.

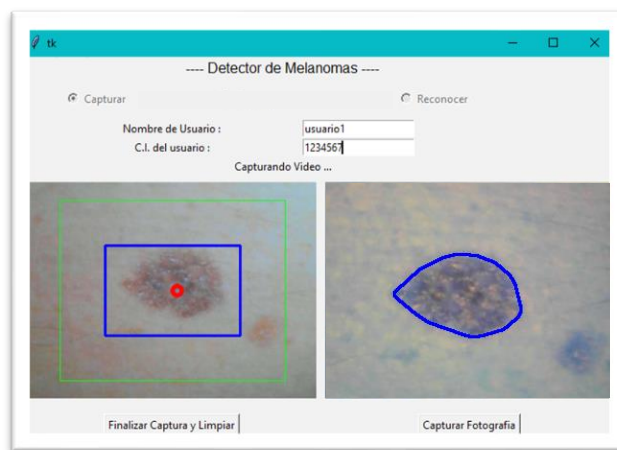


Imagen 3.16: se observa en la ventana la imagen secundaria en BGR, los datos del paciente y los botones para limpiar y capturar la imagen, además los botones deshabilitados de capturar, entrenar y reconocer.

- **Reconocer:** el botón de reconocer permite al usuario final (medico dermatólogo) seleccionar una imagen dentro de su galería o dentro de su carpeta de “usuarios” y poder predecir si la imagen pudiese pertenecer al caso de melanomas o carcinomas (ver imagen 3.17), al seleccionar la opción de reconocer se activan los siguientes procesos:
 - al lado izquierdo en la parte inferior se muestra la imagen seleccionada.
 - en la parte superior de esta imagen se muestra con texto claro el pronóstico del prototipo, el nivel de asertividad y si el resultado fuese de un posible

melanoma se mostraría las causas tomando en cuenta el diagnóstico del A, B, C, D y E.

- en la parte derecha se muestra el recorte de la lesión sobre un fondo oscuro, esta lesión está en tono de grises, esto con el fin de ver el contorno con más claridad y la de ver posibles bultos en la lesión.



Imagen 3.17: se observa en la ventana la imagen secundaria seleccionada, el pronóstico en texto claro y los porcentajes de exactitud.

- **Finalizar captura y Limpiar:** este botón permite limpiar las cajas de texto de nombre y carnet del paciente, además de terminar el proceso de capturar la imagen mediante la cámara web, también limpia el pronóstico y la imagen seleccionada del proceso de reconocer.
- **Capturar Fotografía:** este botón permite guardar los datos del paciente insertados en las cajas de texto, además de guardar la imagen tomada con la cámara y etiquetarla con los datos del paciente en su respectiva carpeta personal dentro de la computadora del médico dermatólogo.

Una vez terminado el entorno gráfico para la presentación final al usuario, se realizaron las pruebas de rendimiento del prototipo, en las cuales se abarca los métodos de validación cruzada, optimizadores y *learning rate*, el primer método permitió demostrar el uso de las redes neuronales como clasificador adecuado en mi prototipo, con el segundo se pudo

hallar el mejor método actualizador de parámetros-épocas y con el ultimo se pudo encontrar el valor óptimo para ajustar los pesos y *bias* en la red neuronal.

3.5. FASE 5: PRUEBAS DE RENDIMIENTO.

Al llegar a esta fase, se concluye que el prototipo se realizó exitosamente, sin embargo, hace falta realizar pruebas que permitan dar a conocer el nivel de acierto de predicción, pues al tratarse de redes neuronales existen varios parámetros que nos hacen cuestionar si el camino que seguimos hasta ahora es el correcto.

Primeramente, nos cuestionamos si el método de aprendizaje es el correcto para nuestro prototipo, es decir sabemos que existen varios métodos por el cual una computadora aprende (k-vecinos, arboles de decisión, redes neuronales, etc.), para ello debemos aplicar un método llamado validación cruzada k-Cross, este método se vio en el punto 2.1.4 Validación cruzada.

3.5.1. IMPLEMENTANDO LA VALIDACIÓN CRUZADA.

Ahora bien, cuando apliquemos el método de la validación cruzada K-Cross obtendremos la media y la desviación estándar respecto a la predicción del *dataset* de entrenamiento, si la desviación estándar es mínima entonces se considera que el método de redes neuronales es la correcta para nuestro caso.

Para esta parte utilizare la IDE de Jupyter Notebook en su versión 12.0, pues para realizar las diferentes pruebas necesito un entorno que me permita ejecutar ciertas partes del código sin ejecutar el código completo, además de añadir comentarios y puntos de interés.

Para comenzar el método de validación cruzada empezamos a repartir en cinco partes iguales el *dataset* de entrenamiento, en mi caso cuento con 5261 imágenes para el entrenamiento, dando resultado una partición de 1052 imágenes en cada sección (ver imagen 3.18).

```

validacion Cruzada

1 n=int(len(dataTr)/5)
2 output=[dataTr[i:i + n] for i in range(0, len(dataTr), n)]
3 porcion1=output[0]
4 porcion2=output[1]
5 porcion3=output[2]
6 porcion4=output[3]
7 porcion5=output[4]
8 porcion5.extend(output[5])
9 print(len(porcion1),len(porcion2),len(porcion3),len(porcion4),len(porcion5))

1052 1052 1052 1052 1053

```

Imagen 3.18: se observa la división del *dataset* de entrenamiento en 5 partes iguales.

Luego de dividir en partes iguales, paso a entrenar cada segmento de 4 porciones como la parte de entrenamiento y la porción sobrante será la de prueba, todo esto entrará dentro del método de entrenamiento de mi modelo, dando como resultado un vector de porcentajes de asertividad correspondientes a cada fase del entrenamiento de cada porción (ver imagen 3.19).

```
[80.05697727203369, 96.10266089439392, 96.2927758693695, 99.42965507507324, 99.71482753753662]
```

Imagen 3.19: se observa el vector de porcentajes de asertividad resultante del entrenamiento de cada porción.

Finalmente obtenemos la media y la desviación estándar del vector resultante, dando como respuesta una media de: un 94% de asertividad con una desviación estándar del 7.3% entre cada porción entrenada (ver imagen 3.20).

```

7 media =round(np.mean(resultante),1)
8 desviacion_estandart = np.std(resultante)
9 print("valores de asertividad: "+str(v_a1[-1]),str(v_a2[-1]),str(v_a3[-1]),str(v_a4[-1]),str(v_a5[-1]))
10 print("se tiene que como media el valor: "+str(media)+"% de asertividad")
11 print("se tiene que como desviacion estandart: +/- "+str(round(desviacion_estandart,1))+"% de variacion")

valores de asertividad: 80.05697727203369 96.10266089439392 96.2927758693695 99.42965507507324 99.71482753753662
se tiene que como media el valor: 94.3% de asertividad
se tiene que como desviacion estandart: +/- 7.3% de variacion

```

Imagen 3.20: se observa el vector de porcentajes de asertividad resultante del entrenamiento de cada porción.

El valor de la desviación estándar nos quiere decir que la variación entre cada porcentaje de asertividad respecto a cada porción es muy similar pues apenas alcanza al 7% de variación entre evaluaciones, en otras palabras, no existe *overfitting* (que los valores de

entrenamiento no están muy dispersos unos de otros) y que **el modelo de redes neuronales es funcional para evaluar el *dataset***, también podemos observar que la media nos da un valor aceptable del 94% de asertividad.

Ahora bien, una vez que demostramos que el modelo de redes neuronales puede ser aplicado a nuestro *dataset*, debemos preguntarnos ¿Qué método de optimización aplico a mi modelo para que mi red neuronal pueda aprender y obtener buenos resultados?, para responder esa pregunta existen muchos métodos, los más populares se mencionan en el punto 2.1.5, en allí se describen 3 herramientas: SGD, RMSProp y ADAM. Tendremos que probar cada uno de estos métodos para comparar el nivel y el número de épocas entre cada uno y optar por el que tiene el mayor nivel de asertividad en menos épocas de entrenamiento.

3.5.2. IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN SGD.

Primeramente, definiremos el *dataset* de entrenamiento y el de prueba, porque si tomamos el *dataset* completo de 17000 imágenes el proceso de entrenamiento en esta parte llegaría a ser muy extenso, incluso podría demorar días enteros en entrenar cada método de optimización, es por eso que se consideró tomar solo en cuenta 1000 imágenes de las cuales se destinó 700 imágenes para el entrenamiento y 300 para la prueba, la distribución de las imágenes con diagnóstico de carcinomas y melanomas se hizo de manera aleatoria, es decir que existe un cierto desbalance de imágenes entre ambos grupos, seguidamente crearemos 2 métodos que nos permitirán realizar dichas pruebas.

El primer método se denomina “validación”, este entrenara el modelo y validara los resultados de acuerdo al conjunto de prueba, contara como parámetros de entrada los *datasets* de entrenamiento y prueba, además de un modelo predefinido y la cantidad de épocas a considerar (ver imagen 3.21).

El segundo método se denomina “evaluación”, este se encargará de evaluar el modelo hasta una cantidad N de épocas, solo se terminará de evaluar el modelo cuando se alcance el índice de aceptación introducido como un parámetro del método (ver imagen 3.22).

```
1 def validacion(x_e,y_e,x_p,y_p,model,epocas):
2     entre=model.fit(x_e,y_e,batch_size=32,epochs=epocas,validation_data=(x_p,y_p))
3     a=model.evaluate(x_p,y_p)
4     return a[1]
```

Imagen 3.21: en este código se entrena y prueba el modelo con los parámetros de entrada del método de evaluación.

```
def evaluacion(modelo,porcentaje,nombre):
    prediccion=0
    epocas=1
    while(True):
        if prediccion < porcentaje:
            prediccion=(validacion(x_e,y_e,x_p,y_p,modelo,1))*100
            epocas +=1
        else:
            print("==> Para el metodo "+nombre+" se utilizo: "+str(epocas-1)+" epocas para llegar a mas del "+str(porcentaje))
            break
```

Imagen 3.22: en este fragmento de código se valida si el entrenamiento alcanza un cierto nivel de asertividad.

Una vez definido los métodos para validar y evaluar el modelo se comienza a crear una variable denominada modelo2 en donde almacenaremos un modelo totalmente nuevo, luego escogeremos el modelo del optimizador referente a SGD con un valor de *learning rate* de 0.002 (por defecto), luego pasamos a compilar el modelo con las siguientes características: el optimizador escogido (SGD), una función de perdida referida a la entropía cruzada y finalmente mostramos las métricas correspondientes al entrenamiento. Después de compilar el modelo pasamos a evaluarlo hasta que llegue a más del 80% de asertividad (ver imagen 3.23).

```

1 v_eS=[]
2 v_aS=[]
3 evaluacion(modelo2,80,"SGD",v_eS,v_aS)

Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.2060 - acc: 0.9614 - val_loss: 1.0204 - val_acc: 0.6900
300/300 [=====] - 1s 2ms/sample - loss: 1.0204 - acc: 0.6900
Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.0712 - acc: 0.9871 - val_loss: 1.6321 - val_acc: 0.6533
300/300 [=====] - 1s 2ms/sample - loss: 1.6321 - acc: 0.6533
Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.0489 - acc: 0.9957 - val_loss: 1.5770 - val_acc: 0.6667
300/300 [=====] - 1s 2ms/sample - loss: 1.5770 - acc: 0.6667
Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.0527 - acc: 0.9943 - val_loss: 1.2116 - val_acc: 0.7367
300/300 [=====] - 1s 2ms/sample - loss: 1.2116 - acc: 0.7367
Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.0669 - acc: 0.9886 - val_loss: 1.4728 - val_acc: 0.6900
300/300 [=====] - 1s 2ms/sample - loss: 1.4728 - acc: 0.6900
Train on 700 samples, validate on 300 samples
700/700 [=====] - 6s 8ms/sample - loss: 0.0440 - acc: 0.9971 - val_loss: 1.6554 - val_acc: 0.6567
300/300 [=====] - 1s 2ms/sample - loss: 1.6554 - acc: 0.6567
Train on 700 samples, validate on 300 samples

```

Imagen 3.23: en este fragmento de código se hace referencia a la creación y la validación del modelo SGD, además se muestra los resultados de cada época con su respectiva evaluación.

Los resultados después del entrenamiento fueron los siguientes:

- El valor de asertividad máximo que pudo alcanzar el modelo **SGD** fue de 0.7367 o un **73.67%** respecto al *dataset* de prueba.
- El valor de la función de perdida fue de **1.2116** respecto al *dataset* de prueba.
- **No se pudo alcanzar la meta de más de un 80% de asertividad**, así que tuve que parar el proceso cuando cumplió las 2 horas de entrenamiento y cuando llego a reconocer a la perfección el *dataset* de entrenamiento.
- Luego de que el modelo alcanzase el nivel de asertividad del 73%, este bajo poco a poco mientras más épocas continuaba entrenado, es decir existió un *overfitting*.

Como conclusión final del método de optimización SGD se tendría lo siguiente:

- **El método no se ajusta al *dataset*** que poseo, pues apenas llega a un 73% de asertividad y luego este nivel cae paulatinamente.
- Para llegar al nivel del 73% **se necesitó demasiadas épocas**, lo que implica mucho tiempo para un resultado no ideal.

3.5.3. IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN RMSPROP.

Para aplicar este método de optimización se realizó exactamente los mismos pasos que cuando se aplicó SGD, solo que esta vez usaremos RMSProp con el mismo valor del *learning rate* y un nivel a alcanzar del 80% o superior (ver imagen 3.24).

metodo RMSprop

```
1 modelo3=creacion_modelo()

1 modelo_RMSprop = keras.optimizers.RMSprop(learning_rate=lr)
2 modelo3.compile(optimizer=modelo_RMSprop,loss='binary_crossentropy',metrics=['accuracy'])

1 v_eR=[]
2 v_aR=[]
3 evaluacion(modelo3,80,"RMSprop",v_eR,v_aR)

Train on 700 samples, validate on 300 samples
700/700 [=====] - 11s 15ms/sample - loss: 0.0152 - acc: 0.9943 - val_loss: 5.2579 - val_acc: 0.8000
300/300 [=====] - 1s 3ms/sample - loss: 5.2579 - acc: 0.8000
==> Para el metodo RMSprop se utilizo: 48 epocas para llegar a mas del 80% de asertividad
```

Imagen 3.24: en este código se crea y valida el modelo RMSProp, en abajo se observa el resultado después de entrenar 48 épocas y alcanzar el 80% de asertividad.

Los resultados fueron los siguientes:

- **Se pudo alcanzar** el nivel esperado de un 80% de asertividad.
- El valor de la **función de perdida fue de 5.2579** respecto al *dataset* de prueba.
- Para llegar a ese nivel de asertividad el modelo tubo que entrenarse y validarse por **48 épocas**.

Las conclusiones son las siguientes:

- El método de optimización RMSProp es mucho más efectivo que SGD respecto a mi *dataset*.
- Para poder llegar al 80% de asertividad el modelo tardo 48 épocas, cada época le tomo 11 segundos en entrenar y 1 segundo en validar, por lo que podemos decir que aproximadamente se tardó 8 minutos con 8 segundos en entrenar el modelo.

- En comparación del SGD, el método RMSProp tardó mucho menos y se llegó al nivel esperado, por lo tanto, es factible utilizar este método de optimización para mi prototipo.

3.5.4. IMPLEMENTACIÓN DEL MÉTODO DE OPTIMIZACIÓN ADAM.

Ahora toca entrenar el modelo con el método de optimización de Adam, para ello utilice el mismo valor del *learning rate* y se espera entrenar hasta que logre alcanzar el nivel de más de un 80% de asertividad (ver imagen 3.25).

metodo ADAM

```
1 modelo1=creacion_modelo()

1 modelo_adam = keras.optimizers.Adam(learning_rate=lr)
2 modelo1.compile(optimizer=modelo_adam,loss='binary_crossentropy',metrics=['accuracy'])

1 evaluacion(modelo1,80,"ADAM")

-----
Train on 700 samples, validate on 300 samples
700/700 [=====] - 7s 9ms/sample - loss: 0.0143 - acc: 0.9971 - val_loss: 1.1931 - val_acc: 0.8033
300/300 [=====] - 1s 2ms/sample - loss: 1.1931 - acc: 0.8033
==> Para el metodo ADAM se utilizo: 15 epocas para llegar a mas del 80% de asertividad
```

Imagen 3.25: en esta imagen se muestra la creación y validación del modelo ADAM, después de entrenar y evaluar 15 épocas hasta llegar al 80% de asertividad.

Los resultados fueron los siguientes:

- **Se pudo alcanzar el nivel requerido** que es de un 80%.
- La función de pérdida nos da un valor de **1.1931** con los datos de prueba.
- El modelo se entrenó y se validó durante **15 épocas**.

Las conclusiones son:

- Se pudo alcanzar el nivel esperado de asertividad del 80% en tan solo 15 épocas, también el valor de la función de pérdida en comparación del método RMSProp, por lo tanto, en el momento de realizar las pruebas respectivas con imágenes habría que tomar estas observaciones muy en cuenta.

- El modelo tardo en entrenar 7 segundos y en validar 1 segundo, por lo tanto, se tardó 1 minuto con 10 segundos en llegar al 80% de asertividad con 15 épocas.
- Es más veloz y alcanza un nivel de asertividad mucho mejor que el método RMSProp, por lo tanto, este método es un buen candidato para aplicarlo a mi prototipo.

En conclusión, para proceso del aprendizaje de mi red neuronal **utilizare el método de optimización ADAM**, por la cantidad baja de épocas y el nivel de asertividad que tiene respecto a mi *dataset*.

Todos los procesos de entrenamiento respecto a los diferentes modelos de evaluación se encuentran registrados y al alcance del lector en el siguiente link de descarga: https://drive.google.com/file/d/1Kty00_Jb9qfOaK1VyJaqE-boT7vrjVmo/view?usp=share_link

Una vez definido el método de optimización para que mi red neuronal pueda ajustar los pesos y *bias*, surge la siguiente pregunta: ¿Cómo puedo mejorar el nivel de predicción o asertividad de mi prototipo?, es aquí donde entra la variable *learning rate*.

3.5.5. AJUSTE DEL LEARNING RATE DENTRO DEL MÉTODO ADAM.

La variable del *learning rate* sirve como parámetro multiplicador en el proceso de actualización de los pesos y *bias* de las neuronas por medio del proceso de *backpropagation*, en otras palabras, me ayudara a ajustar los parámetros de aprendizaje en mi red neuronal, estos están dentro de las ecuaciones de cada método de optimización y por defecto esta asignado el valor de 0.002, para más información consultar el punto 2.1.6.

Para poder encontrar el valor del *learning rate* más adecuado a mi modelo tuve que probar diferentes valores comenzando desde 0.0002 hasta 0.1, para ello utilice 3 vectores donde almacene diferentes rangos y luego los puse a prueba con el entrenamiento de mi modelo

(ver imagen 3.26), también utilice el método de optimización ADAM para ajustar los pesos y *bias*, pues fue la seleccionada por el anterior punto.

```
vector=[0.002,0.004,0.006,0.008,0.01]
vector2=[0.02,0.04,0.06,0.08,0.1]
vector3=[0.0002,0.0004,0.0006,0.0008,0.0009]
```

Imagen 3.26: en este fragmento de código se muestran vectores que almacenan los valores de prueba para el learning rate.

Seguidamente probaremos entrenando cada valor del primer “vector” en la estructura anterior referente a los optimizadores, si el valor del *learning rate* alcanza a más del 70% de asertividad lo marcaremos como posible candidato a ser elegido (ver imagen 3.27):

```
train on 700 samples, validate on 300 samples
Epoch 1/2
700/700 [=====] - 7s 10ms/sample - loss: 1.5079 - acc: 0.8014 - val_loss: 1.3649 - val_acc: 0.7600
Epoch 2/2
700/700 [=====] - 7s 10ms/sample - loss: 0.7974 - acc: 0.8600 - val_loss: 1.2361 - val_acc: 0.7300
300/300 [=====] - 1s 2ms/sample - loss: 1.2361 - acc: 0.7300
*****Para el metodo ADAM se utilizo: 3 epocas para llegar a mas del 70% de asertividad con el indice LR= 0.002

Train on 700 samples, validate on 300 samples
Epoch 1/2
700/700 [=====] - 7s 10ms/sample - loss: 4.0888 - acc: 0.4743 - val_loss: 0.8914 - val_acc: 0.7433
Epoch 2/2
700/700 [=====] - 7s 10ms/sample - loss: 0.5732 - acc: 0.7671 - val_loss: 0.6629 - val_acc: 0.7100
300/300 [=====] - 1s 2ms/sample - loss: 0.6629 - acc: 0.7100
*****Para el metodo ADAM se utilizo: 3 epocas para llegar a mas del 70% de asertividad con el indice LR= 0.004
```

Imagen 3.27: en esta imagen se muestra las épocas y los valores del *learning rate* que alcanzaron a más del 70% de asertividad.

Como podemos observar en la imagen 3.28, tenemos 2 posibles candidatos:

- El valor del *learning rate* de **0.002** que alcanzo el **73%** de asertividad con un valor en la función de pérdida del 1.2361 en tan solo **3 épocas**.
- El valor del *learning rate* de **0.004** que alcanzo el **71%** de asertividad con un valor en la función de pérdida del 0.6629 en tan solo **3 épocas**.

En conclusión, tenemos que el valor de **0.002** que viene por defecto es el más adecuado, esto por tener un nivel de asertividad más grande que el de 0.004, sin embargo, hay que tomar en cuenta que su valor en la función de pérdida es mucho mayor al otro candidato. También vemos que no existe otro posible candidato, pues los demás no logran pasar demás del 67% de exactitud en 10 épocas de prueba, entonces decimos que a partir del valor 0.004 en adelante el nivel de asertividad va empeorando.

Ahora probemos mediante el entrenamiento los valores del “vector2” (ver imagen 3.28).

```

/00/00 [=====] - 9s 13ms/sample - loss: 0.6922 - acc: 0.5443 - val_loss: 0.6900 - val_acc: 0.5500
Epoch 3/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6893 - acc: 0.5443 - val_loss: 0.6886 - val_acc: 0.5500
Epoch 4/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6968 - acc: 0.5043 - val_loss: 0.6881 - val_acc: 0.5500
Epoch 5/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6920 - acc: 0.5443 - val_loss: 0.6890 - val_acc: 0.5500
Epoch 6/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6878 - acc: 0.5443 - val_loss: 0.6902 - val_acc: 0.5500
Epoch 7/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6937 - acc: 0.5071 - val_loss: 0.6888 - val_acc: 0.5500
Epoch 8/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6905 - acc: 0.5443 - val_loss: 0.6910 - val_acc: 0.5500
Epoch 9/10
700/700 [=====] - 9s 13ms/sample - loss: 0.6892 - acc: 0.5443 - val_loss: 0.6891 - val_acc: 0.5500
Epoch 10/10
700/700 [=====] - 8s 12ms/sample - loss: 0.6955 - acc: 0.5443 - val_loss: 0.6905 - val_acc: 0.5500
300/300 [=====] - 1s 2ms/sample - loss: 0.6905 - acc: 0.5500
No se pudo encontrar el porcentaje adecuado en 10 epocas PASANDO A otro LR

```

Imagen 3.28: podemos observar que no existe un posible candidato con los valores del vector2.

Como podemos evidenciar desde los valores del “vector” ya se venía advirtiéndose que mientras más grande sea el valor del *learning rate* menor será el porcentaje de asertividad, por lo tanto, concluimos que el modelo no funcionaría bien si tuviese ajustado un valor mayor a 0.004.

Ahora pasamos a probar los valores del “vector3”, estos son menores a los del “vector” (ver imagen 3.29).

```

300/300 [=====] - 1s 2ms/sample - loss: 10.8375 - acc: 0.7400
*****Para el metodo ADAM se utilizo: 3 epocas para llegar a mas del 70% de acertividad con el indice LR= 0.0002
Test on 300 samples: validata on 300 samples

Epoch 2/2
700/700 [=====] - 10s 15ms/sample - loss: 15.7592 - acc: 0.8029 - val_loss: 21.9667 - val_acc: 0.7300
300/300 [=====] - 1s 4ms/sample - loss: 21.9667 - acc: 0.7300
*****Para el metodo ADAM se utilizo: 4 epocas para llegar a mas del 70% de acertividad con el indice LR= 0.0004
Test on 300 samples: validata on 300 samples

Epoch 4/4
700/700 [=====] - 11s 16ms/sample - loss: 68.7483 - acc: 0.7157 - val_loss: 35.1369 - val_acc: 0.7533
300/300 [=====] - 1s 4ms/sample - loss: 35.1369 - acc: 0.7533
*****Para el metodo ADAM se utilizo: 3 epocas para llegar a mas del 70% de acertividad con el indice LR= 0.0006
Test on 300 samples: validata on 300 samples

Epoch 4/4
700/700 [=====] - 11s 16ms/sample - loss: 0.6845 - acc: 0.9500 - val_loss: 6.4317 - val_acc: 0.7567
300/300 [=====] - 1s 4ms/sample - loss: 6.4317 - acc: 0.7567
*****Para el metodo ADAM se utilizo: 5 epocas para llegar a mas del 70% de acertividad con el indice LR= 0.0008
Test on 300 samples: validata on 300 samples

Epoch 4/4
700/700 [=====] - 10s 14ms/sample - loss: 14.1188 - acc: 0.8257 - val_loss: 9.3032 - val_acc: 0.7567
300/300 [=====] - 1s 4ms/sample - loss: 9.3032 - acc: 0.7567
*****Para el metodo ADAM se utilizo: 4 epocas para llegar a mas del 70% de acertividad con el indice LR= 0.0009
Test on 300 samples: validata on 300 samples

```

Imagen 3.29: podemos observar a 5 posibles candidatos para el valor del *learning rate*.

Contamos con 5 valores que llegaron a más del 70%:

- El valor del *learning rate* de **0.0002** que alcanzo el **70%** de asertividad con un valor en la función de pérdida del 10.8375 en tan solo **3 épocas**.
- El valor del *learning rate* de **0.0004** que alcanzo el **73%** de asertividad con un valor en la función de pérdida del 21.9667 en tan solo **4 épocas**.
- El valor del *learning rate* de **0.0006** que alcanzo el **75.33%** de asertividad con un valor en la función de pérdida del 35.1369 en tan solo **3 épocas**.
- El valor del *learning rate* de **0.0008** que alcanzo el **75.66%** de asertividad con un valor en la función de pérdida del 6.4317 en tan solo **5 épocas**.
- El valor del *learning rate* de **0.0009** que alcanzo el **75.67%** de asertividad con un valor en la función de pérdida del 9.3032 en tan solo **4 épocas**.

Tenemos como posibles candidatos a los valores de 0.0006, 0.0008 y 0.0009. sí nos fijamos en el primer candidato su valor de la función de perdida es muy grande en comparación de los demás, así que queda descartado, si nos fijamos en el segundo candidato este posee más una época extra en comparación del tercer candidato, además su valor de asertividad es muy similar, por lo tanto, me quedare con el valor de *learning rate* de 0.0009.

En conclusión, tenemos que los valores de 0.0009 y 0.002 son los candidatos más óptimos para el valor del *learning rate*. **Elegí el valor de 0.0009 como valor de mi *learning rate*** pues tenemos un nivel alto de pronóstico en comparación del valor 0.002, además la diferencia de épocas entre los valores es de tan en 1 además el valor de la función de perdida entre ambas no es demasiada grande, estas pruebas están disponibles para el lector en el siguiente enlace de descarga: https://drive.google.com/file/d/1Gev0ycZfIWl-Ssiw6F3pm2k2z_1OQiaA/view?usp=share_link .

Ahora que demostramos el uso de la red neuronal en mi prototipo por medio de la validación cruzada, encontramos el método de optimización más adecuado a mi *dataset* (ADAM) y hallamos el valor más óptimo para mi *learning rate*, es hora de realizar las

pruebas de validación con ejemplos probando el prototipo que después las analizaremos mediante graficas derivadas de la matriz de confusión.

CAPÍTULO 4: EVALUACIÓN DE RESULTADOS

Esta sección tiene como objetivo demostrar la hipótesis planteada en el capítulo 1, además validar los resultados obtenidos del modelo clasificador en el capítulo 3, para ello se hace uso de métodos como la curva ROC y la de precisión-sensibilidad, sin embargo, antes debo declarar el origen y el proceso de limpieza de los datos que use para entrenar el modelo.

4.1 RECOLECCIÓN DE LOS DATOS.

Los datos de entrenamiento y los de prueba (17540 imágenes diagnosticadas) fueron recolectados de dos fuentes confiables:

- Mediante el ingeniero electrónico Francisco Reales de la universidad Javeriana de Colombia, en total son 2637 imágenes para el entrenamiento y 660 para la prueba.
- La segunda fuente hace referencia al *dataverse* de la universidad de Harvard, en donde se descargó 3561 imágenes para la prueba y 10682 para el entrenamiento.
- Todas separadas en grupos y en carpetas diferentes e individuales.

Para más referencia ir al punto 3.4.1 adquisición.

4.2 ANÁLISIS DE LOS DATOS.

Para poder trabajar con los datos recolectados fue necesario realizar una limpieza, es decir renombrarlos con etiquetas de números positivos enteros, redimensionarlos a 224x224 píxeles, almacenarlos en carpetas según el porcentaje de entrenamiento o de prueba y finalmente asignarles una etiqueta que los defina en los casos de melanoma (1) y carcinoma (0), esto porque el modelo entrenara y validara cada ejemplo mediante estas etiquetas, para más referencia visitar el punto 3.4.3 preprocesamiento.

4.3 DEMOSTRACIÓN DE LA HIPÓTESIS.

Para demostrar mi hipótesis mencionada en el punto 1.4 hipótesis, se hace uso de dos métodos de validación: la curva de ROC y el método de la precisión-sensibilidad, estos métodos están explicados teóricamente en el punto 2.1.7 métodos de evaluación para mi modelo.

Ambos métodos estadísticos sirven para validar modelos de clasificación, sin embargo, existen algunos puntos a tomar en cuenta antes de aplicarlos en mi modelo, estos están declarados en la investigación del ingeniero en telecomunicaciones Luis Torrez de la universidad de Madrid (themachinelearners, 2018):

- Las curvas ROC se deberían utilizar cuando más o menos existen las mismas observaciones para ambas clases y cuando la salida sea un valor **binario**.
- Las curvas de precisión sensibilidad se deberían utilizar cuando existe un notable **desbalance** entre el número de observaciones de cada clase.
- En el caso de las curvas de ROC mientras más cercano a 1 este el valor de la red neuronal entrenada mucho mejor será, pues el 1 significa un modelo que reconoce los casos a la perfección.

En conclusión:

- Utilizare el método de las curvas de ROC al concluir el entrenamiento de mi modelo, pues los casos de melanomas y carcinomas son muy parecidos y poseen bastantes similitudes en rasgos y formas, también se ajusta muy bien a mi salida binaria (carcinoma=0 y melanoma=1).
- También utilizare el método de curvas de precisión-sensibilidad al terminar el proceso de las curvas ROC, esto porque en el *dataset* que poseo contiene un numero dispar de imágenes de melanomas y de carcinomas, lo que lo hace un conjunto de datos desbalanceado a la hora de entrenar mi modelo.

4.3.1 APLICACIÓN DEL MÉTODO DE CURVAS DE ROC DENTRO DEL MODELO.

Para calcular la efectividad de mi clasificador es necesario obtener el porcentaje de predicción de mi modelo respecto a los datos de prueba, seguido de calcular los *scores* de validación respecto a los datos de prueba sin entrenar y los datos de prueba ya entrenados, luego vamos a contrastar los resultados verdaderos positivos contra los falsos positivos, en otras palabras vamos a comparar los casos que pronostico el modelo como positivos a melanoma y que en realidad son portadores de un melanoma contra los que salieron positivos a melanoma y que en realidad son portadores de carcinoma, esto se muestra en la gráfica 4.1:

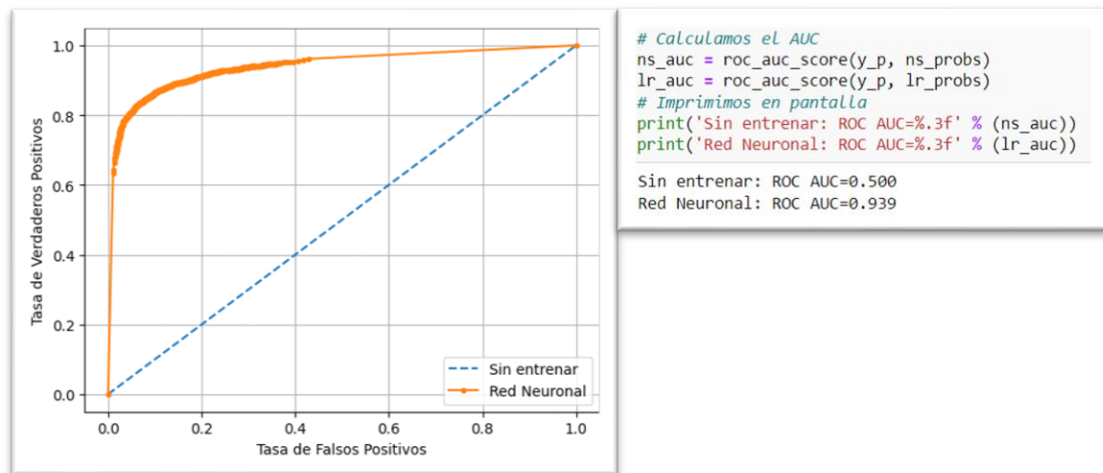


Imagen 4.1: en la parte izquierda observamos la gráfica de la curva ROC con valores aceptables, a la derecha el valor del área por debajo de la curva ROC de los modelos que no están entrenados y los que si están entrenados (Red Neuronal).

Debemos recordar que:

- valores pequeños en el eje **X** indican pocos casos de falsos positivos y muchos de verdaderos negativos.
- valores grandes en el eje **Y** indican casos elevados de verdaderos positivos y pocos de falsos negativos.

Tenemos como resultado:

- el área por debajo de la curva ROC del modelo entrenado nos da como resultado un valor de 0.939, lo que nos dice que su complemento estaría dado por: $1 - 0.939 = 0.061$ para llegar al máximo que en este caso sería 1 (perfección).
- La curva esta más alejada del modelo sin entrenar y más cerca del valor perfecto de asertividad.

Tenemos como conclusión:

- **El modelo es aceptable** para predecir si un paciente pudiese tener un posible caso de melanoma.
- Nuestro modelo tiene valores agrupados muy cercanos a 0 respecto al eje X y valores agrupados cercanos a 1 en el eje Y, lo que significaría que existen más casos de verdaderos negativos y de verdaderos positivos, es decir que **el modelo es apto para predecir si un paciente tuviese un posible caso de melanoma o carcinoma.**

4.3.2 APLICACIÓN DEL MÉTODO DE PRECISIÓN-SENSIBILIDAD DENTRO DEL MODELO.

Para este método necesitaremos de la precisión y de la sensibilidad del modelo, recordemos que la precisión verifica todas las veces que el modelo dio positivo, de cuantas fueron realmente positivas (melanomas) y la sensibilidad es la capacidad de poder detectar correctamente la enfermedad entre los pacientes.

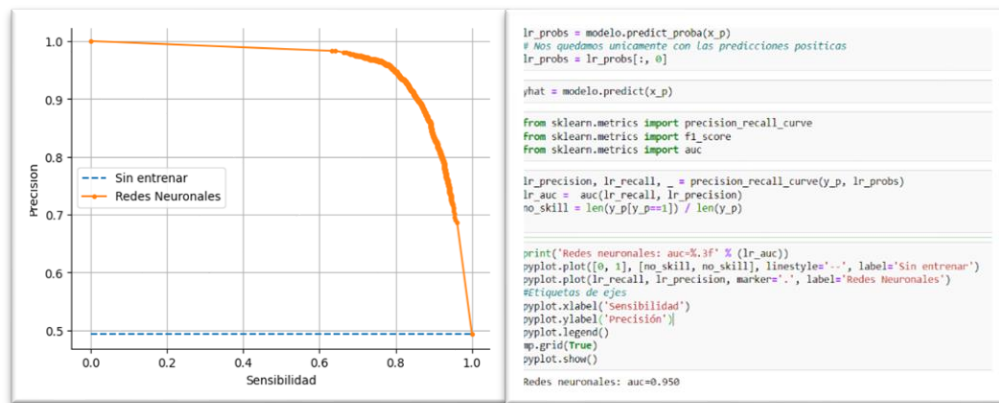


Imagen 4.2: en la parte izquierda observamos la gráfica de la sensibilidad-precisión, a la derecha el código para graficar el resultado y al final se muestra el área debajo de la curva (auc).

Tenemos como resultado:

- El área por debajo de la curva es de un **0.950**, es decir que tanto como sensibilidad y precisión del modelo están bien ajustados.
- Existen una gran cantidad de valores centrados en una sensibilidad grande.
- Podemos observar que cuando el modelo da un diagnóstico verdadero positivo, en la mayoría de las veces acierta con los resultados positivos (melanoma) en el paciente.
- En la parte de la sensibilidad podemos apreciar que el modelo es capaz de detectar un posible caso de melanoma ya que está muy cerca de alcanzar la línea de la perfección.

Concluimos que:

- El modelo **puede predecir muy bien un diagnóstico de un melanoma.**
- La precisión y la sensibilidad casi llegan a la perfección de diagnosticar casos de melanomas en pacientes.

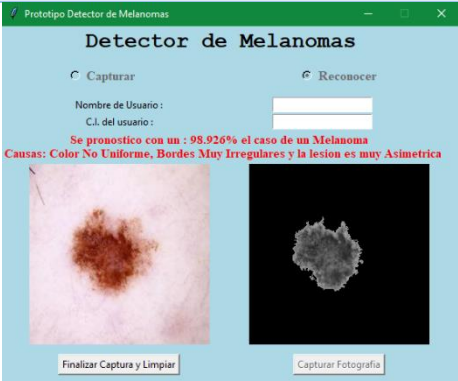
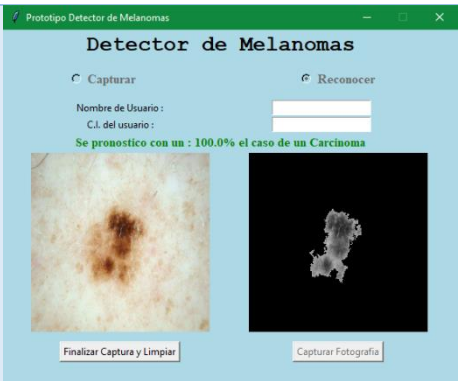
Finalmente, con los resultados presentados por los métodos de validación de la curva ROC y sensibilidad-precisión nos dan a entender que el modelo esta apto para poder reconocer casos de posibles melanoma y carcinomas en pacientes.

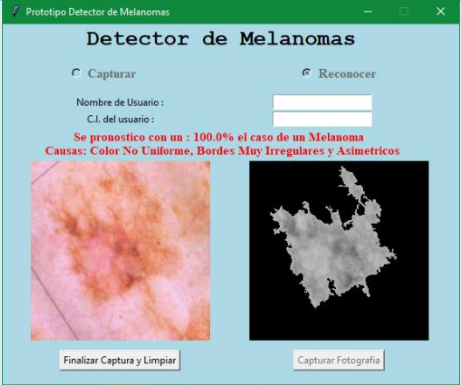
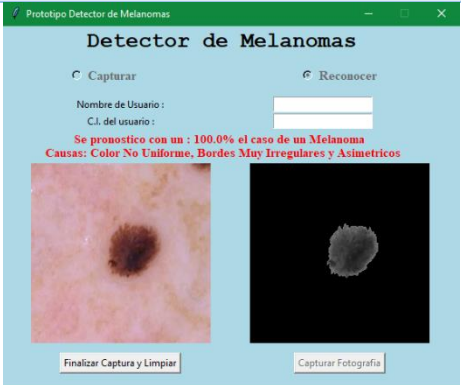
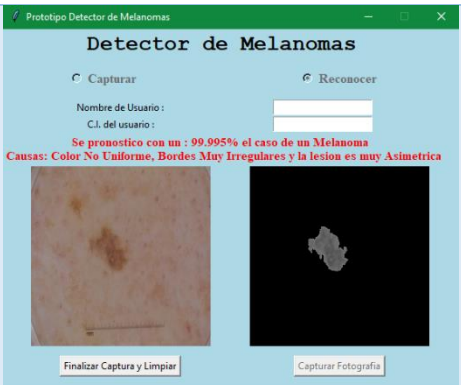
4.4 DEMOSTRACIÓN POR CASOS.

En esta sección pasaremos a probar el prototipo con 20 imágenes de la carpeta prueba, en donde 10 serán de carcinoma y 10 de melanoma, para esta prueba tome imágenes que tengan características muy similares (color, forma, etc.) para poner a prueba la efectividad del prototipo a la hora de clasificar.

- Imágenes aleatorias de la carpeta **melanoma** (10 ejemplos).

Prototipo	Diagnostico
	Verdadero positivo (melanoma 100%)
	Verdadero positivo (melanoma 100%)

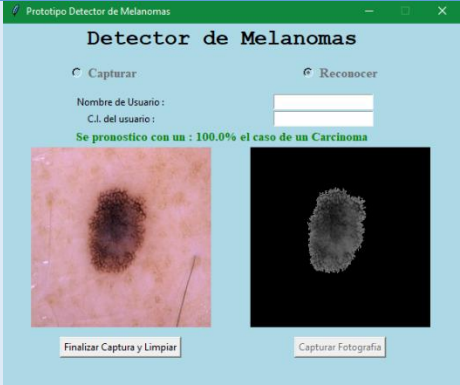
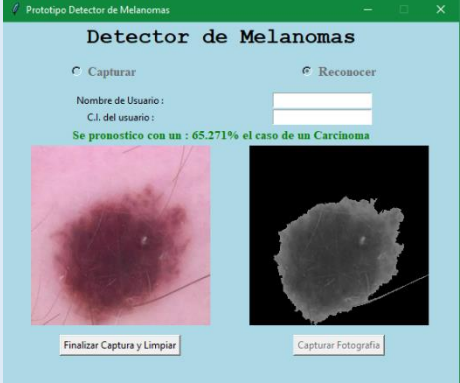
 Prototipo Detector de Melanomas Detector de Melanomas ☐ Capturar ☒ Reconocer Nombre de Usuario : C.I. del usuario : Se pronostico con un : 100.0% el caso de un Melanoma Causas: Color No Uniforme, Bordes Muy Irregulares y Asimetricos Finalizar Captura y Limpiar Capturar Fotografia	Verdadero positivo (melanoma 100%)
 Prototipo Detector de Melanomas Detector de Melanomas ☐ Capturar ☒ Reconocer Nombre de Usuario : C.I. del usuario : Se pronostico con un : 98.926% el caso de un Melanoma Causas: Color No Uniforme, Bordes Muy Irregulares y la lesion es muy Asimetrica Finalizar Captura y Limpiar Capturar Fotografia	Verdadero positivo (melanoma 98.9%)
 Prototipo Detector de Melanomas Detector de Melanomas ☐ Capturar ☒ Reconocer Nombre de Usuario : C.I. del usuario : Se pronostico con un : 100.0% el caso de un Carcinoma Finalizar Captura y Limpiar Capturar Fotografia	Falso negativo (carcinoma 100%)
 Prototipo Detector de Melanomas Detector de Melanomas ☐ Capturar ☒ Reconocer Nombre de Usuario : C.I. del usuario : Se pronostico con un : 100.0% el caso de un Melanoma Causas: Color No Uniforme, Bordes Muy Irregulares y Asimetricos Finalizar Captura y Limpiar Capturar Fotografia	Verdadero positivo (melanoma 100%)

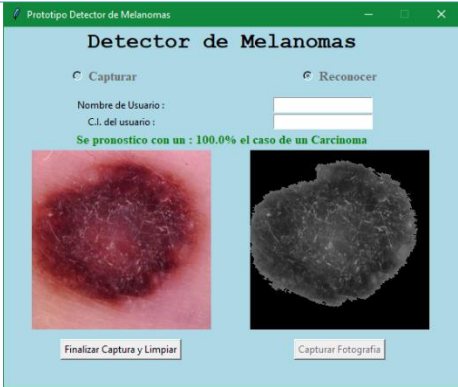
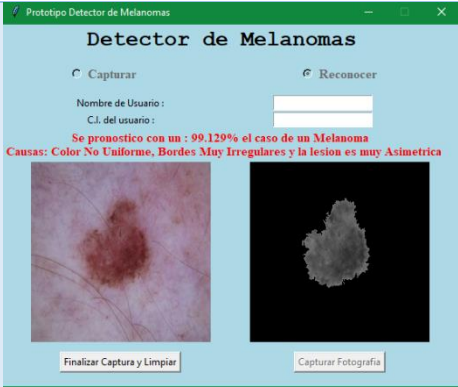
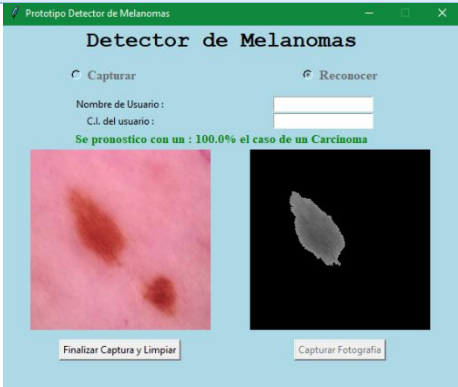
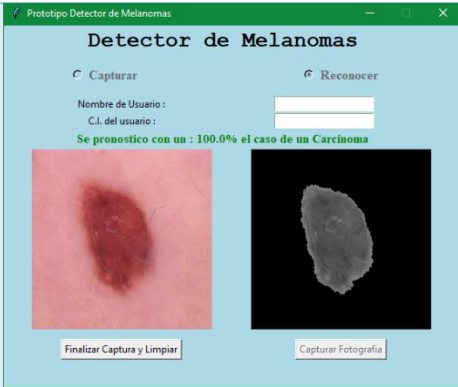
		Verdadero positivo (melanoma 100%)
		Verdadero positivo (melanoma 100%)
		Verdadero positivo (melanoma 99%)
		Verdadero positivo (melanoma 100%)

El prototipo aserto en 9 diagnósticos de manera correcta y solo fallo en 1 diagnostico. Recordemos que tome como ejemplo las imágenes que más se parecen a casos de carcinomas.

Tabla 4.1: En la parte izquierda observamos en texto claro la predicción del ejemplo de la derecha (caso melanoma).

- Ahora realizaremos 10 ejemplos con la carpeta de carcinomas:

Prototipo	Diagnostico
	Verdadero Negativo (100% carcinoma)
	Verdadero Negativo (100% carcinoma)
	Verdadero Negativo (65% carcinoma)

	Verdadero Negativo (100% carcinoma)
	Verdadero positivo (99% melanoma)
	Verdadero Negativo (100% carcinoma)
	Verdadero Negativo (100% carcinoma)

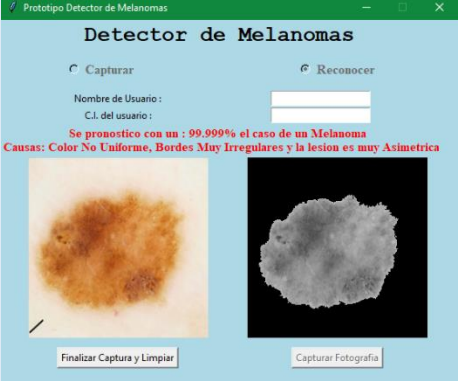
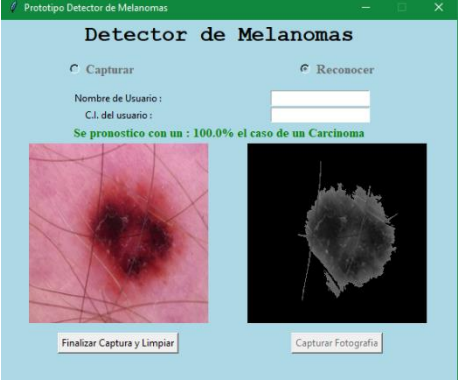
	Verdadero positivo (98% melanoma)
	Verdadero positivo (99% melanoma)
	Verdadero Negativo (100% carcinoma)
El prototipo aserto en 7 diagnósticos de manera correcta y fallo en 3 diagnósticos.	

Tabla 4.2: en la parte izquierda observamos los resultados del diagnóstico del prototipo y a la derecha el diagnostico (carcinoma).

Según los resultados del punto 4.4 demostración por casos, se llega a formar una matriz de confusión (ver tabla 4.3) de la siguiente manera:

- De 10 casos de melanomas se detectaron correctamente 9.
- De 10 casos de carcinomas se detectaron correctamente 7.

predicción			
realidad		carcinoma	melanoma
	carcinoma	7	3
	melanoma	1	9

Tabla 4.3: observamos que los valores de color azul son los casos que se detectaron correctamente y los de color rojo los que no.

CAPÍTULO 5: CONCLUSIONES Y RECOMENDACIONES

5.1 CONCLUSIONES.

En este capítulo se culmina la presente investigación y el desarrollo del prototipo detector de melanomas basado en redes neuronales convolucionales, también se debe recalcar que este prototipo llega a identificar un posible caso de melanoma con una confiabilidad de más del 80%, permitiendo a la vez cumplir con los objetivos planteados en el presente trabajo.

- Este prototipo obtiene una imagen del área afectada del paciente, esta imagen es redimensionada a 224x224 píxeles para que pueda ser compatible con la entrada de la red neuronal convolucional.
- Para el entrenamiento del prototipo fue necesario obtener una gran cantidad de ejemplos diagnosticados con los casos de melanomas y carcinomas, gracias a la página *Dataverce* de la universidad de Harvard y al ingeniero Francisco Reales se obtuvo más de 17000 ejemplos.
- Según los documentos referidos en los puntos del estado del arte y antecedentes, los resultados de la visión por computador detectando lunares, manchas o melanomas utilizando métodos de aprendizaje automático como k-vecinos cercanos, soporte de máquinas vectoriales (SVM) entre otros, no fueron de gran ayuda a la hora de detectar dichos objetos o diagnosticar los casos referidos, esto se debe a que en muchas de estas técnicas solo identifican algunos patrones de los objetos como ser: bordes, el tamaño o el patrón de color, sin embargo en mi prototipo se utilizó la metodología de las redes neuronales convolucionales para detectar múltiples patrones que correspondan a una clase en particular (melanoma o carcinoma) y así obtener un diagnóstico predictivo más eficaz y por lo tanto subir el porcentaje de asertividad.

- También se desarrolló un sistema gráfico para la interacción del médico dermatólogo con el prototipo, este se desarrolló en TKinter del propio Python3, este prototipo cuenta con 2 botones selectores principales las cuales cumplen las funciones de: sacar una fotografía a la lesión del paciente y diagnosticar un posible caso de melanoma.
- Una vez abierto el entorno grafico del prototipo y seleccionada la opción de diagnosticar, el proceso tardaría dependiendo del equipo computacional que tenga el medico dermatólogo, sin embargo se realizó un diagnóstico con una laptop que cuenta con un procesador i3 de 4° generación sin GPU dedicada y una computadora de escritorio con un procesador i7 de 11° generación con GPU RTX 3060 y se obtuvo un rango de 10 a 3 segundos de procesamiento de la respuesta correspondientemente a los equipos mencionados.

5.2 RECOMENDACIONES.

- Para poder aumentar la exactitud de reconocimiento del prototipo detector de melanomas, es necesario realizar estas dos acciones:
 - aumentar la colección de imágenes en diagnósticos de melanomas y carcinomas dentro de la carpeta “data/entrenamiento” y aplicar el entrenamiento del prototipo.
 - también es necesario modificar la estructura de la red convolucional y la red neuronal profunda, añadiendo o quitando capas y neuronas, de tal manera que el resultado provendría de la prueba y error.
- Para obtener buenos resultados en el diagnóstico del prototipo es necesario contar con una buena iluminación al momento de tomar una fotografía al paciente, en caso contrario las fotografías tendrían una sombra alrededor de la lesión lo cual podría generar un diagnóstico erróneo con tendencias a ser un melanoma.

FUENTES DE LA INFORMACIÓN.

ANEXOS.

En esta sección insertare todas las evidencias respecto a las pruebas de evaluación de mi prototipo por medio de expertos médicos dermatólogos y programadores.

Se realizo una encuesta con 7 preguntas de retroalimentación dirigido a los expertos médicos dermatólogos (ver anexo 1) y se realizaron cesiones de demostración y evaluación de mi prototipo a expertos programadores.

A continuación, se presenta la encuesta de retroalimentación que se realizó al médico dermatólogo Cristian Quiroga del centro de salud particular QuiroDerma ubicado en Ciudad Satélite de El Alto (ver anexo 2). Ademas se adjunta el sello de la institucion y una fotografia donde se evidencia la visita al centro médico.

Cuestionario hacia expertos en el área de la medicina (dermatólogos).

1. ¿Qué tan satisfecho(a) está con la **confiabilidad** del prototipo detector de melanomas?
 - a. Extremadamente satisfecho(a).
 - ☒ b. Muy satisfecho(a).
 - c. Algo Satisfecho(a).
 - d. No tan satisfecho(a).
 - e. Nada satisfecho(a).
2. ¿Qué tan satisfecho(a) está con la **facilidad** de uso del prototipo detector de melanomas?
 - a. Extremadamente satisfecho(a).
 - ☒ b. Muy satisfecho(a).
 - c. Algo Satisfecho(a).
 - d. No tan satisfecho(a).
 - e. Nada satisfecho(a).
3. ¿Qué tan satisfecho(a) está con la **aparición** del prototipo detector de melanomas?
 - a. Extremadamente satisfecho(a).
 - ☒ b. Muy satisfecho(a).
 - c. Algo Satisfecho(a).
 - d. No tan satisfecho(a).
 - e. Nada satisfecho(a).
4. ¿Qué tan **útil** es el prototipo detector de melanomas a la hora de dar un diagnóstico de un posible caso de melanoma?
 - ☒ a. Muy útil.
 - b. Lo usaría de vez en cuando.
 - c. No lo usaría.
5. ¿En el momento del uso del prototipo detector de melanomas, existió algún problema de **congelamiento** o bloqueo?
 - ☒ a. No.
 - b. En caso de que sí hubiese existido algún tipo de congelamiento por favor detalle la opción que ejecuto antes de que existiese el problema.
6. ¿Recomendaría el prototipo detector de melanomas a algún otro colega médico dermatólogo?
 - ☒ a. Si lo recomendaría.
 - b. No lo recomendaría.

7. Si tuviese alguna sugerencia para mejorar este software, por favor sea libre de dejar un comentario breve.

Justifica la respuesta del sistema es bueno a
costos es la calidad (Derma base para)

IRSIDERMA
Dermatológico Local

Anexo 1: este cuestionario lo completo el doctor Cristian Quiroga, las preguntas sirvieron de retroalimentación para mejorar el prototipo y su visualización en los resultados.



Anexo 2: fotografía del consultorio del doctor Cristian Quiroga probando el prototipo detector de melanomas.

Seguidamente se anexa dos vínculos donde el primero lleva a la prueba del prototipo por el grupo “NeuroTech” de la Universidad Católica Boliviana y el segundo a la prueba realizada por el M Sc. Reynaldo Zeballos Daza.

1. Prueba del prototipo por el representante de Neurotech Bolivia:
https://drive.google.com/file/d/1OGMhTUXix2kMnQsQmXQBtTzZiL8ThqfZ/view?usp=share_link
2. Prueba del prototipo realizada por el M Sc. Javier Reynaldo Zeballos Daza:
https://drive.google.com/file/d/1a_BDNmvhRm_q3hZ4l1vHoXcU8UFhvyX/view?usp=share_link

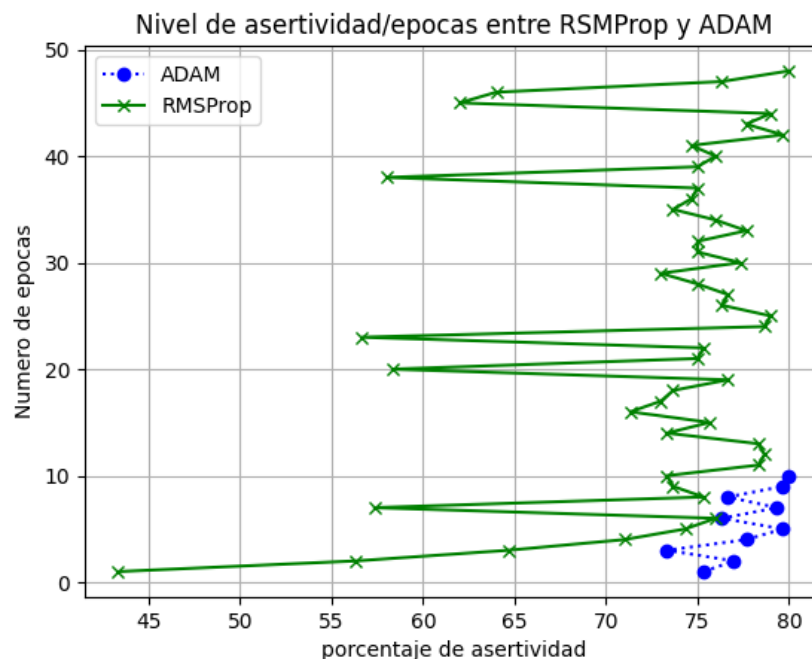
Para llegar al resultado del 88% de asertividad, se tuvo que pasar muchos ciclos de desarrollo en la parte de la red neuronal y convolucional, estos se describen de la siguiente manera:

1. Para encontrar el método de optimización correcto tuve que probar con diferentes valores de *Learnin rate* y estructuras convolucionales y neuronales, en este caso

utilice el valor de 0.001 y trabaje con 1 capa convolucional de 32 *kernels* y una capa oculta de 50 neuronas, este ciclo lo descarte por que los valores de “val_acc” (asertividad) eran muy distantes de cada época de entrenamiento por lo que generaban distorsión de los datos a la hora del entrenamiento, esto se lo puede evidenciar en el documento que dejo en el siguiente link.

a. El link de GitHub es el siguiente:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/1_MetodosDeOptimizacion_2.ipynb

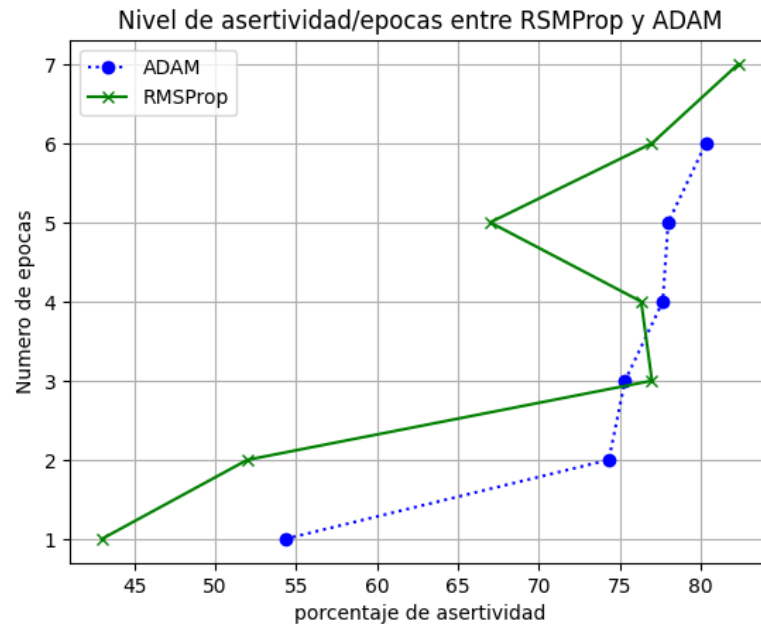


Anexo 3: grafica donde se muestra el comportamiento de los 2 métodos en cada época de entrenamiento.

2. En este ciclo intente modificando el valor del *learning rate* a 0.0009, lo cual dio el resultado donde el método RMSprop ganaba al método ADAM en asertividad, sin embargo, el método ganador lo hacía en una época más respecto a su competidor y los resultados de asertividad en cada época no eran continuos a diferencia de su competidor, sin embargo, los resultados eran malos, por lo cual intente mejorar la estructura de la red convolucional para mejorar el resultado.

a. Link del documento de HitHub:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/1_MetodosDeOptimizacion.ipynb



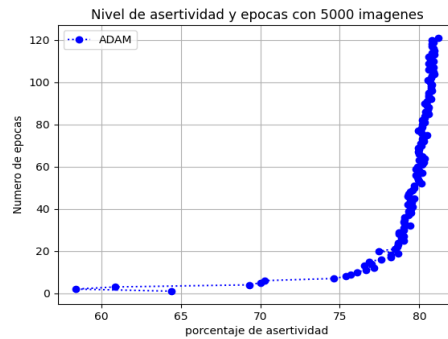
Anexo 4: grafica donde se muestra el comportamiento de los 2 métodos en cada época de entrenamiento.

3. En este ciclo, utilicé el valor del *learning rate* de 0.0008 y probé con diferentes estructuras dentro de la red convolucional y neuronal, en este caso probe con 5000 imágenes y dos capas convolucionales de 8 y 16 *kenels* respectivamente, los resultados llegaban al 81% de asertividad en 121 épocas, pero cuando intenté con más imágenes, este valor cayo de manera brusca.

a. El link se en encuentra:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/Prototipo-10000-2CapasConvolucionales.ipynb

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 8)	224
conv2d_1 (Conv2D)	(None, 220, 220, 16)	1168
max_pooling2d (MaxPooling2D)	(None, 110, 110, 16)	0
flatten (Flatten)	(None, 193600)	0
dense (Dense)	(None, 128)	24780928
dense_1 (Dense)	(None, 70)	9030
dense_2 (Dense)	(None, 1)	71
Total params: 24,791,421		
Trainable params: 24,791,421		
Non-trainable params: 0		



Anexo 5: a la izquierda se muestra la estructura que utilice y a la derecha el nivel de comportamiento de cada época respecto al asertividad.

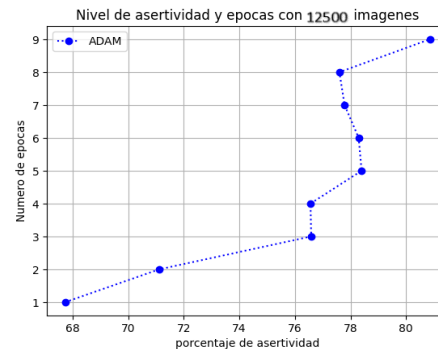
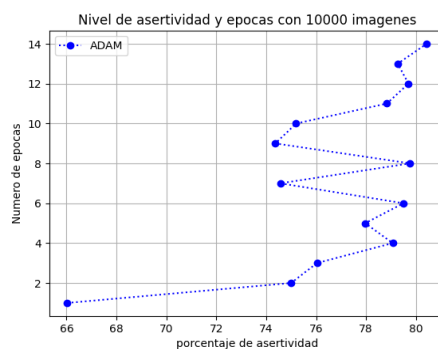
4. En este ciclo solo utilice una capa convolucional de 32 *kernels* y una capa oculta en mi red neuronal de 50 neuronas, probe para 10000 y 12500 imágenes ambos llegan al 80% de asertividad, pero con distintas épocas de entrenamiento, además tiene mucha distorsión entre “val_acc” de cada época, por lo cual descarte esta estructura.

a. Link de los códigos de 10000 y 12500 imágenes en GitHub es:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/Prototipo-10000.ipynb

y

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/Prototipo-12500.ipynb



Anexo 6: a la izquierda se muestra el nivel de asertividad -épocas con 10000 imágenes y a la derecha con 12500 imágenes.

5. También pude probar la anterior arquitectura para un nivel de más del 82% y con 17000, sin embargo, no se pudo sobrepasar más del 79%, es decir que el nivel de

asertividad de la anterior arquitectura ira en decremento después de 12500 imágenes de entrenamiento.

a. Link del documento en GitHub:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/Prototipo-17000-2CapasConvolucionales2NeuronalesFALLO.ipynb

```
1 modelo_adam = keras.optimizers.Adam(learning_rate=0.0008)
2 modelo.compile(optimizer=modelo_adam,loss='binary_crossentropy',metrics=['accuracy'])
3 v_eA=[]
4 v_aA=[]
5 evaluacion(modelo,82,"ADAM",v_eA,v_aA)

12000/12000 [=====] - 130s 11ms/sample - loss: 5.5004e-04 - acc: 0.9998 - val_loss: 3.3507 - val_acc: 0.7902
5000/5000 [=====] - 8s 2ms/sample - loss: 3.3507 - acc: 0.7902
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.0651 - acc: 0.9823 - val_loss: 2.6268 - val_acc: 0.7414
5000/5000 [=====] - 8s 2ms/sample - loss: 2.6268 - acc: 0.7414
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.0189 - acc: 0.9950 - val_loss: 2.2156 - val_acc: 0.7662
5000/5000 [=====] - 8s 2ms/sample - loss: 2.2156 - acc: 0.7662
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.0033 - acc: 0.9990 - val_loss: 2.9514 - val_acc: 0.7738
5000/5000 [=====] - 8s 2ms/sample - loss: 2.9514 - acc: 0.7738
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 130s 11ms/sample - loss: 0.0943 - acc: 0.9755 - val_loss: 2.1765 - val_acc: 0.7522
5000/5000 [=====] - 8s 2ms/sample - loss: 2.1765 - acc: 0.7522
Train on 12000 samples, validate on 5000 samples
```

Anexo 7: se muestra los resultados de los entrenamientos, podemos ver que después de llegar al 79% de asertividad esta baja.

6. Luego probe añadiendo 2 capas convolucionales de 8 y 16 *kernels* respectivamente y añadí neuronas a la capa oculta hasta llegar a 70, todo este proceso creo 24791421 parámetros calculables, mucho mayores a los anteriores modelos y por lo tanto tardo mucho más en cada entrenamiento, también debo aclarar que el nivel de asertividad no paso de más del 77%.

a. Link del documento en GitHub:

https://github.com/NaNoFernando/anexos_para_tesis/blob/b645bb957d6c3b91cbdc676f301ebdf701f99c05/Prototipo-17000-2CapasConvolucionales MUESTRAfallo.ipynb

```

1 modelo_adam = keras.optimizers.Adam(learning_rate=0.0008)
2 modelo.compile(optimizer=modelo_adam,loss='binary_crossentropy',metrics=['accuracy'])
3 v_eA=[]
4 v_aA=[]
5 evaluacion(modelo,85,"ADAM",v_eA,v_aA)

12000/12000 [=====] - 131s 11ms/sample - loss: 5.0612e-10 - acc: 1.0000 - val_loss: 4.3044 - val_acc: 0.7714
5000/5000 [=====] - 9s 2ms/sample - loss: 4.3044 - acc: 0.7714
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.9620 - acc: 0.8975 - val_loss: 0.9730 - val_acc: 0.6730
5000/5000 [=====] - 9s 2ms/sample - loss: 0.9730 - acc: 0.6730
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.5156 - acc: 0.7792 - val_loss: 1.0772 - val_acc: 0.7016
5000/5000 [=====] - 9s 2ms/sample - loss: 1.0772 - acc: 0.7016
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.3007 - acc: 0.8740 - val_loss: 1.3308 - val_acc: 0.6582
5000/5000 [=====] - 9s 2ms/sample - loss: 1.3308 - acc: 0.6582
Train on 12000 samples, validate on 5000 samples
12000/12000 [=====] - 131s 11ms/sample - loss: 0.2505 - acc: 0.8925 - val_loss: 1.8760 - val_acc: 0.7122
5000/5000 [=====] - 9s 2ms/sample - loss: 1.8760 - acc: 0.7122
Train on 12000 samples, validate on 5000 samples

```

Anexo 8: se muestra claramente un descenso en el valor del nivel de asertividad después de llegar a 77%.

7. Como pude demostrar llegar al nivel del 88% de asertividad me costó un montón de pruebas y errores, todas estas pruebas mostradas y muchas más con diferentes estructuras convolucionales y neuronales están dentro de mi colección en GitHub que se puede encontrar en el siguiente link: https://github.com/NaNoFernando/anexos_para_tesis.git.

REFERENCIAS BIBLIOGRÁFICAS.

acervolima.com. (24 de enero de 2022). *tecnicas-de-optimizacion-para-gradient-descent/*.

Recuperado el 29 de abril de 2022, de www.acervolima.com:

<https://es.acervolima.com/tecnicas-de-optimizacion-para-gradient-descent/>

aprendemachinelearning. (29 de noviembre de 2018). *www.aprendemachinelearning.com*.

Recuperado el 16 de noviembre de 2022, de */como-funcionan-las-convolutional-neural-networks-vision-por-ordenador/*:

<https://www.aprendemachinelearning.com/como-funcionan-las-convolutional-neural-networks-vision-por-ordenador/>

asana.com. (9 de agosto de 2019). */what-is-kanban*. Recuperado el 23 de marzo de 2022, de

asana.com/es/resources/what-is-kanban: <https://asana.com/es/resources/what-is-kanban>

atriainnovation. (22 de octubre de 2019). *www.atriainnovation.com*. Recuperado el 17 de

agosto de 2022, de *que-son-las-redes-neuronales-y-sus-funciones*:

<https://www.atriainnovation.com/que-son-las-redes-neuronales-y-sus-funciones/>

Auto Avance. (8 de octubre de 2019). *Asi funciona un Tesla*. Recuperado el 18 de noviembre de

2021, de Auto Avance blog tecnico automotriz: [https://www.autoavance.co/blog-tecnico-automotriz/tesla-](https://www.autoavance.co/blog-tecnico-automotriz/tesla-funcionamiento/#:~:text=Por%20%C3%BAltimo%2C%20el%20Tesla%20utiliza,dirigida%20directamente%20a%20las%20ruedas.)

[funcionamiento/#:~:text=Por%20%C3%BAltimo%2C%20el%20Tesla%20utiliza,dirigida%20directamente%20a%20las%20ruedas.](https://www.autoavance.co/blog-tecnico-automotriz/tesla-funcionamiento/#:~:text=Por%20%C3%BAltimo%2C%20el%20Tesla%20utiliza,dirigida%20directamente%20a%20las%20ruedas.)

Baptist Health South Florida. (29 de marzo de 2019). *baptisthealth.net*. Recuperado el 7 de

agosto de 2022, de baptisthealth.net/baptist-health-news:

[https://baptisthealth.net/baptist-health-news/es/detectando-el-cancer-de-la-piel-una-nueva-tecnologia-avanzada-de-imagenes-corporales-ahora-esta-disponible-en-miami-cancer-](https://baptisthealth.net/baptist-health-news/es/detectando-el-cancer-de-la-piel-una-nueva-tecnologia-avanzada-de-imagenes-corporales-ahora-esta-disponible-en-miami-cancer-institute/#:~:text=Waibel, varias%20formas%2C%20tama%C3%B1os%20y%20colores.)

[institute/#:~:text=Waibel, varias%20formas%2C%20tama%C3%B1os%20y%20colores.](https://baptisthealth.net/baptist-health-news/es/detectando-el-cancer-de-la-piel-una-nueva-tecnologia-avanzada-de-imagenes-corporales-ahora-esta-disponible-en-miami-cancer-institute/#:~:text=Waibel, varias%20formas%2C%20tama%C3%B1os%20y%20colores.)

BBVA. (11 de junio de 2020). *Inteligencia artificial para detectar COVID-19 en menos de un*

segundo sobre radiografías. Recuperado el 17 de noviembre de 2021, de BBVA en el mundo: <https://www.bbva.com/es/inteligencia-artificial-para-detectar-covid-19-en-menos-de-un-segundo-sobre-radiografias/>

BENSE, T. (2 de noviembre de 2007). *teledet.com.uy*. Recuperado el 14 de diciembre de 2021,

de BENSE, T. *tutorial-imagenes-satelitales/imagenes-satelitales-tutorial*:

<http://www.teledet.com.uy/tutorial-imagenes-satelitales/imagenes-satelitales-tutorial.htm>

Bootcamp AI. (3 de septiembre de 2019). *7-increíbles-ejemplos-de-visión-por-computadora*. Recuperado el 28 de agosto de 2022, de <https://bootcampai.medium.com/>:
<https://bootcampai.medium.com/7-incre%C3%ADbles-ejemplos-de-visi%C3%B3n-por-computadora-835c307b76aa>

cancer.org. (1 de junio de 2019). *cancer.org*. Recuperado el 22 de agosto de 2022, de factores-de-riesgo: <https://www.cancer.org/es/cancer/cancer-de-piel-tipo-melanoma/causas-riesgos-prevencion/factores-de-riesgo.html>

cancer.org. (27 de diciembre de 2019). *tratamientos-y-efectos-secundarios*. Recuperado el 23 de agosto de 2022, de www.cancer.org:
<https://www.cancer.org/es/tratamiento/tratamientos-y-efectos-secundarios/tipos-de-tratamiento/inmunoterapia/citocinas.html>

carislifesciences. (24 de febrero de 2018). *www.carislifesciences.com*. Recuperado el 28 de septiembre de 2022, de www.carislifesciences.com/products-and-services/molecular-profiling: https://www.carislifesciences.com/products-and-services/molecular-profiling/?gclid=CjwKCAjwv4SaBhBPEiwA9YzZvDk4Tfbyu-N61ufLmveWgwdip1rFReZosaltMiBqll9CLiZfptjXTxoC_8MQAvD_BwE

Carlos Santana. (7 de septiembre de 2020). *Redes convolucionales*. Recuperado el 1 de junio de 2022, de DotCsv Carlos Santana: <https://www.mtconsulting.es/agencia-conferenciantes-barcelona/contratar-carlos-santana-dotcsv>

CBINSIGHTS. (12 de septiembre de 2019). *From Drug R&D To Diagnostics: 90+ Artificial Intelligence Startups In Healthcare*. Recuperado el 1 de diciembre de 2021, de RESEAR CHBRIEFS.

cinfasalud. (9 de junio de 2015). *cinfa.com*. Recuperado el 15 de agosto de 2022, de cinfasalud.cinfa.com: <https://cinfasalud.cinfa.com/p/melanoma/>

Codificando_Bits. (27 de marzo de 2019). *Codificando_Bits.com*. Recuperado el 17 de 1 de 2023, de [redes-convolucionales-introduccion](http://www.codificandobits.com):
<https://www.codificandobits.com/blog/redes-convolucionales-introduccion/>

codigoespagueti. (14 de julio de 2020). *codigoespagueti.com*. Recuperado el 28 de agosto de 2022, de <https://codigoespagueti.com/>:
<https://codigoespagueti.com/noticias/tecnologia/que-es-y-como-se-clasifica-la-inteligencia-artificial/>

- CRESPO, ALGORRY. (10 de septiembre de 2019). *¿Deep Learning y clasificación de imágenes?* Recuperado el 14 de diciembre de 2021, de [idecor.cba.gov.ar introduccion-a-las-redes-neuronales-convolucionales-para-clasificacion-de-imagenes](https://idecor.cba.gov.ar/introduccion-a-las-redes-neuronales-convolucionales-para-clasificacion-de-imagenes): <https://idecor.cba.gov.ar/introduccion-a-las-redes-neuronales-convolucionales-para-clasificacion-de-imagenes>
- CRESPO, F. y ALGORRY A. (10 de septiembre de 2020). *idecor.cba.gov.ar*. Recuperado el 21 de julio de 2022, de <https://idecor.cba.gov.ar>: <https://idecor.cba.gov.ar/introduccion-a-las-redes-neuronales-convolucionales-para-clasificacion-de-imagenes>
- Crespo, I. (2020). una selva de sinapsis. En I. C. Perlego, *Lo que escondes, una selva de sinapsis* (pág. 150). Barcelona: Perlego.
- dataEvo. (14 de julio de 2018). *Validación Cruzada (Cross Validation) en Machine Learning*. Recuperado el 10 de diciembre de 2022, de www.dataevo.com.ar: <https://waisdata.com/en/servicios/modelos-personalizados/>
- diegocalvo. (7 de diciembre de 2018). <https://www.diegocalvo.es/>. Recuperado el 2 de agosto de 2022, de *funcion-de-activacion-redes-neuronales*: <https://www.diegocalvo.es/funcion-de-activacion-redes-neuronales/>
- El Pais. (8 de Mayo de 2021). *La joven española que utiliza la inteligencia artificial para detectar el cáncer*. Recuperado el 17 de noviembre de 2021, de El Pais Transformación Digital: <https://elpais.com/tecnologia/2021-05-09/la-joven-espanola-que-utiliza-la-inteligencia-artificial-para-detectar-el-cancer.html>
- ENZYME BLOG. (20 de julio de 2020). *enzymeadvisinggroup que son las redes neuronales y como se aplican*. Recuperado el 7 de diciembre de 2021, de ENZYME BLOG advising group: <https://blog.enzymeadvisinggroup.com/redes-neuronales-que-son-y-aplicaciones>
- Enzyme F. (23 de julio de 2018). *Enzyme F*. Recuperado el 22 de agosto de 2022, de *redes-neuronales-que-son-y-aplicaciones*: <https://enzyme.biz/blog/redes-neuronales-que-son-y-aplicaciones>
- fuerzatres. (21 de julio de 2011). *fuerzatres.com*. Recuperado el 24 de abril de 2022, de www.fuerzatres.com: www.fuerzatres.com/kanban
- Gliese710. (5 de noviembre de 2019). *exponentis.es*. Recuperado el 16 de noviembre de 2022, de *como-dividir-un-conjunto-de-entrenamiento-en-dos-partes-train-test-split*: <http://exponentis.es/como-dividir-un-conjunto-de-entrenamiento-en-dos-partes-train-test-split>

- GONZÁLES, L. (16 de septiembre de 2018). *ligdigonzalez.com*. Recuperado el 14 de diciembre de 2021, de GONZÁLES, L. diferencia-entre-aprendizaje-supervisado-y-no-supervisado: <https://ligdigonzalez.com/diferencia-entre-aprendizaje-supervisado-y-no-supervisado>
- HealtBigData. (2 de agosto de 2020). *juanbarrios.com*. Recuperado el 3 de agosto de 2022, de HealtBigData_redes-neurales-convolucionales: <https://www.juanbarrios.com/redes-neurales-convolucionales/>
- iartificial. (18 de octubre de 2020). *www.iartificial.net*. Recuperado el 21 de marzo de 2022, de redes-neuronales-desde-cero-ii-algo-de-matematicas: <https://www.iartificial.net/redes-neuronales-desde-cero-ii-algo-de-matematicas/>
- Iberomed. (17 de julio de 2019). *Iberomed.com*. Recuperado el 4 de agosto de 2022, de <https://www.iberomed.com/es>: <https://www.iberomed.es/blog/2019/07/17/abrir-una-clinica-dermatologica-este-es-el-material-que-necesitas/>
- IBM. (23 de marzo de 2021). *ibm.com*. Recuperado el 10 de marzo de 2022, de [ibm.com/conceptos basicos de redes neuronales: https://www.ibm.com/docs/es/spss-modeler/SaaS?topic=networks-basics-neural](https://www.ibm.com/docs/es/spss-modeler/SaaS?topic=networks-basics-neural)
- Instituto Nacional del Cancer. (10 de enero de 2019). *aprendizaje-profundo-cancer-cuello-uterino-examen-de-deteccion*. Recuperado el 10 de diciembre de 2021, de [cancer.gov Instituto Nacional del Cancer: https://www.cancer.gov/espanol/noticias/comunicados-de-prensa/2019/aprendizaje-profundo-cancer-cuello-uterino-examen-de-deteccion](https://www.cancer.gov/espanol/noticias/comunicados-de-prensa/2019/aprendizaje-profundo-cancer-cuello-uterino-examen-de-deteccion)
- Instituto nacional del cancer. (21 de Julio de 2020). *NIH*. Recuperado el 01 de enero de 2022, de NIH Instituto nacional del cancer melanoma: <https://www.cancer.gov/espanol/publicaciones/diccionarios/diccionario-cancer/def/melanoma>
- Inti.com.bo. (21 de julio de 2020). *www.inti.com.bo*. Recuperado el 14 de febrero de 2022, de www.inti.com.bo/la-sobreexposicion-al-sol-causaria-el-90-de-los-casos-de-cancer-de-piel-en-bolivia: <https://www.inti.com.bo/la-sobreexposicion-al-sol-causaria-el-90-de-los-casos-de-cancer-de-piel-en-bolivia/>
- IVO fundacion de ecologia valediana. (14 de septiembre de 2015). *ivo.com*. Recuperado el 2022 de marzo de 16, de cancerdepielmelanomaycarcinoma.com: www.ivo.com/cancerdepielmelanomaycarcinoma.com
- javeriana. (15 de septiembre de 2017). *javeriana.edu.co*. Recuperado el 27 de septiembre de 2022, de repository.javeriana.edu.co:

<https://repository.javeriana.edu.co/bitstream/handle/10554/38692/Francisco%20Andr%c3%a9s%20Reales%20Castro.pdf?sequence=1&isAllowed=y>

Mamani, F. S. (2007). *Prediccion de rasgos faciales utilizando inteligencia artificial*. La Paz Bolivia: Tesis de grado. Recuperado el 14 de diciembre de 2021

mayoclinic. (5 de diciembre de 2020). *diseases-conditions*. Recuperado el 22 de agosto de 2022, de diseases-conditions/skin-cancer: <https://www.mayoclinic.org/es-es/diseases-conditions/skin-cancer/symptoms-causes/syc-20377605>

medicina y salud pública. (25 de enero de 2021). *www.medicinaysaludpublica.com*. Recuperado el 2022 de marzo de 18, de www.medicinaysaludpublica.com/melanoma: www.medicinaysaludpublica.com/melanoma

misistemainmune.es. (26 de noviembre de 2015). *papel-del-interferon-gamma-en-la-defensa-antiviral*. Recuperado el 21 de agosto de 2022, de <https://www.misistemainmune.es/>: <https://www.misistemainmune.es/inmunologia/componentes/papel-del-interferon-gamma-en-la-defensa-antiviral>

Molleda, J. (2008). *Tecnicas de vision por computador para la reconstruccion en tiempo real de la forma 3D de productos laminados. Tesis Doctoral*. Univercidad de Oviedo, Oviedo, España.

mql5. (17 de noviembre de 2017). *www.mql5.com*. Recuperado el 5 de agosto de 2022, de www.mql5.com/es/articles/3473: [https://www.mql5.com/es/articles/3473](http://www.mql5.com/es/articles/3473)

mskcc. (20 de enero de 2022). *cancer-care*. Recuperado el 15 de septiembre de 2022, de www.mskcc.org/es/cancer-care: <https://www.mskcc.org/es/cancer-care/types/melanoma/diagnosis>

newsnetwork. (7 de febrero de 2014). *mayoclinic.org*. Recuperado el 15 de marzo de 2022, de newsnetwork.mayoclinic.org: <https://newsnetwork.mayoclinic.org/discussion/estudio-descubre-drastico-aumento-en-cancer-de-piel-entre-las-personas-de-mediana-edad/>

oracle. (23 de agosto de 2019). *oracle.com*. Recuperado el 26 de agosto de 2022, de <https://www.oracle.com/>: <https://www.oracle.com/mx/artificial-intelligence/what-is-ai/>

quish.tv. (5 de enero de 2022). *es.quish.tv*. Recuperado el 2022 de abril de 27, de <https://es.quish.tv/>: <https://es.quish.tv/guide-tensorflow-keras-optimizers-analytics-india-magazine#:~:text=Los%20optimizadores%20son%20la%20clase,optimizadores%20de%20To%20Tensorflow%20Keras>

- RODRÍGUEZ, C. y DORADO R. (8 de septiembre de 2015). *RODRÍGUEZ, C. y DORADO ¿Por qué implementar Scrum?* Recuperado el 25 de febrero de 2022, de <https://journal.universidadean.edu.co/>: <https://journal.universidadean.edu.co/index.php/Revistao/article/view/1253/1218/>
- S. Arteaga. (15 de enero de 2017). *Sistema de deteccion de enfermedades de piel similar a un dermatologo*. Recuperado el 10 de diciembre de 2021, de S. Arteaga: <https://computerhoy.com/noticias/life/sistema-detecta-cancer-piel-precision-dermatologo-57598>
- scielo. (7 de septiembre de 2012). *ve.scielo.org*. Recuperado el 15 de agosto de 2022, de <https://ve.scielo.org/scielo/>: https://ve.scielo.org/scielo.php?script=sci_arttext&pid=S0016-35032012000300010
- scielo. (15 de enero de 2015). *scielo.isciii.es*. Recuperado el 3 de marzo de 2022, de scielo.isciii.es/clasificacion_dentro_red_neuronal: https://scielo.isciii.es/scielo.php?script=sci_arttext&pid=S1578-84232015000100003
- scielo. (4 de octubre de 2021). *scielo.org.mx*. Recuperado el 6 de octubre de 2022, de www.scielo.org.mx/scielo/: https://www.scielo.org.mx/scielo.php?pid=S1405-55462020000200751&script=sci_arttext&tlng=es#c1
- sitiobigdata. (25 de febrero de 2022). *sitiobigdata.com/*. Recuperado el 25 de abril de 2022, de <https://sitiobigdata.com/>: <https://sitiobigdata.com/2018/08/28/redes-neuronales-convolucionales-vista-rapida-implementacion/#>
- Sonka, M. (1993). *Image processing analysis and machine vision*. Reino Unido Londres: Champ and Hall.
- thelancet. (25 de julio de 2020). *gynaecological-cancers*. Recuperado el 25 de agosto de 2022, de <https://www.thelancet.com/>: <https://www.thelancet.com/series/gynaecological-cancers>
- themachinelearners. (27 de septiembre de 2018). *www.themachinelearners.com*. Recuperado el 3 de diciembre de 2022, de [curva-roc-vs-prec-recall/](https://www.themachinelearners.com/curva-roc-vs-prec-recall/): <https://www.themachinelearners.com/curva-roc-vs-prec-recall/>
- TICONA, R. C. (2020). *SISTEMA DE RECONOCIMIENTO DE IMÁGENES PARA LA IDENTIFICACIÓN DE LA ARTRITIS REUMATOIDE EN MANOS BASADO EN REDES NEURONALES CONVOLUCIONALES*. La Paz Bolivia: Tesis de Grado. Recuperado el 14 de diciembre de 2021

uc3m. (27 de septiembre de 2017). *e-archivo.uc3m.es*. Recuperado el 2 de octubre de 2022, de
bitstream/handle: [https://e-
archivo.uc3m.es/bitstream/handle/10016/27843/TFG_Paula_Fernandez_Blazquez.pdf?
sequence=1&isAllowed=y](https://e-archivo.uc3m.es/bitstream/handle/10016/27843/TFG_Paula_Fernandez_Blazquez.pdf?sequence=1&isAllowed=y)

usta. (17 de marzo de 2022). *usta.edu.co*. Recuperado el 5 de octubre de 2022, de
repository.usta.edu.co:
[https://repository.usta.edu.co/bitstream/handle/11634/45517/2022sebastianria%C3%
B1o.pdf?sequence=1](https://repository.usta.edu.co/bitstream/handle/11634/45517/2022sebastianria%C3%B1o.pdf?sequence=1)

VIU. (10 de Septiembre de 2019). *Reconocimiento de imágenes: software y ejemplos*.
Recuperado el 14 de diciembre de 2021, de Univercidad VIU:
<https://www.universidadviu.com/reconocimiento-de-imagenes-software-y-ejemplos>

La Paz, marzo de 2023

Señor

Ph.D. José María Tapia Baltazar

Director

Carrera de Informática

Facultad de Ciencias Puras y Naturales - UMSA

Presente

Ref. Aval de Conformidad del Perfil de Tesis de Grado

De mi consideración.

Tengo a bien dirigirme a su persona para darle a conocer, que luego de efectuar el seguimiento a la estructura y contenido del Perfil de Tesis de Grado, titulado “**DETECTOR DE CÁNCER DE PIEL A TRAVÉS DE UNA FOTOGRAFÍA UTILIZANDO VISIÓN POR COMPUTADORA**”, elaborado por el universitario MIGUEL FERNANDO APAZA COLQUE, con C.I. 5477212 LP, y habiendo el postulante realizado las respectivas correcciones a mis observaciones, y no existiendo impedimento alguno en la propuesta, me corresponde **dar mi conformidad y/o aval**, para que el postulante pueda realizar la defensa de la Tesis de Grado, para optar al título de Licenciado en Informática con mención en Ingeniería de Sistemas Informáticos, de acuerdo a normas y reglamentos vigentes.

Sin otro particular, me despido de usted con las consideraciones más distinguidas.

Atentamente.

M Sc. Reynaldo Javier Zeballos Daza

Docente Tutor